



Scalable Analytics on Multi-Streams Dynamic Graphs

Muhammad Ghufuran Khan

► To cite this version:

Muhammad Ghufuran Khan. Scalable Analytics on Multi-Streams Dynamic Graphs. Databases [cs.DB]. Institut Polytechnique de Paris, 2025. English. ⟨NNT : 2025IPPAX083⟩. ⟨tel-05351773⟩

HAL Id: tel-05351773

<https://theses.hal.science/tel-05351773v1>

Submitted on 6 Nov 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Scalable Analytics on Multi-Streams Dynamic Graphs

Thèse de doctorat de l'Institut Polytechnique de Paris
préparée à l'École polytechnique

École doctorale n°626 École doctorale de l'Institut Polytechnique de Paris (EDIPP)
Spécialité de doctorat: Informatique

Thèse présentée et soutenue à Palaiseau, le 18/09/2025, par

MUHAMMAD GHUFRAN KHAN

Composition du Jury :

Dario Colazzo Professor, Université Paris-Dauphine, France	Président, Rapporteur
George Fletcher Professor, Eindhoven University of Technology, Netherlands	Rapporteur
Angela Bonifati Professor, Lyon 1 University, France	Examinatrice
Stefania Dumbrava Associate Professor, ENSIIE – Télécom SudParis, France	Examinatrice
Silviu Maniu Professor, Université Grenoble Alpes, France	Examineur
Yannis Velegrakis Professor, Utrecht University, Netherlands	Examineur
Angelos-Christos Anadiotis Consulting Member Of Technical Staff, Oracle, Switzerland	Co-encadrant de thèse
Ioana Manolescu Inria Senior Researcher, Institut Polytechnique de Paris, France	Directrice de thèse

*To my father, whose wish has always been to see his children earn a PhD— this work is a
fulfillment of that dream.*

To my wife, for her unwavering support, patience, and strength throughout this journey.

*To my parents, brothers, and sister, for their love and encouragement in every phase of
my life.*

And to my son, whose smile gave me the strength to keep going.

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Ioana Manolescu, and co-supervisor Angelos Anadiotis, for their invaluable support, patience, and guidance throughout my PhD journey. Their mentorship and continuous encouragement played a crucial role in helping me accomplish this achievement. I sincerely thank the members of my thesis jury for their time, constructive feedback, and willingness to review my work. I am profoundly grateful to my parents, brothers, and sister for their unwavering support, understanding, and encouragement during this long and challenging period. A very special thank you goes to my wife for her unconditional support, especially during the most difficult phases of my PhD. Her strength, care for our son, and sacrifice allowed me to fully dedicate myself to this thesis. Finally, I would like to thank all the members of the CEDAR team for their insightful discussions during our team meetings.

Contents

Abstract	9
Résumé	10
1 Introduction	13
1.1 Motivation	13
1.2 Challenges and contributions	16
1.3 Publications	18
1.4 Manuscript organization	19
2 Graph data management overview	21
2.1 RDF graphs and the SPARQL language	21
2.2 Connectivity graphs and associated algorithms	25
2.3 Property graphs and associated query languages	27
2.3.1 Cypher	29
2.3.2 Gremlin	30
2.3.3 Property Graph Query Language (PGQL)	32
2.4 Classification of graph data management systems	35
2.5 Disk-based graph database systems	36
2.5.1 Disk based non-native graph data stores	37
2.5.2 Disk-based native graph data stores	38
2.6 Non-native data stores	40
2.6.1 Object-oriented databases	40
2.6.2 Key-value stores	41

2.6.3	Document-based stores	42
2.6.4	Relational database management systems	43
2.7	Summary	45
3	In-memory graph data management systems	46
3.1	Basic concepts	46
3.1.1	Edge list	47
3.1.2	Compressed sparse rows (CSR)	48
3.1.3	Adjacency list (AL)	49
3.2	In-memory data stores	49
3.2.1	LLAMA	51
3.2.2	GraphOne	52
3.2.3	STINGER	54
3.2.4	LiveGraph	55
3.2.5	Teseo	57
3.2.6	Sortledton	59
3.2.7	Spruce	60
3.2.8	Design comparison and discussion	61
3.3	Graph data management with updates propagated ooo	63
3.4	Summary	66
4	Core HAL Storage System Overview	67
4.1	Preliminaries	67
4.2	Storage system overview	68
4.3	The vertex array (VA)	70
4.4	The stream-time ordered adjacency list (STAL)	73
4.5	Insertion	76
4.5.1	In-order insertion	76
4.5.1.1	In-order insertion over existing UPI	78
4.5.1.2	In-order insertion over existing AR	79
4.5.1.3	In-order insertion after GC	80
4.5.2	Out-of-order insertion	80

4.5.2.1	Out-of-order insertion over existing UPI	81
4.5.2.2	Out-of-order insertion over existing AR	81
4.5.2.3	Out-of-order insertion after GC	84
4.6	Deletion	84
4.6.1	Garbage collection	86
4.6.2	Deletion of an in-order insertion	86
4.6.3	Deletion of an ooo insertion	87
4.7	HT _s and UPI details	87
4.7.1	Processing of in-order UPI	88
4.7.2	Processing of ooo UPI	90
4.8	Queries	91
4.9	Summary	92
5	Core HAL System Evaluation	94
5.1	Hardware and software settings	94
5.2	Workloads	94
5.3	Edge insertions	95
5.4	Updates (insertions and deletions)	98
5.5	Analytics	99
5.6	Concurrent read-write workload	101
5.7	Out-of-order insertions	102
5.8	Out-of-order updates	104
5.9	Summary	105
6	HAL Extensions: Historical and Property Graph Queries	106
6.1	Historical queries	106
6.2	Property-rich nodes and edges	111
6.3	Historical queries	113
6.4	Property graph queries	115
6.5	Interactive GUI	118
6.6	Summary	120

7 Conclusion and perspectives	122
7.1 Conclusion	122
7.2 Perspectives	123

List of Figures

1.1	Sample dynamic graph with out-of-order updates.	15
2.1	Example RDF graph: person, attributes, and relationships.	22
2.2	Connectivity graph example: connections in a social network	25
2.3	Sample property graph: scientific research collaboration network	28
2.4	Example of an adjacency list representation of a social network using a key-value store	41
2.5	Example of an edge list representation of a social network using a key- value store	41
2.6	Graph data represented in a document store: two vertex documents and one edge document.	43
2.7	Social network users represented as vertices.	44
2.8	Friendship connections represented as edges.	44
3.1	Edge list representation of the example graph shown in Figure 2.2 . . .	47
3.2	CSR representation of the graph in Figure 2.2	48
3.3	Adjacency list representation of the graph in Figure 2.2	49
3.4	Example graph used to illustrate internal data layouts of graph storage systems.	51
3.5	LLAMA data layout representation	51
3.6	GraphOne data layout representation	54
3.7	STINGER data layout representation	55
3.8	LiveGraph data layout representation	56

3.9	Illustration of the timestamp-based MVCC protocol used in LiveGraph. Adapted from LiveGraph [126], Figure 3.	57
3.10	Teseo data layout representation	58
3.11	Sortledton data layout representation	60
3.12	Illustration of the Hyper protocol showing versioned edge inserts in a dynamic graph. Adapted from Sortledton [41], Figure 6.	60
3.13	Spruce data layout representation	61
3.14	Sample dynamic graph with out-of-order updates.	64
4.1	Sample dynamic graph.	68
4.2	Sample vertex entry VA[0].	71
4.3	HT ₀ through insertions and deletions.	72
4.4	UPI fields and their bit spans.	89
4.5	Out-of-order UPI fields and their bit spans.	91
5.1	Graph datasets in our evaluation.	95
5.2	Graph500-24 scalability analysis.	95
5.3	Random order insertion throughput.	96
5.4	Ordered insertion throughput.	96
5.5	Throughput on updates workloads	97
5.6	Memory usage on Graph500-24 update workload.	98
5.7	Performance evaluation on graph analytics.	100
5.8	Throughput and query performance analysis using varying updates work- loads on read/write workloads.	102
5.9	Performance on in- and ooo insertions.	103
5.10	Performance on in- and ooo insertions.	103
6.1	Performance of historical queries at different moments in the update history.	113
6.2	Edge insertion time across SNB relationships.	115
6.3	Screen captures of HAL's interactive GUI.	121

List of Tables

2.1	Comparison of various graph database systems (native: native graph databases, ds: distributed storage, mem: In-memory support, tr: transactions support, mvcc: multi-version concurrency control protocol support)	36
3.1	Complexity comparison across systems	62
4.1	Possible cases of ooo insertion over existing AR.	81
5.1	Throughput variations when varying the swap percentage sw , on graph500-24 and graph500-22 workloads.	105
6.1	Estimated number of nodes and edges in LDBC SNB Interactive v1 datasets at different scale factors.	116

Abstract

Many real-time applications, including financial platforms, social networks, and transportation systems, rely on dynamic graphs constructed from multiple data streams. These streams are highly dynamic, and updates may be received out of order at the database, making it challenging to support: (i) scalable ingestion with concurrent analytics; (ii) accurate analytics under out-of-order (ooo) updates; (iii) efficient analytics on both present and historical graph snapshots; (iv) efficient storage and handling of structural and property graph updates.

This thesis introduces **HAL**, a novel *in-memory dynamic graph database engine* designed to address these challenges. HAL processes graph updates from multiple sources while supporting concurrent, consistent accurate analytics. At its core, HAL employs the *Stream-Time Ordered Adjacency List (STAL)*, a data structure that maintains updates in stream-time order, regardless of their arrival order. A lightweight, graph-aware multi-version concurrency control protocol ensures consistency during concurrent update ingestion and analytics. Experiments show that HAL achieves an ingestion rate of up to **7.5 million updates per second**, outperforming the best existing systems by **2.5×**, and provides up to **357×** speedups for analytical queries in the presence of ooo updates.

In the thesis, we first present the **core HAL data store system**, detailing its design principles, stream-time indexing, and concurrency control. We evaluate its ingestion and query performance under diverse workloads. Next, we describe two important extensions: support for **historical queries**, allowing users to reconstruct the graph at any past stream-time moment. Second, we enable **property-rich graph support** by allowing arbitrary properties on nodes and edges.

Résumé

De nombreuses applications en temps réel, telles que les plateformes de trading financier, les réseaux sociaux et les systèmes de transport intelligents, reposent sur des graphes dynamiques continuellement construits à partir de multiples sources de données en flux. Ces flux évoluent rapidement, et les mises à jour peuvent être reçues dans le désordre par la base de données, rendant difficile la prise en charge simultanée : (i) d'une ingestion évolutive avec des requêtes analytiques concurrentes ; (ii) d'analyses précises en présence de mises à jour reçues hors ordre (*out-of-order*, ooo) ; (iii) d'analyses efficaces sur des instantanés de graphes actuels et historiques ; et (iv) d'un stockage et d'une gestion efficaces des mises à jour structurelles et des propriétés.

Cette thèse présente HAL, un nouveau moteur de base de données de graphes dynamiques en mémoire conçu pour relever ces défis. HAL permet une ingestion et des analyses concurrentes tout en maintenant de fortes garanties de cohérence, même lorsque les mises à jour issues de flux multiples arrivent dans un ordre temporel désynchronisé. Au cœur de HAL se trouve la Stream-Time Ordered Adjacency List (STAL), une structure de données innovante qui conserve les mises à jour du graphe selon leur temps d'émission d'origine plutôt que leur ordre d'arrivée. Cette approche préserve l'évolution temporelle correcte du graphe et garantit des résultats analytiques précis.

HAL repose sur un protocole de contrôle de concurrence multi-version (MVCC) conscient de la structure du graphe, assurant l'isolation transactionnelle et la cohérence des versions lors des mises à jour et des lectures simultanées. Ce protocole exploite les dépendances structurelles du graphe afin de minimiser les coûts de synchronisa-

tion et d'atteindre une forte évolutivité parallèle. De plus, HAL intègre un mécanisme efficace de gestion mémoire et de compactage, qui maintient les performances et l'efficacité du stockage même sous des charges continues et à haute fréquence de mises à jour.

Des évaluations expérimentales approfondies, réalisées sur des jeux de données synthétiques et de référence, démontrent que HAL atteint jusqu'à 7,5 millions de mises à jour par seconde, dépassant les systèmes en mémoire les plus performants jusqu'à $73\times$, tout en maintenant une faible latence d'exécution des requêtes. Pour les charges analytiques affectées par des mises à jour hors ordre, HAL offre jusqu'à $357\times$ d'accélération de l'exécution des requêtes, démontrant que la cohérence temporelle et la concurrence peuvent coexister efficacement sans compromettre les performances.

Au-delà de son noyau de stockage, la thèse introduit deux extensions majeures du système. La première prend en charge les requêtes historiques, permettant aux utilisateurs de reconstruire l'état exact du graphe à tout instant passé du temps de flux, fonctionnalité essentielle pour le débogage, l'audit et les analyses temporelles. La seconde étend HAL aux graphes riches en propriétés, autorisant l'ajout de paires clé-valeur arbitraires sur les nœuds et les arêtes et permettant des requêtes analytiques combinant propriétés et parcours topologique. L'architecture de HAL sépare le stockage structurel du stockage des propriétés afin d'optimiser les accès hétérogènes et d'améliorer la montée en charge et les performances sous des charges mixtes.

Une interface de visualisation interactive accompagne HAL, démontrant sa capacité à gérer les mises à jour hors ordre et à illustrer comment une mauvaise gestion temporelle dans d'autres systèmes peut conduire à des résultats analytiques inexacts. Cet outil interactif met en évidence l'impact pratique et la robustesse de la conception de HAL pour les applications en temps réel et multi-flux.

En résumé, cette thèse fait progresser l'état de l'art en gestion des graphes dynamiques en introduisant un nouveau moteur de stockage en mémoire optimisé pour les graphes dynamiques multi-flux et hors ordre. Elle propose un mécanisme d'ordonnancement basé sur le temps de flux (STAL) assurant une cohérence temporelle précise, ainsi qu'un protocole MVCC conscient du graphe permettant les mises

à jour transactionnelles et les requêtes concurrentes. En outre, elle intègre des extensions efficaces pour les graphes historiques et riches en propriétés, garantissant la scalabilité et la cohérence sous des charges de travail variées. À travers une validation expérimentale approfondie, HAL démontre de meilleures performances, une meilleure évolutivité et une plus grande précision que les systèmes existants, établissant ainsi une base solide pour la prochaine génération de bases de données de graphes dynamiques capables de gérer la complexité, la concurrence et le désordre temporel inhérents aux flux de données modernes en temps réel.

Chapter 1

Introduction

We start by outlining the motivation for our work in Section 1.1. We then list the challenges raised and our contribution in Section 1.2, before presenting the publications resulting from this thesis in Section 1.3 and the manuscript organization in Section 1.4.

1.1 Motivation

In the modern digital landscape, vast volumes of data are continuously generated in the form of high-velocity **streams** [39]. These streams originate from diverse sources such as sensor networks [65], social networks [126], transportation systems [66], etc. Streaming data often captures **interactions and dependencies** between various real-world entities, such as users in a social network, devices in an IoT environment, or virtual machines in a data center. Modeling and analyzing these interconnections is essential for understanding system behavior and supporting informed decision-making in real time.

These real-time interactions naturally form a **graph** structure, where entities are represented as nodes and their connections as edges. In many real-time applications, such as IoT networks [51], intelligent transportation systems [70], and computer networks [126], these connections are highly **dynamic**. Nodes and edges may appear and disappear rapidly due to mobility, failures, routing, or environmental conditions. Such evolving relationships give rise to dynamic graphs, where both the topology and

associated properties of the graph change continuously over time. The growing need to model and analyze dynamic graphs has led to the emergence of graph data management systems [99]. Prominent examples include Neo4j [96], Oracle Spatial and Graph [86], and ArangoDB [8] which provide persistent storage, support for property graphs, and declarative query languages. However, these systems are disk-resident, making them relatively slow for supporting high write throughput and low-latency queries, as disk access is inherently slower than RAM. Recent advancements in RAM capacity have motivated research to push further by proposing native, in-memory graph database systems [126, 41, 64, 102, 76, 58, 38], which aim to significantly improve write throughput and reduce query latency for dynamic workloads.

This thesis focuses on dynamic graph workloads, particularly in settings *where edges updates are continuously added and deleted to a single graph, from multiple update streams*. The dynamic graphs are stored in a transactional graph database. Each edge update or deletion carries a *source (stream) time* S_T , assigned at the moment when it was emitted, and an *arrival (or transaction) time* W_T , assigned when the graph database receives it. We assume that *all the stream timelines are reconcilable*, that is: a global order can be established over any set of updates coming from multiple streams. This can be achieved via well-known distributed systems techniques, e.g., [61]. Furthermore, updates may be received at the database **out-of-order** [45] (**ooo**, in short): due to different latencies on the propagation paths between the data source and the database, there may be two edge operations (insertion and/or deletion) u_1, u_2 , issued at stream times $S_T^1 < S_T^2$, concerning the same or different edges, such that u_2 is received at the database at a time W_T^2 when u_1 has not yet been received. Therefore, ooo updates are ordered differently based on source time (S_T) and on database time (W_T).

Such scenarii are frequent in IoT applications [65, 37], where changes in the network topology cause edges to appear and disappear rapidly. In IoT networks [51], issues such as multi-path routing [124, 71], route fluctuations [18], link-layer retransmission [40, 62], and router forwarding [75] may lead to ooo updates. For instance, in Intelligent Transportation Systems, two main scenarii can occur:

1. Nodes (sensors) are fixed, e.g., intersections, while edges reflect route seg-

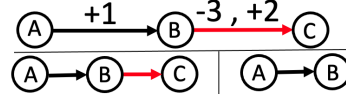


Figure 1.1: Sample dynamic graph with out-of-order updates.

ments whose status may vary in real-time due to anomalies such as traffic jams, accidents, road closures, etc. [66, 125, 70, 29, 44, 100]. If a sensor sends an update indicating the opening of a route, followed by another update for its closure, receiving these ooo may lead to misrepresent the current traffic situation.

2. Nodes move, e.g., Unmanned Aerial Vehicle (UAVs); edges denote communication links between them [97]. As UAVs move, these links appear and disappear rapidly. This dynamic behavior is crucial for security surveillance and monitoring, agricultural field inspection [21, 117, 53, 77], etc. If an edge denoting the connection between two UAVs is created then removed, and the updates are received ooo, again path planning may be impacted.

ooo updates—where insertions and deletions may arrive at the graph database in a different order than they were generated—occur frequently in real-world settings. These situations require careful interpretation of updates to maintain an accurate graph state. Existing systems often ignore such disorder, leading to incorrect final graphs. Figure 1.1 shows a simple example illustrating the impact of ooo updates. At the top, each edge is annotated with an insertion (+) and/or a deletion (-) operation, along with its stream time (S_T), shown in the order of arrival at the database. For example, the edge $B \rightarrow C$ receives a deletion emitted at time 3 before the insertion emitted at time 2. Existing systems, such as [64, 126, 41], assume that updates are in-order and process them as they arrive. As a result, such early deletions are ignored and the edge $B \rightarrow C$ incorrectly remains in the graph. Red edges mark those affected by ooo propagation. Without handling such cases, the system may return incorrect query results. As illustrated at the bottom, the incorrect graph state (left) differs from the correct one (right) in the highlighted red edges.

1.2 Challenges and contributions

As demonstrated by the motivating scenarios, data stores managing dynamic graphs must balance the tradeoff between high-throughput transactional writes from multiple data streams and accurate analytical queries in parallel—even in the presence of ooo updates. This thesis addresses four challenges that are fundamental to advancing the capabilities of dynamic graph data management.

- **C1: Scalable ingestion and concurrent analytics.** Supporting high-throughput ingestion from multiple data streams while enabling consistent and concurrent analytical queries remains a central challenge for dynamic graph systems.
- **C2: Present and historical snapshot query processing.** Efficiently supporting both current and historical graph snapshots, based on stream time S_T , requires careful storage layout and indexing strategies.
- **C3: Accurate analytics under ooo update arrivals.** The system must preserve the accuracy of analytical queries, even when updates are received ooo.
- **C4: Storing structural and property updates efficiently.** Dynamic graph stores should support two fundamentally different access patterns: structural updates, which drive graph traversal and analytical algorithms; and property updates, which support attribute-based filtering. These workloads differ significantly in data locality and access paths. To support both efficiently, the data store should decouple the storage of topological graph updates from the storage of graph property updates. Keeping them separate allows for the independent optimization of data access paths for these two largely orthogonal components of the property graph.

Thesis contributions. To address the aforementioned challenges, this thesis makes the following key contributions to the design and implementation of dynamic graph database systems:

- **A novel in-memory graph store, History Adjacency List (HAL), for dynamic multi-stream updates.** We propose HAL, a native in-memory graph data store

designed to balance high-throughput transactional writes with efficient and accurate analytical scans in parallel, addressing the trade-offs between update performance and query consistency on dynamic graphs.

- **The Stream-Time Ordered Adjacency List (STAL).** We propose the STAL data structure, a stream-time-sorted adjacency list that stores edge updates in the order of their stream time (S_T), regardless of their arrival order (W_T). STAL is built on top of a lightweight, graph-aware multi-version concurrency control (MVCC) protocol [119], enabling high-throughput ingestion and consistent analytical queries to run in parallel, even under ooo update scenarios.
- **Accurate analytical query execution under ooo updates.** We propose a novel data structure which we use to identify the most likely insertion-deletion operation pairs over the same edge, and integrate it fully within our store, even after garbage collection. This mechanism enables the system to reconstruct correct graph states over time and ensures accurate query results, even in the presence of ooo updates.
- **Decoupling of structural and property graph updates.** HAL decouples topological (structural) updates from property updates at the storage layer. This design allows independent optimization of access paths for graph traversal and attribute-based filtering, improving performance for both structural analytics and property-centric queries.
- **Support for both present and historical state queries.**

HAL’s data storage layout is optimized to efficiently support both present graph state queries and historical queries based on stream time (S_T).

- **Comprehensive experimental evaluation.** We conduct an extensive performance study over synthetic and benchmark workloads, comparing HAL with state-of-the-art systems. Results show that HAL achieves up to $73\times$ higher ingestion throughput and up to $357\times$ faster analytics, particularly under ooo update scenarios.

- We develop a visual and interactive GUI that illustrates HAL’s handling of ooo updates, highlighting the impact on analytical query accuracy when such updates are not correctly supported by the system.

1.3 Publications

The work presented in this thesis has been disseminated through national and international conferences publications. The following list includes five papers co-authored during my PhD research, presented in (most to least recent).

These publications reflect the contributions of this thesis: the core HAL system, its extension to support historical and property graph queries, and their practical demonstration.

Core HAL system design and evaluation (Chapters 4, 5):

3. Angelos Christos Anadiotis, Muhammad Ghufra Khan, and Ioana Manolescu. *Dynamic Graph Databases with Out-of-Order Updates*. *Proceedings of the VLDB Endowment*, Volume 17, Issue 13, pages 4799–4812, 2024.
4. Angelos Christos Anadiotis, Muhammad Ghufra Khan, and Ioana Manolescu. *Scalable Analytics on Multi-Streams Dynamic Graphs*. In *BDA 2023 – 39ème Conférence sur la Gestion de Données – Principes, Technologies et Applications*, October 2023, Montpellier, France. hal-04212814.
[Informal presentation of initial HAL design and performance results.]
5. Muhammad Ghufra Khan. *Scalable Analytics on Multi-Streams Dynamic Graphs*. In *BDA 2022 – 38ème Conférence sur la Gestion de Données – Principes, Technologies et Applications*, October 2022, Clermont-Ferrand, France. hal-03903287.
[PhD proposal paper presenting the research problem and system vision.]

System extensions and interactive GUI (Chapters 6):

1. Angelos Christos Anadiotis, Muhammad Ghufra Khan, and Ioana Manolescu. *Growing Up HAL: Historic and Property Graph Queries*. In *LSGDA 2025 – 4th International Workshop on Large Scale Graph Data Analytics*, London, UK, in conjunction with VLDB 2025.

2. Angelos Christos Anadiotis, Muhammad Ghufuran Khan, and Ioana Manolescu. *Catching up with Disorder: Dynamic Graphs with Out-of-Order Updates* (Demo Paper). In *Companion of the 2025 International Conference on Management of Data*, pp. 15–18.

This thesis consolidates the contributions from the above publications. In particular, the core HAL system is discussed in Chapters (4, 5), while its extensions are presented in Chapter 6. Informal feedback from the BDA (*Bases de Données Avancées* - the informal French database association) community helped refine system design decisions in the earlier stages of this research.

1.4 Manuscript organization

This thesis is structured into seven chapters, grouped into three main parts: (i) background and related work, (ii) core system design and evaluation, and (iii) extensions and conclusions.

- **Chapter 2 - Graph data management overview:** This chapter provides a general overview of graph data models and graph database systems, with an emphasis on disk-based systems, both native and non-native. It introduces RDF, connectivity, and property graphs. The chapter also includes a classification of systems based on storage model, transactional guarantees, and concurrency mechanisms. This background sets the context for understanding the broader data management landscape within which this thesis is framed..
- **Chapter 3 - In-Memory graph data management systems:** This chapter narrows the focus to in-memory graph database systems, which are more directly related to the technical space addressed in this thesis. It surveys representative systems such as LLAMA [76], STINGER [38], GraphOne [58], LiveGraph [126], Teseo [64], Sortledton [41], and Spruce [102], comparing their architectural designs, handling updates, and concurrency control approaches. This comparative analysis helps the reader better understand the motivation behind the thesis work, by highlighting the gaps and limitations that this research aims to address.

- **Chapter 4 - Core storage system design:** This chapter introduces the core technical contribution of the thesis: a novel native in-memory graph storage engine designed for multi-stream dynamic graph workloads. It presents the internal architecture, including the Stream-Time Ordered Adjacency List (STAL) and the Vertex Array (VA), and describes how the system supports both in-order and ooo updates. The chapter also details the concurrency control mechanism, garbage collection, and insertion-deletion logic used to maintain correctness and efficiency under concurrent operations.
- **Chapter 5 - Core storage system evaluation:** This chapter presents an empirical performance evaluation of the proposed system using synthetic, and real-world datasets. It analyzes ingestion throughput, query performance, and system behavior under concurrent workload and ooo updates workload. Comparisons with state-of-the-art systems demonstrate the effectiveness and scalability of the thesis work.
- **Chapter 6 - Extensions to the core storage system:** This chapter extends the core design to support two advanced capabilities: (i) historical graph queries, enabling users to reconstruct past graph states using stream-time semantics; and (ii) property-rich graph support, allowing the storage and querying of arbitrary attributes on nodes and edges. A visual demonstration scenario is also presented to showcase the impact of ooo updates on query accuracy and to illustrate how the system correctly handles such scenarios in an interactive setting.
- **Chapter 7: Conclusion and future directions:** The final chapter summarizes the key contributions of the thesis and outlines several directions for future research.

Chapter 2

Graph data management overview

In this chapter, we trace the history of graphs from a data management perspective, describing various models that have been developed to represent graphs. These include: native graph models such as Resource Description Framework (RDF) graphs (Section 2.1), connectivity graphs (Section 2.2), and property graphs (Section 2.3). We then present a classification of graph database systems, distinguishing between native and non-native graph stores (Section 2.4), and we outline non-native stores in Section 2.6. The contributions of this thesis concern the design and development of a native in-memory store; thus, we discuss comparable native stores, that are a useful basis for comparison, in the next chapter (Related Work).

2.1 RDF graphs and the SPARQL language

The Resource Description Framework (RDF) is a standardized data model developed by the World Wide Web Consortium (W3C) [115] for structuring and exchanging information across different formats. The first RDF specification dates from 1999 [122], while its latest version (RDF 1.1) was published in 2014 [121]. As per the RDF specification [115], RDF is structured around three pairwise disjoint sets:

- *International resource identifiers* (IRIs) serve as unique identifiers for resources, enabling them to be uniformly referenced across RDF datasets.

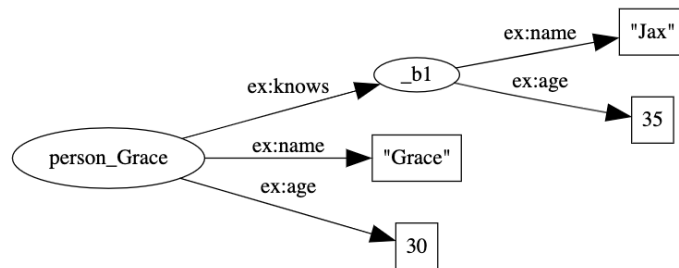


Figure 2.1: Example RDF graph: person, attributes, and relationships.

- *Literals* are data values; by default interpreted to be of type String, they can also be explicitly typed as Strings, numbers, dates, or a variety of other basic types [121].
- *Blank nodes* designate resources for which an IRI is not known. Syntactically, a blank node name starts with `_`, e.g., `_b1`; is an ID that is only unique within the context of the enclosing dataset, but carries no meaning (and is not guaranteed to be unique) outside of it. In contrast, an IRI unambiguously designates the same resource, in any dataset.

An RDF triple consists of three components: subject, predicate, and object. The subject can be identified by either an IRI or a blank node. The predicate is an IRI, while the object can take the form of an IRI, a blank node, or a literal value. A set of RDF triples can be seen as a graph, in which the subject and object are nodes, while the predicate acts as the directed edge connecting these nodes, representing the relationship between them. As shown in Figure 2.1, the RDF graph represents the relationship between two person (Grace and Jax) and their attributes. The primary subject node in this graph is `<http://example.org/person_Grace>`, which is identified by a IRI. This node is connected to two object nodes via directed edges: `ex:name` and `ex:age`. The `ex:name` edge links the subject node to a literal object with the value "Grace", while the `ex:age` edge connects it to another literal object with the value "30". Furthermore, the graph shows a relationship between `<http://example.org/person/Grace>` and another node through the predicate `ex:knows`. This predicate links the subject node to an object node (`_b1`, the blank node). The `_b1` node is further connected to two object nodes: `ex:name`, which links it to the literal "Jax", and `ex:age`, which connects

it to the literal “35”.

RDF graphs, also referred to as Knowledge Graphs (KGs) or Knowledge Bases (KBs), can be queried using SPARQL [113]. SPARQL, recognized as the *de facto* W3C-standard query language for RDF graphs, was initially introduced as *SPARQL 1.0* in 2008 and subsequently expanded to *SPARQL 1.1* in 2013 [113, 114]. The extended language incorporated property paths, allowing to check for the existence of graphs paths whose edge labels match certain regular expressions. It also introduces aggregates, sub-queries, an update language, and standardized result serialization formats. Evaluation is formally defined as *sub-graph matching*, where each triple pattern in the query must homomorphically map to triples in the data, producing a set of variable bindings that satisfy all required patterns. The language offers four query forms, namely SELECT, CONSTRUCT, ASK, and DESCRIBE, and supports optional matches, unions, filters, and ordered output. Result sets can be streamed in XML, JSON, CSV, or TSV formats [114].

Using the graph in Figure 2.1, the query in Listing 2.1 returns the name (and, when present, the age) of every person known by Grace.

```
1 PREFIX ex: <http://example.org/>
2
3 SELECT ?friendName ?friendAge
4 WHERE {
5     <http://example.org/person_Grace> ex:knows ?x .
6     ?x ex:name ?friendName .
7     OPTIONAL { ?b1 ex:age ?friendAge . }
8 }
```

Listing 2.1: SPARQL query to retrieve the names (and ages) of persons known by Grace

Here, the WHERE clause specifies two mandatory triple patterns: one that binds the intermediate node ?x via the ex:knows edge from person_Grace, and another that captures the name of the friend through the ex:name predicate. An OPTIONAL clause is used to retrieve the friend’s age. A SPARQL engine returns only those bindings that satisfy the mandatory patterns, while augmenting them with any available

optional data. In the case of the graph in Figure 2.1, the query yields a single result: `?friendName = "Jax"` and `?friendAge = 35`.

Major RDF databases, capable of storing RDF graphs and supporting the SPARQL query language, include Virtuoso, AllegroGraph, and StarDog. RDF is extensively employed to represent diverse datasets, including domain-specific knowledge graphs such as ESKG, NASA’s Earth Science Knowledge Graph, and a wide array of collections catalogued in the Linked Open Data (LOD) Cloud. Prominent examples include:

- **DrugBank Knowledge Graph (Bio2RDF release)** — represents comprehensive biochemical and pharmacological information on a wide range of experimental drugs. Its RDF representation, made available through the Bio2RDF project, is accessible via a public SPARQL endpoint [26].
- **UniProt Knowledge Graph** — encodes extensive protein-related information. The data can be queried via a public SPARQL endpoint [16].
- **LinkedGeoData** — converts OpenStreetMap data into an RDF knowledge graph consisting of hundreds of millions of spatial entities, enriched with interlinks to other datasets in the LOD Cloud [16].

RDF has also been used to describe information in large-scale, many-topic information repositories such as Yago [92], DBPedia [15], and WikiData [112]. . These are large, extensive sources of general knowledge rather than being confined to a single topic; they are constructed by combining information extracted from many sources and in particular Web sources [118].

RDF graphs can be exploited in conjunction with *ontologies*. An ontology is a set of statements (or axioms) about the classes and properties that may occur in an RDF graph; it allows to capture application-specific knowledge that users have about a domain. Ontologies can be expressed in a variety of ontology languages, with varied expressive power. The most commonly used are RDF Schema [116] and OWL (Web Ontology Language), closely related to Description Logic dialects [13]. In the presence of ontologies, RDF graphs contain, beyond the explicit triples (edges), also some *implicit* ones, that are not given in the graph but hold thanks to the ontology. For instance, if an ontology states that when x knows y , then x is a Human and so is y , a query that asks for all Humans can be answered with Grace and Jax in the

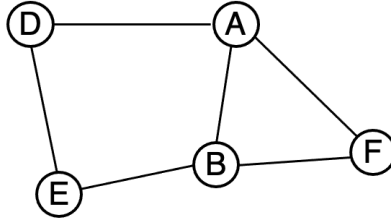


Figure 2.2: Connectivity graph example: connections in a social network

above example. Had the ontology been absent, this query may have had an empty answer, because no resource was explicitly declared of type Human. The efficient processing of queries in the presence of ontologies has been studied, e.g., in [42, 25]. In the remainder of this thesis, we do not consider RDF graphs or ontologies further.

2.2 Connectivity graphs and associated algorithms

Connectivity graphs are a very simple type of graphs, with unlabeled edges, and with nodes typically endowed only with an ID. Formally, we define a connectivity graph as an undirected or directed graph $G = (V, E)$ where:

- V is a finite set of **vertices** representing the entities in the system.
- $E \subseteq V \times V$ is a set of **edges** representing the relationships (connections) between pairs of vertices.

Connectivity graphs are widely utilized in various applications [126, 66, 125], particularly in social networks, to explore user relationships and recommend potential connections. As shown in Figure 2.2, each node represents an individual user, while the edges illustrate existing friendships between them. On a social network connectivity graph, for instance, it is possible to identify mutual friends shared by two users; in Figure 2.2, vertices **A** and **B** both share vertex F as a common friend.

In practice, connectivity graphs are processed using topology-driven procedures that perform iterative scans over the entire graph or relevant subgraphs, either for a fixed number of iterations or until convergence [34]. Such iterative techniques support applications including recommendation engines, shortest-path discovery, and other forms of topology-focused graph analysis. The most common workloads are as follows:

- **Triangle counting:** Triangle counting involves measuring the number of closed triplets—sets of three vertices that are all mutually connected—in a graph. For example, in the graph shown in Figure 2.2, nodes A, B, and F form a triangle, indicating a strong social cohesion. This metric is widely used in social network analysis, fraud detection, and recommendation systems [91]. Triangle counting across the graph helps identify tightly connected groups, which can be used for friend-of-a-friend recommendation. For instance, nodes A and E are not directly connected, but both are connected to B (and E is also connected to D, who is connected to A), suggesting that A and E share multiple common neighbors. A triangle-counting-based recommendation algorithm could use this information to suggest E as a potential friend to A (or vice versa), especially if adding the edge A–E would create one or more new triangles, such as A–B–E.
- **PageRank:** PageRank (PR) is an iterative algorithm that assigns an importance score to each vertex by repeatedly propagating influence through the graph structure. Originally developed to rank web pages [89] in the Google search engine, PageRank is widely used in social network analysis to identify highly influential users. The core idea is that a node is considered important if it is connected to other important nodes. For example, in the graph shown in Figure 2.2, nodes B and A are likely to receive higher PageRank scores because they are connected to multiple other nodes, including each other.
- **Weakly Connected Components:** The Weakly Connected Components (WCC) algorithm [50] identifies groups of nodes in a graph such that each node in a group is reachable from any other node when the graph is treated as undirected—that is, the direction of edges is ignored, even in directed graphs. WCC are typically identified in the early stages of graph analysis to reveal the overall structure and to isolate disjoint subgraphs. In Figure 2.2, all nodes $\{A, B, D, E, F\}$ are part of a single weakly connected component, since a path exists between any two nodes.
- **Breadth-first search (BFS):** In Breadth-First Search (BFS) [50], each vertex in a graph is assigned a level that corresponds to the length of the shortest path

from a given source vertex, called root. The traversal begins at the root with depth 0, and continues by visiting vertices in increasing order of distance: those one edge away are labeled depth 1, followed by depth 2 and so forth. BFS is commonly used to compute the shortest paths in connectivity graphs, as it guarantees a minimal number of hops are traversed from a source vertex to each reachable node. When applied to the graph in Figure 2.2 with A as the source, the algorithm places vertices B , D , and F at depth 1, since each is directly connected to A . Vertex E appears at depth 2, reachable via either $A \rightarrow D \rightarrow E$ or $A \rightarrow B \rightarrow E$.

Unlike traditional OLAP queries, such graph workloads are implemented in systems-level languages such as C++ or Java, and rely on the low-level APIs provided by the underlying graph processing frameworks [34].

2.3 Property graphs and associated query languages

The term **property graph** is used across various publications, but there is no universally accepted and standardized definition. In this thesis, we adopt the definition provided in [4]. Formally, a property graph is defined as a tuple $G = (V, E, \lambda, \phi)$ where:

- V is a finite set of **vertices** (nodes).
- $E \subseteq V \times V$ is a finite set of **directed edges** (relationships).
- $\lambda : V \cup E \rightarrow \mathcal{L}$ is a **labeling function** that assigns a single label from the label set $\mathcal{L}$ to each vertex and edge.
- $\phi : (V \cup E) \times \mathbf{P} \rightarrow \text{Val}$, where $\mathbf{P}$ is a finite set of property keys and **Val** is the set of atomic values. For any vertex $v \in V$ and property key $p \in \mathbf{P}$, the expression $\phi(v, p) = a$ means that $a \in \text{Val}$ is the value assigned to property p on the vertex v . Similarly, for an edge $e \in E$, $\phi(e, p) = a$ indicates that a is the value assigned to property p on the edge e in the property graph G .

Unlike connectivity graphs, which only capture the structural relationships between entities, property graphs store, beyond these connections, properties attached to vertices and edges. This supports more analyses across various applications, including social networks [126], fraud detection [126], and intelligent transportation

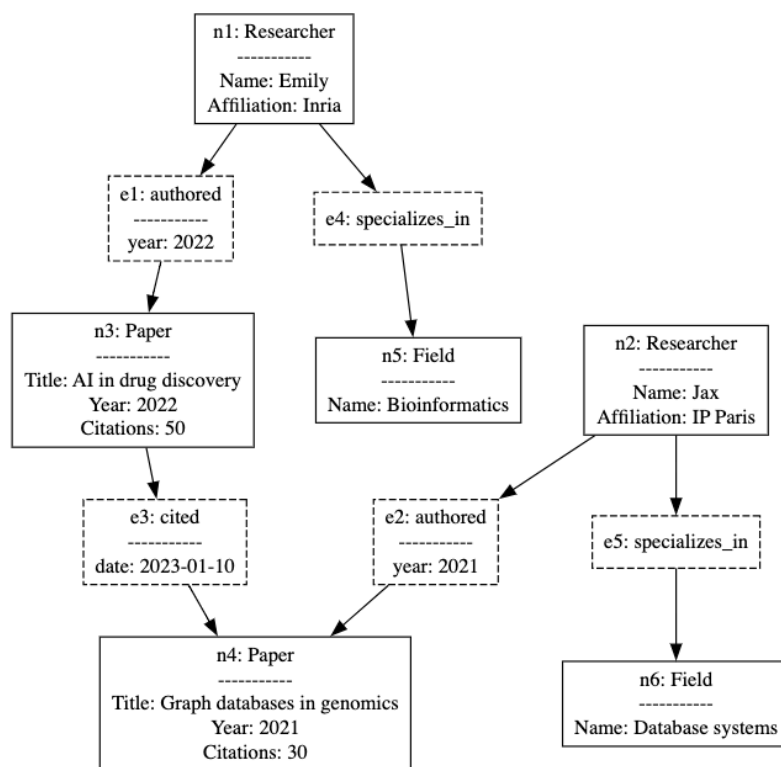


Figure 2.3: Sample property graph: scientific research collaboration network

systems [70] among others. For example, Figure 2.3 shows a property graph illustrating the collaboration between researchers and their contribution to scientific knowledge. In this graph, researchers are depicted as nodes with attributes such as name and affiliation, whereas their research papers form another set of nodes linked through authorship and citation relationships. For example, Emily, a researcher from Inria specializing in bioinformatics, authored a paper titled *“AI in drug discovery”* in 2022, which has received 50 citations. This paper cites a prior work, titled *“Graph databases in genomics”*, published in 2021 with 30 citations, authored by Jax, a researcher from IP Paris specializing in database systems.

Querying and processing PGs Property graphs can be queried using a variety of graph query languages that support operations such as pattern matching, traversal, and filtering over nodes and edges. These include Cypher [49], Gremlin [47], PGQL [109], G-Core [64], SQL/PGQ [35], GQL [35], AQL [8], and TSL [101]—which are tailored to specific use cases and differ in their syntax and execution models. In the following subsections, we examine three widely adopted graph query languages: Cypher (Section 2.3.1), Gremlin (Section 2.3.2) and PGQL (Section 2.3.3).

2.3.1 Cypher

Cypher is a declarative query language originally developed by Neo4j to facilitate the expression of complex graph queries using a syntax influenced by SQL [96]. Although it was initially proprietary to Neo4j, Cypher has since been adopted by various other graph database systems [49, 60, 74, 90]. The query language provides declarative constructs for complex graph traversal, search, joins and more. A Cypher query typically begins with a pattern-matching clause that describes the subgraph structures to locate. The query results are returned as a table-like collection of variable bindings, representing mappings between query variables and elements in the graph. These results can be further refined using relational-style operators. Final outputs are often serialized into a hierarchical format, such as JSON, making them readily compatible with downstream systems and applications [34].

```
1 MATCH (r:Researcher)-[:specializes_in]->(f:Field {name: "
```

```

    Bioinformatics"})),
2      (r)-[:authored {year: 2022}]->(p:Paper)
3  RETURN r.name, p.title

```

Listing 2.2: Cypher query to find researchers who specialize in Bioinformatics and authored a paper in 2022

As an example, consider the property graph shown in Figure 2.3, where entities such as researchers, papers, and fields are modeled as nodes, and relationships like *authored*, *cited*, and *specializes_in* connect them. To retrieve the names of researchers who specialize in the field of “*Bioinformatics*” and have authored a paper in the year 2022, the Cypher query shown in Listing 2.2 can be used. This query identifies any researcher node *r* that is connected via a *specializes_in* relationship to the field node *Bioinformatics*, and via an *authored* relationship (with the year 2022) to a paper node *p*. It then returns the researcher’s name and the title of the paper. Based on the structure depicted in Figure 2.3, this query would yield the result *Emily*, “*AI in drug discovery*”.

In addition to basic pattern matching, which can be seen as not very different from conjunctive select-project-join queries, Cypher supports regular path expressions (RPE) queries, which go beyond in that they can be matched by paths of arbitrary length in the graph. RPEs are specified using a Kleene-star syntax, e.g., an RPE labeled *[REL*]* matches paths of arbitrary length composed of edges solely labeled *REL*. RPEs enable checking for connectivity among nodes along certain paths, irrespective of the path length; this is a useful feature in graph analysis. However, Cypher’s RPE support is limited compared to more expressive languages like PGQL [109] (covered below), as it only supports Kleene-star and concatenation [12].

2.3.2 Gremlin

Gremlin [47] is another graph query language designed to support both graph traversals and pattern matching. Unlike declarative query languages that define a standalone syntax, Gremlin is implemented as a set of composable traversal steps, embedded within general-purpose host languages such as Groovy, or Python. Gremlin

supports two execution models: OLTP (Online Transaction Processing), designed for low-latency operations such as local graph traversals, lookups, and small updates; and OLAP (Online Analytical Processing), which enables batch analytics over the entire graph. However, OLAP support remains relatively limited across vendors [34]. From this point onward, we refer to Gremlin OLTP as the default execution model.

Central to Gremlin is the concept of the *traverser*—an abstraction that represents a pointer to a graph element, such as a vertex, edge, or property, along with a local computational state. Gremlin provides over 80 modular operations, called steps, which operate over sets of traversers. These steps enable users to express complex graph operations, such as navigating across vertex connections, filtering based on attributes, aggregating or deduplicating results, and mutating the graph structure. Queries are written in a chained, pipeline-like format that can be both compact and expressive, though this low-level style may sometimes reduce readability compared to more declarative languages like Cypher [34].

One common pattern in Gremlin queries involves maintaining intermediate results—such as collecting a list of direct friends to exclude from a recommendation set—by using constructs like arrays or sets. The ‘dedup()’ step, for instance, serves a role analogous to SQL’s ‘DISTINCT’ keyword by eliminating duplicate traversers. Compared to Cypher, the processing model of Gremlin is more imperative in that processing steps are explicitly chained. However, techniques to optimize Gremlin programs have been proposed in prior research [4, 120], exploiting the equivalences that can be shown to hold between different Gremlin queries (or programs).

```
1 g.V().has('Researcher', 'name', 'Emily') // Step 1: Find
    researcher named Emily
2 .out('authored') // Step 2: Get papers she authored
3 .has('title', 'AI in drug discovery') // Step 3: Filter for
    specific paper
4 .out('cited') // Step 4: Follow cited edges
5 .values('title') // Step 5: Get title(s) of cited paper(s)
```

Listing 2.3: Gremlin traversal to find titles of papers cited by Emily’s 2022 paper “*AI in drug discovery*”

For example, in Figure 2.3, to retrieve the title of the paper cited by Emily’s 2022 publication *‘AI in drug discovery’*, one can perform a directed traversal starting from the researcher node labeled with name Emily. The query first follows the authored edge to reach papers she wrote, filters for the one titled *‘AI in drug discovery’*, and then traverses the cited edge to find the referenced paper. Finally, it extracts the title property of the cited work, as shown in Listing 2.3. Based on the Figure 2.3 example, this traversal returns the publication *“Graph databases in genomics”*.

Among the various databases that support Gremlin, notable practical systems include OrientDB [27], Neo4j [96], and JanusGraph [52]. Gremlin does support recursive traversals [4], which it can express imperatively, using the steps repeat(), emit(), and until(). However, Gremlin lacks declarative regular path expressions (RPEs) for expressing navigational queries with regex-like syntax over edge labels.

2.3.3 Property Graph Query Language (PGQL)

The **Property Graph Query Language** (PGQL) is a declarative language designed for querying property graphs. It combines expressive graph-pattern matching with a familiar SQL-like syntax, allowing users to formulate queries using constructs such as SELECT, FROM, and WHERE, while also supporting vertex and edge pattern matching, as well as *regular path queries*. Query results are returned as a tabular set of variable bindings, facilitating integration within SQL-based workflows [109].

```
1 SELECT r.name, p.title
2 FROM GRAPH_TABLE(propGraph
3     MATCH (r IS Researcher)-[:specializes_in]->(f IS Field {name =
4         'Bioinformatics'}),
5         (r)-[a IS authored]->(p IS Paper)
6     WHERE a.year = 2022
7 )
```

Listing 2.4: PGQL query to find researchers who specialize in Bioinformatics and authored a paper in 2022

In Figure 2.3, the query shown in Listing 2.4 returns the name of each researcher (r.name) and the title of each paper (p.title) they authored in the year 2022, under

the condition that the researcher is connected by a `specializes_in` edge to a `Field` vertex with `name = 'Bioinformatics'`, and by an `authored` edge (with the property `year = 2022`) to a `Paper` vertex. The two path patterns in the `MATCH` clause share the common researcher node `r`, ensuring that only researchers satisfying both conditions are selected. Based on the graph in Figure 2.3, this query yields the result: `Emily, "AI in drug discovery"`.

PGQL provides native, powerful support for navigational queries, based on regular path expressions (RPEs) [36, 109]. These expressions allow repetition using quantifiers like `*` and `+`, union using the `|` operator, and implicit concatenation by chaining edge patterns. Further, PGQL allows users to specify whether paths should be matched in a directed mode (each edge should be oriented as specified in the query), or in undirected mode (an edge may match the RPE regardless of the edge direction). PGQL RPEs can also match undirected graphs. Users can control whether they want the matching paths to be:

- `ACYCLIC` (no nodes should appear more than once along a path);
- `SIMPLE` (no repeated nodes, except possibly at the start and end);
- `TRAIL` (no repeated edge, but may contain repeated nodes), or
- `WALK` (nodes and edges may be arbitrarily repeated in a path).

Listing 2.5 illustrates PGQL's support for complex *regular path expression* (RPE) queries by combining a **bounded path expression** with a `TRAIL` restrictor. The query retrieves researchers `r` who authored a paper `p1`, where `p1` cites, through a path of 1 to 3 citation edges, another paper `p2`. The target paper `p2` must have been authored by a researcher `r2` who specializes in `Bioinformatics`, and must have been published in the year 2022.

The use of the quantifier `->{1,3}` restricts the citation trail length to between one and three edges, ensuring bounded exploration. The keyword `TRAIL` ensures that the query only considers citation paths where edges are not repeated, as required by the GPML [36]. Based on the property graph in Figure 2.3, this query returns one result: researcher `Jax`, who authored the paper `"Graph databases in genomics"`, which

cited (via a direct citation) the 2022 Bioinformatics paper "AI in drug discovery" authored by Emily.

```
1 SELECT r.name, p1.title
2 FROM GRAPH_TABLE(propGraph
3     MATCH TRAIL (r IS Researcher)
4         -[:authored]->(p1 IS Paper),
5         (p1)-[:cited]->{1,3}(p2 IS Paper),
6         (p2)<-[:authored]-(r2 IS Researcher)
7             -[:specializes_in]->(f IS Field {name = '
8             Bioinformatics'})
9 WHERE p2.year = 2022
10 )
```

Listing 2.5: PGQL query to find researchers whose paper cites, via a trail of 1 to 3 citations, a 2022 Bioinformatics paper

Query processing in (property) graph databases is still an area of very active research; recent advances encompass several key topics. We mention a few in order to illustrate the breadth of problems recently studied, while this thesis focuses on an efficient in-memory graph store, particularly tailored to correctly support out-of-order updates.

Given the potentially complex graph structure, establishing useful **indexes** is crucial for good query performance. Recent strategies include compact in-memory structures and cyclic representations for evaluating regular path queries and supporting worst-case optimal joins in sublinear space [11, 10]. Indexing on selective patterns has been integrated into the Cypher query pipeline to avoid high intermediate state cardinality and significantly accelerate query performance [57]. For **regular path query evaluation**, algebraic frameworks have been introduced to formalize the evaluation of path queries and treat paths as first-class citizens [7]. Bit-parallel simulation of Glushkov automata has also been proposed as a direct evaluation method for RPQs over compact graph indexes [11]. A schema-based typing system enables precise type inference for regular path queries [31]. Semiring-based provenance techniques further enrich the semantics and efficiency of recursive and path-based

queries [94, 95]. In the area of **schema and constraints**, Property Graph Types have been proposed for property graphs, to support schema discovery and structural validation [20, 6]. Reasoning with graph-generating dependencies enables **constraint enforcement and consistency checking** over property graphs [103]. Additionally, interactive **repair** frameworks allow users to resolve constraint violations through guided graph transformations under neighborhood constraints [55]. Finally, user query assistance techniques have been proposed to aid users in **exploratory graph search** by expanding incomplete queries using language modeling and retrieval principles [72].

2.4 Classification of graph data management systems

Graph database systems can be categorized according to various distinct dimensions, as illustrated in Table 2.1. These dimensions include: *(i)* native versus non-native data stores, where native data stores are inherently designed to handle graph data, whereas non-native data stores reuse ideas and techniques developed to manage data of other models; *(ii)* whether the systems support distributed storage or operate only on a single node; *(iii)* the type of storage supported by the systems: disk-based, in-memory, or both; *(iv)* transactional operation support; *(v)* multi-version concurrency control (MVCC). Transaction support refers to the general ability of a system to provide ACID properties — mainly atomicity (all-or-nothing execution) and isolation (ensuring correctness under concurrency). In-memory systems like Sortledton [41] do not provide full durability in a strict sense, because data is not persisted to disk. However, these systems are still considered transactional because they support atomic and isolated execution of transactions. Sortledton even mentions that durability can be optionally added using techniques like write-ahead logging. Stronger isolation levels, such as snapshot isolation, in contrast, can be obtained using MVCC (Multi-Version Concurrency Control). Some systems, like Neo4j [96], support transactions but do not implement MVCC; they only offer weaker guarantees, such as read committed. This is why we separate general transaction support from MVCC in our classification: a system may support transactions but still lack MVCC.

To discuss these graph database systems in detail, we broadly categorize them

Systems	native	ds	mem	disk	tr	mvcc
Velocity Graph [110]	✗	✓	✗	✓	✓	✗
Titan [123]	✗	✓	✗	✓	✓	✗
MS Graph Engine [101]	✗	✓	✓	✗	✗	✗
ArangoDB [8]	✗	✓	✗	✓	✓	✓
OrientDB [27]	✗	✓	✗	✓	✓	✓
Oracle Spatial and Graph [86]	✗	✓	✗	✓	✓	✓
Neo4j [96]	✓	✓	✗	✓	✓	✗
Sparksee/DEX [78]	✓	✗	✗	✓	✓	✗
GraphOne [58]	✓	✗	✓	✗	✗	✗
LiveGraph [126]	✓	✗	✓	✓	✓	✓
Teseo [64]	✓	✗	✓	✗	✓	✓
Sortedton [41]	✓	✗	✓	✗	✓	✓
STINGER [38]	✓	✗	✓	✗	✗	✗
LLAMA [76]	✓	✗	✓	✓	✗	✗
Spruce [102]	✓	✗	✓	✗	✓	✓
Our work	✓	✗	✓	✗	✓	✓

Table 2.1: Comparison of various graph database systems (native: native graph databases, ds: distributed storage, mem: In-memory support, tr: transactions support, mvcc: multi-version concurrency control protocol support)

based on their storage types into two main groups: in-memory systems and disk-based systems. Since the contribution of this thesis is an in-memory graph store, we discuss in-memory systems, which can be seen as prior work and direct competitors to ours, in Chapter 3. In the sequel of the present chapter, we discuss disk-based systems, analyzing them along the dimensions as mentioned above.

2.5 Disk-based graph database systems

These systems are designed to handle large-scale graph datasets that exceed the capacity of the main memory and offer data persistence, ensuring that graph updates are durably preserved, even after a system shutdown. From a performance perspective, the drawback of disk-based systems is their lower processing speed, due to the higher latency involved in retrieving data from disk storage. Table 2.1 includes nine among the main representative systems in this area.

Disk-based graph-database systems can be grouped into two categories. The first

category comprises **non-native data stores**, which store graph data by representing it in other data models and leveraging storage systems built for those. These include as key-value [33], document-based [8], object-oriented [110], and relational database management systems [14]. These systems are widely adopted because they are mature and well-tested. However, they have not been optimized for graph-specific workloads. In contrast, **native** graph databases are built specifically with the goal to efficiently handle graph operations; their features and performance continue to mature over time. Further, disk-based graph systems can also be categorized based on several dimensions of their architecture, including support for transactions, and data distribution techniques (such as sharding and replication). These architectural dimensions form the basis for our analysis in the following subsections: non-native disk-based systems are discussed in Section 2.5.1, and native disk-based systems are covered in Section 2.5.2.

2.5.1 Disk based non-native graph data stores

These stores adopt various techniques to represent and store the graph data. For example, Oracle Spatial and Graph [87], built on top of the Oracle **RDBMS**, stores vertices and edges as rows in normalized tables, with foreign keys capturing their connections. This often requires multiple joins to reconstruct graph relationships for traversal queries (see Section 2.6.4 for a detailed discussion of relational data stores). **Document-oriented** data stores, such as ArangoDB [8], encapsulate graph data within logically independent documents. Vertices are stored in vertex collections, where each vertex is represented as a document encapsulating the vertex attributes. Edges are stored in separate collections, with explicit references to source and destination vertices. While this design enhances flexibility, it results in costly operations during graph traversal across documents (see Section 2.6.3). **Key-value stores** offer fast lookups by associating each key with a single value, but they lack inherent relationship tracking, which leads to costly graph traversal operations (see Section 2.6.2). Object-oriented data stores, such as VelocityGraph [110], store graph updates by representing vertices and edges as C# objects linked through direct references. Each vertex object typically contains a unique identifier, a label, a dictionary-

like data structure for properties, and collections of incoming and outgoing edge objects. Similarly, each edge object references its source and destination vertex objects and stores its own label and properties (see Section 2.6.1).

As outlined in Table 2.1, non-native disk-based stores generally support transactions and utilize multi-version concurrency control (MVCC) to manage multiple versions of data records. However, some systems, such as Titan [123] and VelocityGraph [110], provide transactional support but lack MVCC. Non-native graph stores support distributed storage, with broad support for both data replication and data sharding. In particular, document stores often support sharding to the same extent as replication [19].

2.5.2 Disk-based native graph data stores

Native graph data stores are specifically built to store, manage, and query graph data efficiently; they adopt custom storage layouts tailored for graph workloads, enabling faster traversal and analytical performance. Among the most prominent native disk-based systems are Neo4j [96], LiveGraph [126], and LLAMA [76].

Neo4j [96] stores graph data natively using a record-based storage engine composed of multiple fixed-size record files. At the core of this architecture are three primary store files: `nodestore.db`, `relationshipstore.db`, and `propertystore.db`. The `nodestore.db` maintains node (vertex) records, where each record includes a unique node identifier along with pointers to the node's first property and its first relationship. The `relationshipstore.db` stores relationship (edge) records. Each relationship record stores the identifiers of its start and end nodes, the relationship type, and bidirectional pointers linking to the previous and next relationships for both connected nodes. Meanwhile, the `propertystore.db` holds all property key-value pairs for both nodes and relationships in the form of fixed-size property records. These property records are organized within linked lists, with each node or relationship referencing the first property in its chain [84]. Neo4j [96] provides ACID transactions and replication but lacks MVCC and native sharding.

LiveGraph [126] and **LLAMA** [76] both support disk-based storage via memory-

mapped files (`mmap`), allowing them to handle graphs that exceed the available RAM. However, both systems are primarily optimized for in-memory performance. In LiveGraph, the storage layout is centered around two main components: vertex blocks and the Transactional Edge Log (TEL). Each vertex is stored in a vertex block, which holds the vertex metadata and associated properties. These blocks follow a *copy-on-write-like mechanism*, meaning that upon a graph update, instead of modifying data in place, a new version of the vertex block is created with the desired new values. This enables safe concurrent updates and supports versioning. To manage multiple versions, LiveGraph maintains a backward chain of historical versions for each vertex block, allowing transactions to access the correct snapshot according to their timestamp.

Edge information, on the other hand, is maintained in the TEL, an append-only log structure that resides in a memory-mapped file. We detail TEL further in Chapter 3, given that its in-memory design is closely related to our thesis work). LiveGraph [126] offers both transactions and MVCC; it does not support distribution.

LLAMA [76] employs a different architectural approach by leveraging multiversioned snapshots. It organizes the vertex metadata in a vertex table, implemented using the Large Multiversioned Array (LAMA) structure. This vertex table is shared across all snapshot versions, enabling efficient memory usage and fast snapshot creation through copy-on-write indirection. As another in-memory graph store, also comparable to our work, LLAMA is discussed in further details in Chapter 3. LLAMA [76] does not provide transactional guarantees nor distributed storage.

Performance evaluations [5, 56, 79, 80, 108] show that in general, non-native graph databases often outperform native systems in transactional graph update workloads, whereas native graph databases tend to excel in read-intensive tasks such as simple queries and global analytics. However, LiveGraph [126] exhibits competitive performance in both domains. Generally speaking, many engineering (implementation) aspects determine the best system for a given type of workload; as systems evolve, such performance comparisons also need to be updated along the time.

Compared with disk databases, in-memory graph engines stand to gain in performance by avoiding disk latency. We explore them in more detail in Chapter 3.

2.6 Non-native data stores

In this section, we discuss various graph storage models leveraging object-oriented, key-value, document-based, and relational databases [19]. The advantage of non-native data stores is that the flexibility allows graph data storage along with other data models. Their disadvantage is that they are not inherently optimized for graph processing. The following sections provide an overview of how different non-native data stores support graph updates.

2.6.1 Object-oriented databases

Graphs can be modeled in object-oriented databases, where data elements and their relationships are represented as objects connected via references. Each vertex can be represented by an object encapsulating the vertex attributes, such as an identifier (ID), label (type), and a set of properties stored in a flexible dictionary-like structure. Similarly, edges that define the connections between vertices are represented as objects that reference their source and destination vertices along with their properties. Each vertex maintains a collection of its adjacent edges to optimize traversal efficiency. For instance, in a social network graph, a person class (vertex) represents individuals with properties such as name and age, while a friendship class (edge) links two person objects, storing properties such as the date when the friendship was established.

An example of an object-oriented graph database is VelocityGraph [110], built on top of VelocityDB [111]. The VelocityGraph is a distributed object graph database in which vertices, edges, and their associated properties are represented as C# objects, each containing references to other objects. VelocityGraph introduces abstractions such as `VertexType`, `EdgeType`, and `PropertyType` to handle vertices, edges, and their associated properties, with each object containing a unique identifier that references its physical storage location. Additionally, each vertex and edge is assigned a label type and utilizes dictionary-like data structures to store its properties [19].

2.6.2 Key-value stores

Key-value stores are a type of NoSQL stores [33] designed for high-performance and highly scalable lookups based on unique keys. The structure of keys and values varies depending on the application. Keys can take different forms, such as simple URIs, hashes, or structured formats, while values are often stored as byte arrays. The general nature of key-value stores allows for the representation of graph data in multiple ways [19].

Key (ID)	Value (Adjacent list)
u1	[u2, u3, u4]
u2	[u1, u3, u4]
u3	[u2, u4]
u4	[u1, u2, u3]

Figure 2.4: Example of an adjacency list representation of a social network using a key-value store

One common approach for representing graph data in a key-value store is based on **adjacency lists**, as illustrated in Figure 2.4. This table depicts an adjacency list representation of a social network using a key-value store. In this table, the first column represents the keys, where each node in the graph is stored as a key, while the second column contains the corresponding values; to each key is attached a list of adjacent nodes. For instance, the first row shows that node u1 is stored as a key, and its adjacent nodes (u2, u3, and u4) are stored as its values.

Key (Edge)	Value (Metadata)
u1:u2	{"relationship": "friend", "since": "2025-01-01"}
u2:u3	{"relationship": "friend", "since": "2025-03-01"}

Figure 2.5: Example of an edge list representation of a social network using a key-value store

Another approach consists of using an **edge list** representation, where each key corresponds to an edge rather than a node, and the value stores relationship metadata. As exemplified in Figure 2.5, a connection between two users is represented as a key by combining their user IDs, while the associated value contains metadata such as the type of relationship and the timestamp indicating when the connection was established. For instance, the friendship between u1 and u2 is stored with the key

$u1:u2$, and its corresponding value specifies the relationship type as "friend" along with the date "2025-01-01".

An example of a practical key-value graph database engine is the **Microsoft (MS) Graph Engine** [101], a distributed key-value store developed for the efficient storage of graph data. The MS graph engine is built on top of Trinity, which is a key-value, globally addressable, distributed in-memory storage system, where keys are referred to as *cell IDs* and values as *cells*. These cells can store diverse data types, including the IDs of other cells. In the MS graph engine, vertices are represented as cells, with a dedicated field containing either a vertex ID or a hashed version. Edges holding only connectivity information related to a vertex v are stored within v 's cell as a list of neighboring vertex IDs. In contrast, when an edge contains rich metadata, both the edge and its associated metadata can be maintained in a separate, dedicated cell to enhance data organization [101].

2.6.3 Document-based stores

Document stores [19] are a type of NoSQL database known for their flexible schema, enabling them to support various data models. In these databases, a document serves as the fundamental unit of storage and is typically represented in a semi-structured format, such as JSON [22] or XML [23]. Documents are organized into *collections*, and multiple collections, together, form a *database*. Document stores extend the key-value model, where each document consists of one or more keys (attributes) paired with values that can belong to any data type. This key-value structure enables the representation of graph data within document stores, where each vertex is stored as an individual document, containing a unique vertex ID along with its associated properties. The storage of edges depends on the implementation. Edges can either be embedded within the source or destination vertex document or stored as separate documents in an edge collection. When edges are maintained as independent documents, each edge document typically includes a unique edge identifier, references to the source vertex (from) and destination vertex (to), along with associated properties.

One practical example is **ArangoDB** [8]. A document in ArangoDB is assigned to

a single collection, and its key attribute must be unique within that collection.

Within ArangoDB, vertex collections store vertex documents, each containing vertex metadata, such as identifiers and associated properties. Edges are stored in edge collections, where each document corresponds to a specific edge along with its associated properties, and indicates the source vertex (from) and destination vertex (to). For graphs having multiple edge label types, ArangoDB creates a separate collection for each edge type, in order to speed up query processing. For instance, traversal is required for a specific edge type (e.g., friend of), storing the edges with this label separately from others reduces the number of edges that the system needs to manipulate at runtime [19].



Figure 2.6: Graph data represented in a document store: two vertex documents and one edge document.

Vertex and edge storage in ArangoDB is exemplified in Figure 2.6. Figures 2.6a and 2.6b show two vertex documents representing users Khan and Kun, with properties such as name, age, and type. Figure 2.6c presents an edge document modeling a `friend_of` relationship between them. The `_from` and `_to` fields link the edge to the respective vertex documents.

2.6.4 Relational database management systems

Relational database management systems (RDBMSs, in short) organize data into tables composed of rows and columns [14, 30, 46]. Each row represents an individual record, such as individual employee information, whereas each column denotes a specific attribute of the data, such as the employee's name or address. The tables use primary keys consisting of one or more columns to uniquely identify each record within the table. Additionally, tables may employ foreign keys, which refer to the pri-

many keys in other tables, enabling relationships such as one-to-many, many-to-one, and many-to-many between tables.

UserID	Name	Age	Location
1	Lydia	24	Paris
2	Hope	27	London
3	Zion	30	Albany
4	Saint	22	Sydney
5	Evan	29	Toronto

Figure 2.7: Social network users represented as vertices.

FriendshipID	UserID1	UserID2	FriendSince
1	1	2	2018-05-12
2	1	3	2019-03-23
3	2	4	2020-07-15
4	3	5	2017-09-10
5	4	5	2021-01-22

Figure 2.8: Friendship connections represented as edges.

In relational databases, graphs can be effectively represented using two distinct tables: one for vertices and another for edges. The vertex table stores each vertex identified uniquely by a primary key, such as a vertex ID, along with their associated properties. The edge table stores relationships as pairs of source and destination vertex IDs; each of these acts as a foreign key referring to the primary keys in the vertex table. Additionally, edge-specific attributes can be included as extra columns in the edge table. Consider a social network scenario as an example as shown in Figure 2.7 and Figure 2.8, in which the vertex table captures user profiles, containing information such as their ID, name, age, and geographic location. The edge table captures friendship relationships between these users by referencing their IDs from the vertex table. Each friendship record can further include details like the date the friendship started.

An example of an RDBMS-based graph system is **Oracle Spatial and Graph** [87], which is built on top of the Oracle database. In this system, the rows in the RDBMS tables represent vertices, and the relationships between these rows define the edges. The properties associated with the vertices and edges are separately maintained as

key-value pairs in the dedicated structures. Additionally, the Oracle Spatial and Graph system provides various built-in parallel graph algorithms including traversal, PageRank computation, path-finding, and community detection [19].

2.7 Summary

Graphs are a natural representation for complex, heterogeneous data. The increase of interest in this data representation has encouraged the development of different flavors of graphs, from the simplest (connectivity graphs) to the most complex-structured (property graphs), also comprising the RDF data model, favored for interoperability and as a Web standard for data sharing.

In this chapter, we presented the main features of these data models and their associated processing paradigms and query languages. We have discussed common graph workloads, and proposed a classification of graph database systems, covering both native and non-native data stores. The chapter concluded with an analysis of *disk-based* graph stores, and one of *non-native* graph stores, highlighting key performance trade-offs. This background sets the stage for Chapter 3, which focuses on *native, in-memory* graph data stores, closely related to the focus of this thesis.

Chapter 3

In-memory graph data management systems

Since the focus of this thesis is on in-memory graph database systems, this chapter presents a detailed discussion of recent systems in this category. Section 3.1 introduces the core in-memory data structures commonly used to store graph updates, establishing a foundation for understanding the layouts of different graph database systems. Building on this, Section 3.2 examines representative in-memory graph database systems, analyzing their layout choices and design decisions across key dimensions shown in Table 2.1

3.1 Basic concepts

This section presents the in-memory data structures employed to store graph updates—an essential aspect that influences the performance, scalability, and design trade-offs of in-memory graph database systems. The most commonly used representations include the edge list (detailed in Section 3.1.1), which is favored for its simplicity and high ingestion throughput; the compressed sparse row (CSR) format (Section 3.1.2), which excels in read-heavy analytical workloads due to its compact, cache-friendly layout; and the adjacency list (Section 3.1.3), which offers a balance between efficient neighborhood scans and update write throughput, making it well-suited for dynamic graph workloads.

A-B	A-D	A-F	B-E	B-F	D-E
0	1	2	3	4	5

Figure 3.1: Edge list representation of the example graph shown in Figure 2.2

3.1.1 Edge list

In an *edge list*, a graph is stored as a collection of edge records, where each edge is represented as a pair (u, v) , indicating a connection between two vertices. This representation offers a compact way to describe a graph, particularly in the case of sparse graphs, as it requires only $\mathcal{O}(|E|)$ space, where $|E|$ denotes the number of edges. The edge list format is especially suited for streaming scenarios, where new edges can be ingested fast by simply appending them to the list, without requiring indexing that could impact the ingestion performance. However, a notable limitation of this format is that it does not support efficient graph analytics, since searching the neighbors of a given vertex involves scanning the entire edge list, incurring a time complexity of $\mathcal{O}(|E|)$. As an example, the graph shown in Figure 2.2, where vertices represent entities (e.g., users) and edges denote undirected relationships between them within a social network graph. The corresponding edge list format, shown in Figure 3.1, expresses the graph as a collection of edge tuples, with each tuple representing a relationship between two connected vertices. In this case, the edge list stores the following set of connections: (A, B), (A, D), (A, F), (B, E), (B, F), (D, E). Despite its limitations for query efficiency, the edge list remains a practical choice for streaming workloads due to its fast, append-only ingestion. GraphOne [58], for instance, uses a circular edge log to ingest streaming graph updates at high throughput. Here and in the sequel, in the context of storing updates to a graph, *log* refers to a data structure that records incoming edges in an append-only edge list format, which supports fast sequential writes. The term *circular* indicates that this log is implemented as a fixed-size buffer that wraps around, reusing space once earlier entries have been archived.

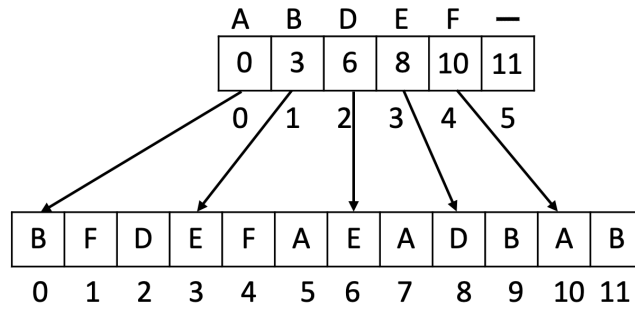


Figure 3.2: CSR representation of the graph in Figure 2.2

3.1.2 Compressed sparse rows (CSR)

The CSR structure consists of two primary components: a *vertex table*, which maps each vertex to the starting offset of its adjacency list, and an *edge table*, which stores all neighbor identifiers in a single, flattened array. The vertex table is implemented as an array of size $|V| + 1$, where each entry at position i points to the starting index of the adjacency list for the i^{th} vertex in the edge table. The final entry marks the exclusive end of the last vertex's adjacency range. For example, consider the graph shown in Figure 2.2 and its CSR storage illustrated in Figure 3.2. The vertex table includes five labeled entries, A to F, plus a sixth entry marking the end of the final adjacency list. Each vertex maps to a segment of the edge table containing its neighbors. For instance, vertex B begins at offset 3 and ends at offset 6 in the edge table, corresponding to neighbors E, F, and A.

CSR is cache-friendly because it provides sequential neighborhood access: all neighbors of a vertex are stored together in a compact neighbor list. This layout allows efficient use of CPU caches during graph traversal. Owing to the cache-friendly layout of both arrays, CSR significantly reduces random memory accesses during neighborhood scans, making it particularly well-suited for in-memory graph analytics workloads dominated by read-heavy traversal patterns. However, CSR is not inherently designed for dynamic updates: inserting or deleting edges typically requires *shifting entries in the edge table* and recomputing the offsets in the vertex table, which can be expensive in terms of write throughput.

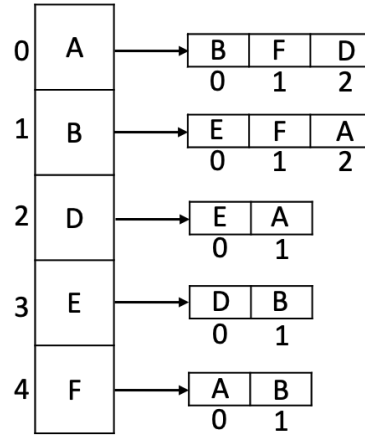


Figure 3.3: Adjacency list representation of the graph in Figure 2.2

3.1.3 Adjacency list (AL)

An *adjacency-list* (AL) layout stores graph content in two core components: a *vertex array* (VA) and a corresponding set of *adjacency lists* (AL). The VA, of size $|V|$, serves as an index mapping each vertex to its own AL. Specifically, the entry at index i points to the beginning of the AL for vertex v_i . These ALs are implemented as variable-length arrays or dynamically managed blocks, where each list holds the neighbors of a specific vertex. Since each AL is allocated independently, updates such as insertions or deletions of neighbors affect only the relevant vertex's AL. The AL format offers a balance between the write throughput of edge list representations and the scan efficiency of CSR, making it well-suited for dynamic graph workloads [41].

For example, consider the undirected graph shown in Figure 2.2, which models a social network. Its corresponding AL representation is depicted in Figure 3.3. Each vertex entry in the VA points to an independently managed neighbor list. For example, vertex A (at index 0) connects to vertices B, F, and D, forming the adjacency list $Adj(A) = [B, F, D]$. Similarly, vertex B maps to $Adj(B) = [E, F, A]$, and so forth for the remaining vertices.

3.2 In-memory data stores

In-memory graph database systems have gained significant popularity in recent years, primarily due to advancements in RAM capacity [63], which enable most databases

to reside entirely in memory. This results in improved write throughput and reduced query latency. Table 2.1 presents an overview of several in-memory graph database systems.

In-memory graph database systems are broadly categorized into *non-native* and *native* data stores. **Non-native data stores**, such as *Microsoft’s Graph Engine* [101], are built on top of general-purpose data models like key-value stores and extend them with graph-specific abstractions. MS Graph Engine, for instance, utilizes *Trinity*, a distributed in-memory key-value store, as its backend, where graph updates (vertices and edges) are modeled as addressable “cells.” These cells support references to other cells, enabling the representation of graph structures. A detailed discussion of the underlying key-value store layout is provided in Section 2.6.2. **Native data stores**—such as *LLAMA* [76], *GraphOne* [58], *STINGER* [38], *LiveGraph* [126], *Sortledton* [41], *Teseo* [64], and *Spruce* [102]—are purpose-built to store and query graph data with low latency and high throughput. These systems diverge significantly in their architectural choices, including their *data layouts*, concurrency control mechanisms (e.g., single-version vs. multi-version), and support for *transactional guarantees*. From this point forward, we concentrate on surveying these native in-memory graph systems in detail, as they form the basis of the design decisions and comparisons relevant to this thesis.

In the following subsections, we discuss the storage layout of each native in-memory graph data store. Specifically, we present the layout designs of LLAMA in Section 3.2.1, GraphOne in Section 3.2.2, STINGER in Section 3.2.3, LiveGraph in Section 3.2.4, Teseo in Section 3.2.5, Sortledton in Section 3.2.6, and Spruce in Section 3.2.7. Then we conclude the chapter by analyzing and comparing their design decisions in Section 3.2.8.

To illustrate the internal data layout of each system, we consider a simple example graph shown in Figure 3.4. This graph evolves through a series of four edge insertions applied in the following order: $0 \rightarrow 1$, $1 \rightarrow 2$, $0 \rightarrow 2$, and $0 \rightarrow 3$.

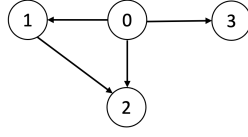


Figure 3.4: Example graph used to illustrate internal data layouts of graph storage systems.

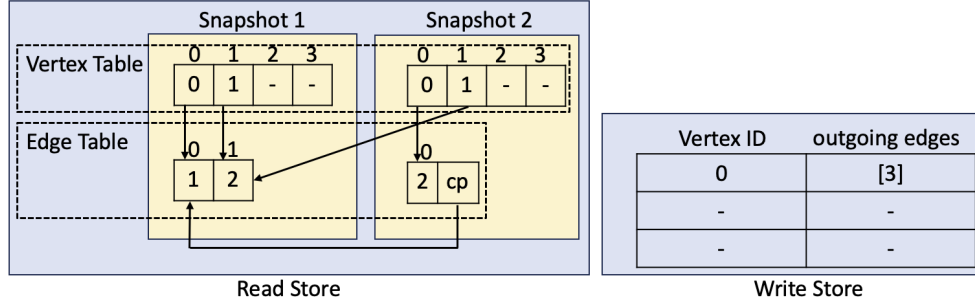


Figure 3.5: LLAMA data layout representation

3.2.1 LLAMA

LLAMA [76] adopts a data layout based on an enhanced *Compressed Sparse Row (CSR)* [98] representation to efficiently manage dynamic graph updates. It organizes graph data into a series of *snapshots*, where each snapshot captures incremental structural changes over time. The vertices are stored in a shared multiversioned vertex table called *LAMA* (Large Multiversioned Array), which supports copy-on-write mechanisms for efficient snapshot isolation. Edges inserted in each snapshot are maintained in fixed-length edge tables as adjacency list fragments. When the adjacency list of a vertex spans across multiple snapshots, *continuation records* are used to maintain logical connectivity—these are metadata pointers embedded in the edge table that link the edge list of a vertex from one snapshot to the next, enabling traversal of fragmented adjacency lists as a unified view. To handle incoming updates, LLAMA maintains a *write store* (a temporary buffer for new edge insertions), which is periodically flushed into the *read store*—a compact, multiversioned CSR layout optimized for fast sequential access to neighborhoods during analytical queries.

To illustrate the internal data layout of LLAMA, consider the example graph from Figure 3.4, which evolves through a sequence of four edge insertions: $0 \rightarrow 1$, $1 \rightarrow 2$, $0 \rightarrow 2$, and $0 \rightarrow 3$. These insertions are captured incrementally across two snapshots

and a write buffer, as shown in Figure 3.5. In *Snapshot 1*, the edges $0 \rightarrow 1$ and $1 \rightarrow 2$ are inserted, and their adjacency lists are stored in the edge table, with the vertex table pointing to their respective starting offsets. *Snapshot 2* contains the insertion $0 \rightarrow 2$, which extends vertex 0's adjacency list. Since vertex 0 already has an entry in Snapshot 1, a *continuation record* (marked as *cp*) is used to logically link the new edge to the previous list. The last edge $0 \rightarrow 3$ is not yet flushed to the read store and is temporarily held in the *write store*, where it is buffered under vertex 0. Furthermore, LLAMA handles deletions by inserting tombstone markers into the write store, rather than physically removing edges. These tombstones take effect only when a new snapshot is triggered, at which point they logically invalidate the corresponding edges in the new snapshot; however, previous versions of those edges remain intact in earlier snapshots, preserving historical versions.

LLAMA does not provide transactional guarantees, but it supports concurrent updates and analytical queries by separating data ingestion from query execution. Updates are first written to a temporary buffer (the write store), while analytics are performed on the read store, which holds the latest materialized snapshot of the graph. This decoupled architecture allows updates and queries to proceed in parallel without contention. However, a key limitation is that the read store does not reflect updates still residing in the write store, meaning that analytics may operate on stale data. To incorporate the most recent updates, a new snapshot must be explicitly triggered to flush the write store into the read store. While this enables up-to-date analytics, the snapshotting process introduces significant computational overhead. For frequent and short-lived traversal queries, this overhead can dominate the total query execution time. Additionally, LLAMA supports disk storage by memory-mapping snapshot files using `mmap`, enabling access to large graphs that exceed main memory capacity.

3.2.2 GraphOne

GraphOne [58] employs a hybrid data layout combining an edge list-based log and an adjacency store to efficiently support real-time graph analytics on dynamic graphs. Incoming edge updates are first stored in a small circular buffer known as the *edge*

log, which maintains them in arrival order using an edge list format. This structure facilitates high-speed ingestion by appending updates sequentially. Periodically, when the number of non-archived edges exceeds a predefined threshold, these edges are transferred to the adjacency store through an archiving phase. The adjacency store uses an adjacency list format, composed of a vertex array, per-vertex edge arrays, and a multi-versioned degree array. Per-vertex edge arrays support chaining to accommodate dynamic updates, while the degree array enables snapshot isolation through versioning. GraphOne applies a dual-versioning technique by maintaining fine-grained versions in the edge log and coarse-grained snapshots in the adjacency store, allowing concurrent batch and stream analytics with consistent views of the graph.

To illustrate GraphOne’s internal data layout, consider the example graph shown in Figure 3.4, which evolves through four edge insertions: $0 \rightarrow 1$, $1 \rightarrow 2$, $0 \rightarrow 2$, and $0 \rightarrow 3$. As depicted in Figure 3.6, these edges are appended to the *edge log* in arrival order. GraphOne applies a threshold-based flushing strategy, where every two unarchived edges trigger an archival process. After the first two updates, a snapshot S_0 is created, marking the log index at position 2. The next two updates similarly result in snapshot S_1 , marking position 4. During the archival process, edges are transferred from the edge log to the read-optimized *adjacency store*, which consists of a vertex array, a chained edge array, and a multi-version degree array. The vertex array maintains pointers to each vertex’s adjacency list, while chained edge arrays allow incremental updates to be stored. In this example, vertex 0 first receives edge $0 \rightarrow 1$ in S_0 , followed by $0 \rightarrow 2$ and $0 \rightarrow 3$ in S_1 , forming a chained list: $[1] \rightarrow [2, 3]$. The multi-version degree array tracks the number of edges per vertex for each snapshot version. For instance, vertex 0 has a degree of 1 in S_0 and a degree of 3 in S_1 , while vertex 1 has a degree of 1 in S_0 . GraphOne handles deletions by appending tombstone entries to the circular edge log, using a negative destination ID to indicate edge removal. These tombstones are treated similarly to regular edge insertions and are processed only during the next snapshot creation. During this archival phase, the deletion entry is interpreted and translated into a special marker (e.g., -1) within the chained edge arrays of the adjacency store. This marker is used to logically invalidate

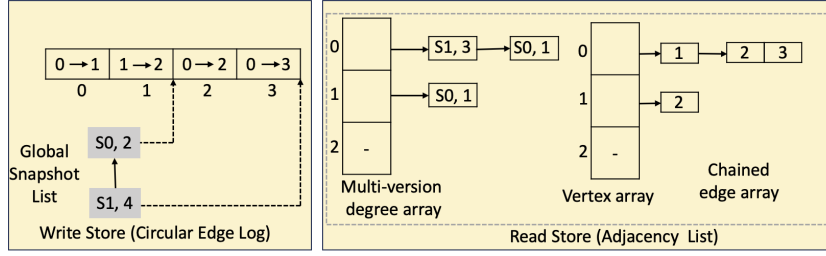


Figure 3.6: GraphOne data layout representation

the corresponding edge in the vertex’s adjacency list, effectively excluding it from future analytical queries.

GraphOne does not provide transactional guarantees, but it supports concurrent updates and analytical queries by separating data ingestion from query execution. Updates are initially written to a temporary circular buffer (the edge log), while analytics are executed on the read store, which consists of materialized snapshots archived in an adjacency list format. This decoupled design allows ingestion and query workloads to proceed in parallel without contention. However, similar to LLAMA, a key limitation is that the read store does not reflect updates that remain in the write store, resulting in potentially stale query results. To access the most recent changes, GraphOne allows the construction of a *private view*, which temporarily merges the edge log with the adjacency store at query time. While this mechanism enables up-to-date analytics, the merging process introduces significant overhead. For frequent and short-lived traversal queries—such as BFS or SSSP—this overhead can dominate the query execution time, limiting GraphOne’s effectiveness for real-time analytical workloads.

3.2.3 STINGER

STINGER [38] employs a dynamic graph data structure designed for high-throughput streaming updates. It adopts an adjacency list-based design, where edges associated with each vertex are stored in a linked list of fixed-size blocks. Each edge in the adjacency list is appended in the order of arrival, enabling fast insertions. Vertices are managed in a pre-allocated vertex table, with each vertex maintaining a pointer to its corresponding list of edge blocks via the Logical Vertex Array (LVA). STINGER sup-

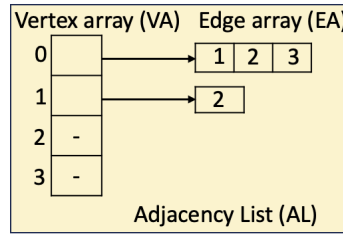


Figure 3.7: STINGER data layout representation

ports parallel edge insertions and deletions, leveraging fine-grained synchronization to ensure thread safety for individual reads and writes.

To illustrate STINGER’s layout, consider the example graph in Figure 3.4, where vertex 0 is connected to vertices 1, 2, and 3, and vertex 1 is connected to vertex 2. This input is represented in STINGER’s structure as shown in Figure 3.7. The graph is maintained using an adjacency list format, where the Vertex Array (VA) maps each vertex ID to a pointer into an Edge Array (EA) that holds its neighbors. The adjacency list of each vertex is implemented as a dynamically extensible linked list of fixed-size edge blocks, typically holding up to 14 edges per block. When new edges are inserted, they are appended in arrival order into the next available slot in the edge block. STINGER supports deletions by directly removing the corresponding edge entry from the fixed-size edge blocks in the adjacency list.

Furthermore, STINGER does not support transactions but provides thread safety for individual edge operations using fine-grained locking. It allows concurrent updates and analytical queries; however, full-graph scans may become inconsistent if executed concurrently with updates. This is because the graph structure can change while a query is traversing it, potentially leading to partially updated or inconsistent results. This design reflects STINGER’s focus on high ingestion throughput and low synchronization overhead, but it comes at the cost of reduced consistency for real-time analytics.

3.2.4 LiveGraph

LiveGraph [126] stores graph edge entries (insertions and deletions) in adjacency lists, one for each source vertex, in the order in which they are received; this facili-

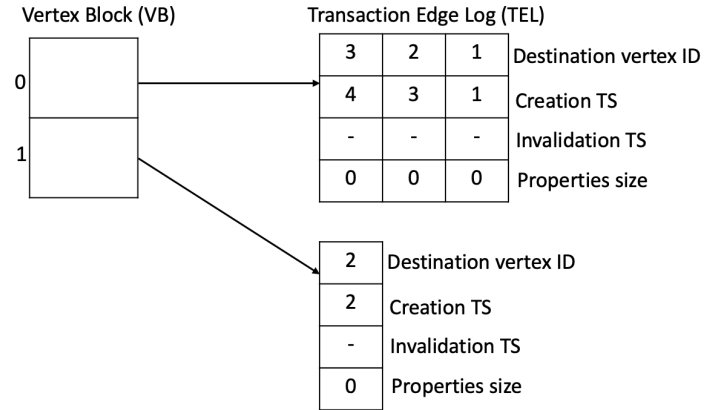


Figure 3.8: LiveGraph data layout representation

tates random access to each vertex. Central to its design is the *Transactional Edge Log (TEL)*, an append-only log-structured block associated with each source vertex that records edge updates in sequential order. Edge entries are tagged with transaction timestamps and invalidation timestamps, to enable multi-version concurrency control (MVCC). Edge properties are stored separately from edge metadata in the TEL. Each vertex in LiveGraph is stored in an individually allocated vertex block, where the previous versions are linked using backward pointers (each version pointing to the previous one). This is backward chaining design enables constant-time insertion of a new version. A Bloom filter in each TEL block allows a fast lookup of existing edges.

To illustrate LiveGraph’s internal layout, consider the example in Figure 3.4, where edges are inserted in the sequence $0 \rightarrow 1$, $1 \rightarrow 2$, $0 \rightarrow 2$, and $0 \rightarrow 3$. These updates are appended to the TEL blocks of their respective source vertices in arrival order, as shown in Figure 3.8. Each entry in the TEL includes the destination vertex ID, a creation timestamp marking the transaction timestamp of the insertion, an invalidation timestamp used to record deletions, and a property size field. For example, the TEL of vertex 0 stores three edges to vertices 1, 2, and 3, each with their respective creation timestamps. The property size field indicates the size of any associated edge properties; in this example, all entries have a size of 0, as no properties are stored. LiveGraph handles deletions by locating the most recent insertion entry of the target edge and updating its invalidation timestamp with the deletion’s transaction time.

Furthermore, LiveGraph supports transactions and enables concurrent updates

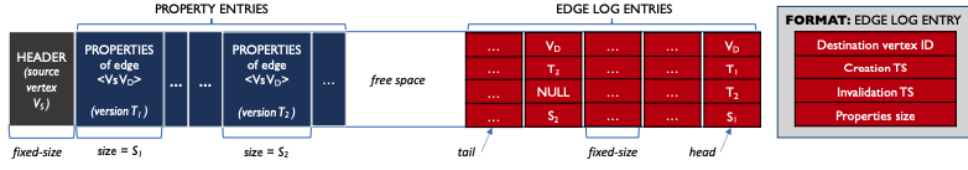


Figure 3.9: Illustration of the timestamp-based MVCC protocol used in LiveGraph. Adapted from LiveGraph [126], Figure 3.

and analytical queries with snapshot isolation guarantees by adopting a timestamp-based MVCC design, in which all versions of an edge are stored together in a single TEL, preserving the arrival order of updates (see Figure 3.9). Writers are assigned transaction timestamps, and readers access the graph using read timestamps. This design allows concurrent reads and writes with minimal contention. Since all edge versions are co-located in the TEL and ordered by insertion time, LiveGraph naturally supports historical queries based on database time, enabling efficient access to past graph states. However, this layout also introduces a performance overhead: *even when only the latest edge version is needed, the entire adjacency list must be scanned, including older versions*. LiveGraph stores extra metadata with each edge to support versioning, which increases memory usage and may lead to higher cache miss rates. The system is capable of scaling to large graphs that exceed main memory capacity, by memory-mapping TEL blocks, using `mmap`, to disk.

3.2.5 Teseo

Teseo [64] is built around the concept of the *fat tree*, which is a modified B+-tree with page-segmented leaves. The graph is entirely stored within these leaves, which are internally organized as sparse arrays [64] to support sorted storage with reserved gaps for incoming insertions. Each segment within a leaf operates in either a read-optimized (ROS) or write-optimized (WOS) mode. In ROS mode, vertices and edges are maintained in sorted order, enabling efficient scans, while WOS segments use unsorted buffers which receive high-rate updates. Vertices are stored alongside their edges, and edges with the same source vertex follow immediately, preserving locality for both scan and update operations. For concurrency control, each segment is protected by a hybrid latch, combining optimistic and pessimistic locking strategies.

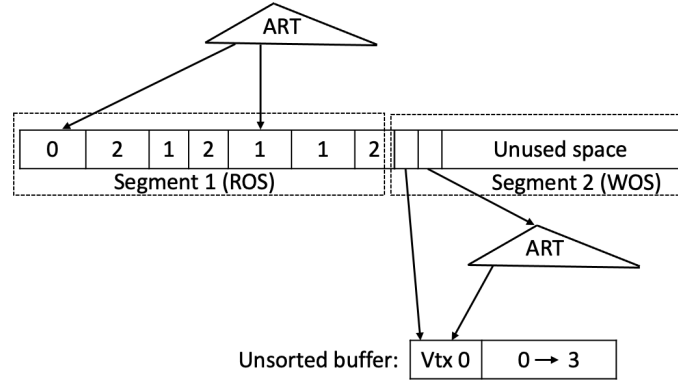


Figure 3.10: Teseo data layout representation

To illustrate Teseo’s internal layout, consider the evolving graph shown in Figure 3.4, which receives edge insertions in the order: $0 \rightarrow 1$, $1 \rightarrow 2$, $0 \rightarrow 2$, and $0 \rightarrow 3$. As depicted in Figure 3.10, the first three edges $0 \rightarrow 1$, $1 \rightarrow 2$, and $0 \rightarrow 2$ have already been flushed to the read-optimized segment (Segment 1), which maintains adjacency lists in sorted order using a compact sparse array format. In this layout, each source vertex (e.g., 0 and 1) is followed by the size of its outgoing edge list and the corresponding destination vertex IDs. These entries are indexed by an Adaptive Radix Tree [63] (ART, in short) to support fast lookups. The most recent update, $0 \rightarrow 3$, is still buffered in the write-optimized segment (Segment 2), which uses an unsorted format to support fast insertions. This buffer is also indexed using a separate ART for efficient access to pending updates. Teseo handles deletions by appending a new version entry to the edge’s version chain, marked as deleted, and linked to the previous version; these previous versions remain accessible until removed by garbage collection.

Teseo supports transactions and enables concurrent updates and analytical queries with snapshot isolation guarantees by adopting an optimistic MVCC model inspired by the Hyper protocol [85], illustrated in Figure 3.12. Edge updates are maintained in version chains and organized within fixed-size adjacency blocks, where entries are sorted by destination ID. The latest version of each edge is stored in the main sorted block, while older versions are kept in a separate version area, allowing analytical queries to access the most recent state efficiently. While this design minimizes con-

tention for read-heavy workloads, frequent updates during query execution may lead to transaction restarts due to the optimistic concurrency strategy, potentially degrading performance. In addition, Teseo does not preserve the insertion order of edge updates and is not optimized for historical queries. The Teseo paper does not mention historical queries. We can however note that Teseo does not preserve the order of edge updates, which makes reconstructing past graph states inefficient and unsuitable for historical query processing, if one were to add it to the system.

Among the surveyed systems, only LiveGraph explicitly mentions the possibility of querying past graph states. This is possible because LiveGraph maintains the arrival order of updates (i.e., database time W_T). However, its support is limited: the paper does not include implementation details or performance evaluations for historical queries. There is one important distinction between our work and LiveGraph: LiveGraph supports historical queries based on database arrival time (W_T), whereas our system supports historical queries based on stream time (S_T).

3.2.6 Sortledton

Sortledton [41] adopts an adjacency list-based data structure where each vertex maintains a *neighborhood set* stored either as a sorted vector or as an *unrolled skip list*[93], depending on the size of the neighborhood. Specifically, small neighborhoods are kept as sorted vectors to allow cache-efficient scans, while larger sets exceeding 512 edges are split into multiple sorted blocks connected through skip list. Each neighborhood is organized into a block structure sorted by destination IDs. To illustrate Sortledton’s internal layout, consider the example graph from Figure 3.4, which evolves through insertions: $0 \rightarrow 1$, $1 \rightarrow 2$, $0 \rightarrow 2$, and $0 \rightarrow 3$. As shown in Figure 3.11, vertex 0’s adjacency list contains neighbors 1, 2, and 3 stored in sorted order, and vertex 1 is linked to vertex 2. For illustration, the layout uses skip lists as the indexing structure, though in practice, Sortledton uses skip lists only after a block exceeds 512 entries.

Sortledton supports transactions and enables concurrent updates and analytical queries with snapshot isolation guarantees by adopting an optimistic MVCC model, similar to Teseo and inspired by the Hyper protocol [85] (illustrated in Figure 3.12).

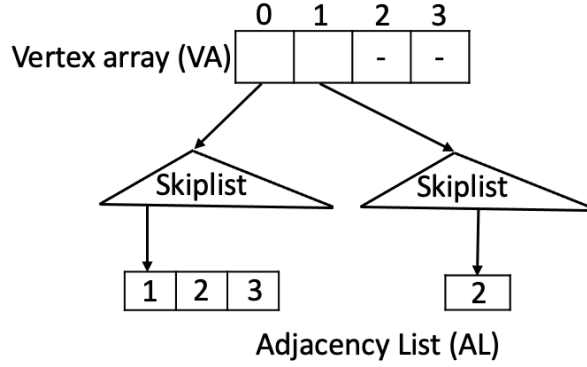


Figure 3.11: Sortedledton data layout representation



Figure 3.12: Illustration of the Hyper protocol showing versioned edge inserts in a dynamic graph. Adapted from Sortedledton [41], Figure 6.

Each edge maintains an inlined version chain directly within the block: the most recent version is stored in the sorted block, while older versions are linked behind it. This structure allows efficient access to the latest edge versions; however, it is not optimized for historical queries, as it does not preserve the arrival order of updates.

3.2.7 Spruce

Spruce [102] relies on a compact tree-like architecture inspired by the van Emde Boas (vEB) tree [102]. The system organizes vertices hierarchically using a multi-level index; vertex insertions and deletions do not necessitate node splits or merges. For edge storage, Spruce adopts an adjacency-list data structure wherein each source vertex maintains a *buffer block* and a *sorted block*. The buffer block temporarily stores new edge insertions in an unsorted form, enabling fast, low-latency updates. Once the number of buffered edges exceeds a predefined threshold, the buffer is consolidated into the sorted block. The sorted block holds edges in ascending order of destination IDs to support efficient analytical scans. For each vertex, there is only a single sorted block at a time, regardless of its out-degree. This design improves cache locality, since edge updates are stored contiguously in both the sorted and the buffer blocks, thus reducing cache misses during scans. However, frequent merging of the buffer block

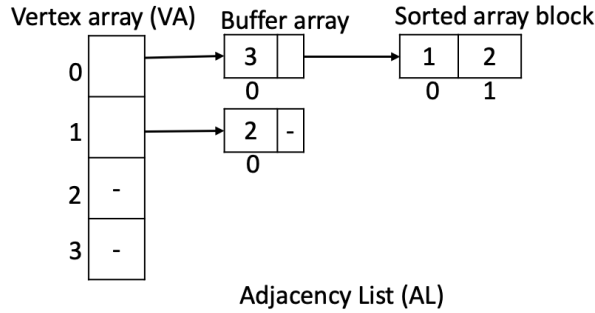


Figure 3.13: Spruce data layout representation

into the sorted block can degrade update throughput and introduce contention with concurrent analytical queries.

To illustrate Spruce’s layout, consider the example graph in Figure 3.4, where the following edge insertions are applied sequentially: $0 \rightarrow 1$, $1 \rightarrow 2$, $0 \rightarrow 2$, and $0 \rightarrow 3$. As depicted in Figure 3.13, the edges $0 \rightarrow 1$ and $0 \rightarrow 2$ have already been moved into the sorted block of vertex 0 after buffer consolidation, while the most recent edge $0 \rightarrow 3$ remains temporarily in the buffer block. Vertex 1, which has only a single outgoing edge ($1 \rightarrow 2$), retains this edge in its buffer since the consolidation threshold has not yet been reached.

Spruce supports transactions and enables concurrent updates and analytical scans with snapshot isolation guarantees, also via an optimistic MVCC protocol inspired by Hyper [85]. Similar to Teseo and Sortledton, the latest versions of edges are maintained in sorted adjacency blocks for efficient access. However, unlike the per-edge version chains used in those systems, Spruce stores previous edge versions in compact per-vertex version logs. This design minimizes the overhead of maintaining individual version chains and improves cache locality.

3.2.8 Design comparison and discussion

The graph systems reviewed differ significantly in their internal designs, leading to various trade-offs between update performance, analytical efficiency, and concurrency control. As summarized in Table 3.1, core graph operations—insert, delete, and lookup—vary in asymptotic cost depending on the system’s data layout and indexing strategy. All systems are single-node, in-memory stores (with optional disk

System	Edge insertion	Edge deletion	Edge look-up
Llama, Stinger	$O(E)$	$O(E)$	$O(E)$
GraphOne	$O(1)$	$O(E)$	$O(E)$
LiveGraph	$O(1)$	$O(E)$	$O(1)$
Teseo	$O(\log(E))$	$O(\log(E))$	$O(\log(E))$
Sortledton	$O(\log(E))$	$O(\log(E))$	$O(\log(E))$
Spruce	$O(E)$	$O(\log(E))$	$O(\log(E))$

Table 3.1: Complexity comparison across systems

support in some cases via mmap), and fall into two broad classes: non-transactional and transactional MVCC-enabled systems.

Non-transactional systems such as STINGER, LLAMA, and GraphOne prioritize high-throughput edge updates, but with differing trade-offs in consistency and query freshness. LLAMA and GraphOne decouple ingestion from analytical queries by buffering updates in write-optimized structures (a delta map in LLAMA and a circular edge log in GraphOne), which are later merged into a read-optimized store through periodic snapshotting. While this design enables fast insertions, it delays the visibility of recent updates during analytics. In contrast, STINGER applies updates directly to its adjacency list without staging them in separate stores. Although this allows low-latency updates, STINGER lacks full thread safety during concurrent graph scans, which may result in inconsistent query results under concurrent updates. STINGER, LLAMA, and GraphOne all incur $\mathcal{O}(|E|)$ time complexity for edge insertions, deletions, and lookups. This is primarily due to the absence of secondary indexes on destination vertex IDs, which necessitates a linear scan over each vertex’s adjacency list to perform edge existence checks.

Among **transactional systems based on optimistic MVCC** (Teseo, Sortledton, and Spruce) enable fast access to the latest edge versions by storing them in sorted adjacency blocks. While efficient for snapshot-based analytics, this approach may lead to performance degradation under heavy update workloads due to transaction restarts caused by reader-writer contention. In contrast, LiveGraph adopts a timestamp-based MVCC model that preserves insertion order and minimizes contention by decoupling readers and writers through logical timestamps. This design can also efficiently support historical queries, as its storage layout preserves the ar-

rival order of updates and stores all edge versions contiguously in the Transactional Edge Log (TEL). In Teseo and Sortledton, adjacency lists are maintained in sorted order by destination IDs, enabling insertions, deletions, and lookups to be performed in $\mathcal{O}(\log |E|)$ time, as shown in Table 3.1. Spruce achieves similar $\mathcal{O}(\log |E|)$ complexity for deletions and lookups, but incurs a higher insertion cost of $\mathcal{O}(|E|)$ due to the merging overhead from the buffer block into a sorted block. LiveGraph supports $\mathcal{O}(1)$ edge insertion and lookup in the common case using Bloom filters as a secondary index for existence checks. However, in the presence of false positives, the cost may degrade to $\mathcal{O}(|E|)$ due to a full scan of the edge log. Deletion in *LiveGraph* requires locating the latest valid version of an edge, resulting in $\mathcal{O}(|E|)$.

3.3 Graph data management with updates propagated ooo

We now discuss the specific challenge addressed by the system we developed in this work: the correct handling of graph updates (edge insertions and deletions) that may reach the database in an order different from the temporal order in which they were emitted. This happens in many real-world streaming scenarios, particularly those involving distributed infrastructures such as IoT or vehicular networks [65, 37, 51]. This phenomenon is referred to as *ooo* updates. Each update carries two timestamps: the *stream time* (S_T), which indicates when the update was emitted by the source, and the *write time* (W_T), which marks when the update is received and applied by the graph database [45]. Due to factors such as multi-path routing [124, 71], link-layer retransmissions [40, 62], or variations in router forwarding delays [75], an update with an earlier S_T may arrive later than one with a later S_T —i.e., $S_T^1 < S_T^2$ but $W_T^1 > W_T^2$.

Figure 3.14 illustrates dynamic graphs with *ooo* update propagation on a graph (at the top) where for each edge, the database receives an insertion, and for most, it also receives a deletion. On each edge, we show the one or two possible operations, insertions (+) and/or deletions (-) *in the order of their arrival* at the database, together with their stream time S_T . For instance, for the edge between A and B, +1, -2 states that the insertion with $S_T=1$ arrives before the deletion with $S_T=2$. On the contrary, for the edge between B and C, the deletion emitted at 6 is received before the inser-

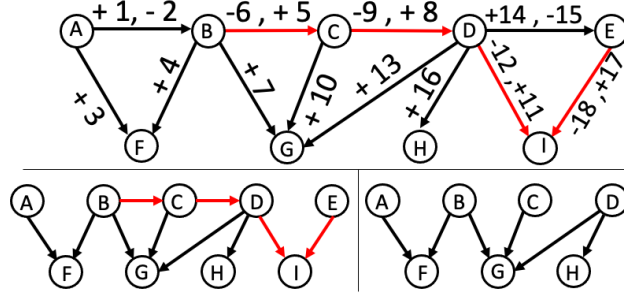


Figure 3.14: Sample dynamic graph with out-of-order updates.

tion emitted at 5. In existing dynamic graph systems, e.g., [64, 126, 41, 76, 58, 38], unaware of possible ooo updates, updates are processed in the order in which they are received on the database site; these systems assume that no deletion arrives in the database before its associated insertion. **Such a deletion is ignored**; the (delayed) edge insertion is applied, leaving the edge in the system. Instead, the deletion should be recorded even before the matching insertion is received; when the insertion arrives, the respective edge should be understood as having been inserted *then* deleted. In our example, after the updates arriving as shown at the top of Figure 3.14, the graph stored by a system that does not handle ooo insertions is the one depicted at the bottom left. However, **the correct final graph** (at the bottom right) accurately depicts the sequence of events based on the order of their generation; the two differ in the edges shown in red at the top, and which were transmitted ooo.

Existing solutions for **handling ooo updates** are well-studied in the **data streaming** literature. They fall into five main categories:

1. *Buffer-based approaches* [82, 54, 69, 43, 106] temporarily store updates in a buffer and sort them before insertion into the database. Query accuracy cannot be guaranteed for updates near the buffer boundaries. Buffering also increases latency, which conflicts with the goals of real-time analytics.
2. In *punctuation-based methods* [67, 68, 9], special markers (or “punctuation”) in the stream indicate that no ooo updates follow. This strategy also incurs delays, as the system must wait for the arrival of the marker before processing can proceed.

3. *Speculation-based approaches* [83, 24, 28] optimistically assume that all updates arrive in order and process queries and analytics on the resulting graph. However, when ooo updates are detected, rollbacks are required. This invalidates previous query results and complicates system behavior.
4. *Approximation-based systems* [1, 32, 107] provide bounded-error estimates for numerical queries. However, this approach is unsuitable for precise graph operations, such as shortest paths, which require access to an accurate and consistent snapshot of the graph state.
5. A naïve approach for handling ooo updates in dynamic graphs is to *maintain all edges in a stream-time ordered list*. However, this design conflicts with the access patterns of most graph operations. For example, graph traversals require random vertex access by ID, followed by a sequential scan over that vertex’s neighborhood. Maintaining stream-time order under ooo updates would require re-sorting nodes, significantly degrading ingestion throughput.

Among these strategies, only buffer-based approaches can be externally applied to existing systems without modifying their core. However, this comes at the cost of reduced query accuracy at the buffer boundaries. Additionally, buffering introduces extra latency due to temporary storage and sorting, which can conflict with the low-latency requirements of real-time analytics.

In contrast with the streaming setting, where relational data is used, **none of the graph databases surveyed above (and, in particular, none among the native, in-memory systems) are capable of handling ooo graph updates**. Even in a single-node or single-stream setting, updates can still arrive out-of-order at the graph database due to lower-layer network behaviors. In particular, multi-path routing allows packets from the same source stream to traverse different network paths, each potentially experiencing different latencies [124, 71], which can result in arrival re-ordering. Because existing systems assume that updates arrive in order, they are unable to handle such cases correctly.

In this thesis, we address the limitations of existing native in-memory graph stores by

designing a **transactional graph system balancing high write throughput with accurate analytical query execution in parallel, even in the presence of ooo updates**. We design and implement our system from scratch, based on a novel a storage layout that supports both present and historical snapshot queries, where historical queries are evaluated based on stream time (the original time at which updates were generated) rather than their database commit time. We describe our system in the next chapters. Our approach draws inspiration from the timestamp-based MVCC architecture employed by LiveGraph, which preserves the arrival order of updates. Furthermore, while our design is optimized for structural graph updates, it also supports arbitrary vertex and edge properties, making it suitable for a wide range of graph analytics workloads. To the best of our knowledge, our system is the first native in-memory transactional graph store to provide native support for out-of-order updates. Its positioning relative to prior systems is summarized in Table 2.1.

3.4 Summary

This chapter provides a comprehensive overview of recent in-memory graph database systems, focusing on their data structures, and design trade-offs. It begins by introducing the fundamental in-memory layouts, namely edge list, CSR, and adjacency list, and analyzes how these impact system performance for dynamic workloads. It then surveys representative native graph stores such as LLAMA [76], GraphOne [58], STINGER [38], LiveGraph [126], Teseo [64], Sortledton [41], and Spruce [102], highlighting their layout choices, how they balance high-throughput transactional updates with concurrent analytical scans, and the approaches used to achieve this. Particular emphasis is placed on the lack of native support for ooo updates in existing systems, which represents a critical gap for real-time streaming applications. This motivates the need for a novel system design, introduced in the next chapter (4).

Chapter 4

Core HAL Storage System

Overview

In this chapter, we describe the design of HAL, our in-memory graph storage engine built from scratch to support high transaction write throughput and accurate, efficient analytical scans in parallel, even in the presence of ooo updates. We begin with preliminaries in Section 4.1, where we define the underlying data model, a directed graph with properties on nodes and edges, and formalize key concepts such as in-order and ooo updates, as well as best-guess associations between insertions and deletions. These definitions establish the foundation for correct update handling in our system. Following this, we present a high-level overview of the HAL storage engine (Section 4.2), which consists of two main structures: a vertex array (VA) and a stream-time ordered adjacency list (STAL). We detail how HAL handles in-order and ooo insertions and deletions (Sections 4.5 and 4.6). Finally, Section 4.8 describes the steps for executing analytical queries on top of HAL.

4.1 Preliminaries

Data model We consider a directed graph G , which at any point in time consists of a set of nodes N and edges E . Each edge has a source and destination vertex from N . Each edge, and each node, may have attributes (or properties). Similar to other systems [64, 126, 41], we **store the graph topology separately** from node

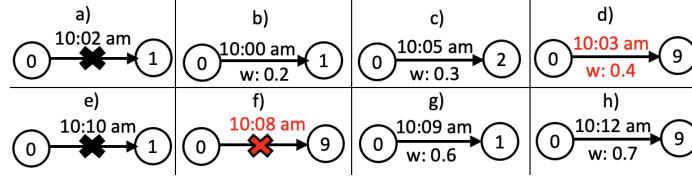


Figure 4.1: Sample dynamic graph.

and/or edge properties, and, in this work, we focus on efficiently handling **updates and queries on the graph topology**. For all such systems, data access paths based on node/edge properties can be added and optimized in a complementary way.

Out-of-order update An update can be judged in-order or ooo only at the database site. Specifically, update u with stream time S_T , received at the database at time W_T , is ooo if (i) u is a deletion and no insertion of the same edge has been received, and/or (ii) the database has already received at a time $W'_T < W_T$ an update with stream time $S'_T > S_T$.

Best-guess associated update At the database, insertions and deletions emitted from multiple streams are possibly received ooo. To build our current view of the graph, at any time, our system tries to **guesses** the associations between insertions and deletions of the same edge. Specifically, leveraging update stream times, to each insertion (respectively, deletion), we either:

- *associate the most likely deletion* (respectively, *insertion*); or
- *consider that none of the deletions* (respectively, *insertions*) *received so far is associated to this insertion* (respectively, *deletion*). In this case, the “missing” operation is either delayed (we will receive it later), or may never be emitted, e.g., some insertions are never followed by deletions.

When, for a given edge, more than one ooo update is received, *our best-guess associations between insertions and deletions may change*. We say then, that an ooo update may steal its associated update, from another. Sections 4.5 and 4.6 detail this.

4.2 Storage system overview

In this section, we present a high-level overview of our novel **History Adjacency List (HAL, in short)** data store. We opted for an *adjacency list design* for our graph stor-

age because it offers a good balance between data access locality, which is important for queries, and high ingestion throughput, which is important for high transactional throughput [41].

The HAL design layout resembles the one of LiveGraph [126], which preserves the arrival order of updates (as detailed in Chapter 3). Specifically, the HAL data layout offers novelty on two fronts: layout optimizations, and a novel strategy for associating the *most likely* insertion-deletion operation pairs, even when they arrive ooo. We describe these innovations next.

Five layout optimizations: The HAL data layout is novel in five ways, compared to the LiveGraph [126] data layout:

1. The arrival order of the adjacency list is maintained based on the stream time;
2. Multiple fixed-size adjacency list blocks are proposed rather than a single adjacency list, thereby reducing costly resize requests;
3. Edge entry metadata is stored separately from the destination IDs; this allows faster access to destination ID scans, which are predominant in analytical queries, thus improving analytical scan performance (details of (i),(ii),(iii)) are in Section 4.4);
4. We propose a novel strategy inspired by cracking [48] to minimize the data access needed to perform a per-edge deletion check during analytical scans concurrent with updates (we detail this in Section 4.8);
5. Garbage collection is performed using writer threads instead of background threads. This approach reduces contention between the background threads and reader/writer threads, leading to better memory management (as we detail in Section 4.6.1).

Most likely insertion and deletion association strategy: We use a secondary index (hash table) that efficiently and accurately associates the most likely insertion and deletion entries in the adjacency list, leveraging their stream time. It can dynamically adjust these associations if multiple ooo updates are received for the same edge,

thereby ensuring correct ordering of the adjacency list by stream time. We detail this in Section 4.3.

Throughout our discussion, an edge of the form $s \rightarrow d$ connects a *source* node with a *destination* node, called simply source and destination below. An *update* (or *entry*) is an insertion or deletion of an edge. To avoid confusions, we call *stream* an individual source of graph updates; *stream time* refers to the time local to that stream.

HAL stores and processes updates in an *adjacency list*, sorted by the source of the inserted/deleted edges; the list is the entry point for any read and write. HAL is *append-only*: insertions and deletions only add data to the store. To keep its size under control, however, HAL content can be *garbage collected* (as we will discuss in Section 4.6.1). HAL aims to *maximize efficiency for in-order updates*, which we expect to be more frequent than ooo ones; if and when needed, dedicated fields are created within HAL to store and exploit ooo updates efficiently. Finally, HAL aims to support graph analytics concurrently with updates from multiple streams.

HAL has two main structures: a *vertex array* (VA, in short), and a *stream time-sorted adjacency list* (STAL, in short), each holding an entry for each graph vertex, denoted, respectively, $VA[s]$ and $STAL_s$. Figure 4.1 illustrates successive states, labeled a) to h), of a **sample graph**. Each edge depicts an update; if it is crossed, it is a deletion, otherwise, an insertion. On each edge we show its stream time, and an edge property, e.g., its weight w . On ooo edges, the stream time is shown in **red**; the cross is also red for ooo deletions. We describe the VA, respectively, STAL in Section 4.3 and 4.4.

4.3 The vertex array (VA)

Each **vertex array (VA)** entry $VA[s]$ consists of several fields. Figure 4.2 illustrates it for the source vertex s having the ID 0. In the figure, a dash (—) designates an empty field (no known value), whereas “...” values or fields whose details we omit, for simplicity. Finally, shaded areas denote metadata (of constant size), which we will introduce as needed, to explain our algorithms.

1. A reference to the STAL entry corresponding to this vertex;
2. The **latest stream time (LST)**, in short) of an in-order update received so far for the source vertex s . For instance, in Figure 4.1, the latest in-order insertion with

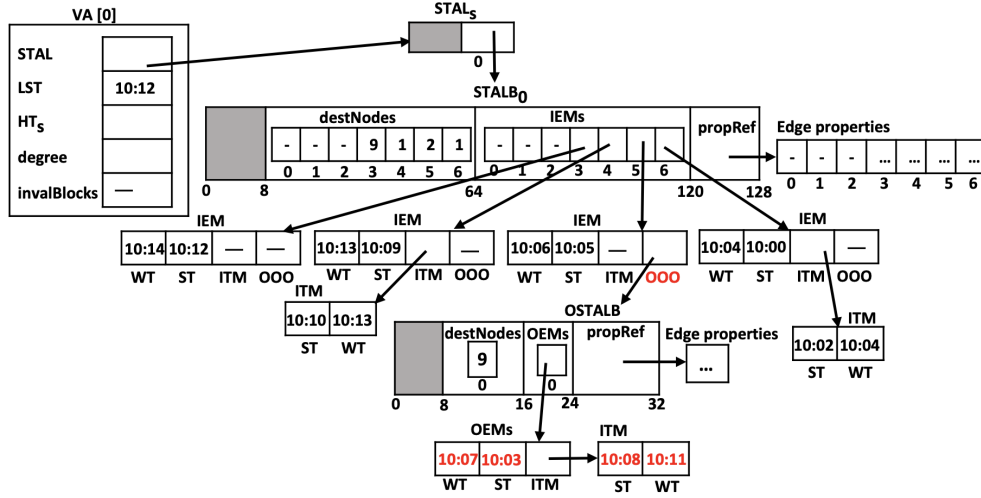


Figure 4.2: Sample vertex entry $VA[0]$.

source 0 is $0 \rightarrow 9$ at 10:12 in h); hence, in Figure 4.2, $VA[0]$ has 10:12 as LST.

3. **degree**: the number of edges inserted but not deleted, currently in $STAL_s$;
4. **invalBlocks** references $STALBs$ (see below) with edges whose source is s , and which should be garbage-collected (cf. Section 4.6.1).
5. A **hash table** (HT_s , in short) whose keys are the destination vertices d for which we have received some $s \rightarrow d$ updates, and whose values we detail below;

Figure 4.3 illustrates the evolution of HT_0 . Each yellow area encloses the data structures created as a consequence of the respective entry in Figure 4.1; dashed arrows trace the evolutions data structures go through with each step, while solid arrows show references among data structures. For a given destination d , HT_s stores, as the graph evolves, *one among the following three data structures*.

Update position and indicator (UPI, in short): The UPI of $s \rightarrow d$, denoted $UPI[s, d]$ encodes the position of the *latest (in-order or ooo) $s \rightarrow d$ insertion* in $STAL_s$. In Figure 4.1, the first insertion for the edge $0 \rightarrow 2$, at c), is in-order, with stream time 10:05. Thus, we create an in-order UPI for destination 2 in HT_0 with this stream time, as shown in Figure 4.3. The dark grey UPI fields in this figure denote the information *encoding the position of the insertion within $STAL_s$* ; this encoding is not immediate, because the block holding an insertion may grow or move, as

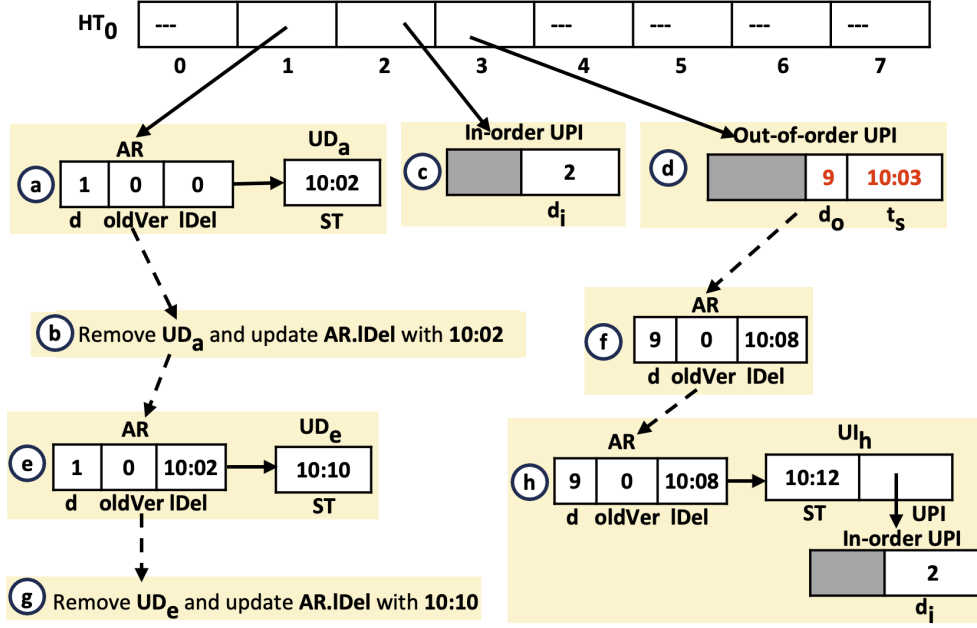


Figure 4.3: HT_0 through insertions and deletions.

the graph changes, yet the UPI always provides constant-time access to the insertion. UPIs and their decoding are detailed in Section 4.7.

Staging area (AR, in short): The AR for an edge $s \rightarrow d$ stores *insertions (deletions) of this edge, waiting for their corresponding deletions (insertions)*. For instance, in Figure 4.3, when we receive a deletion of $0 \rightarrow 1$ at a), HT_0 does not hold a corresponding insertion; we create the staging area for $0 \rightarrow 1$, with an *update deletion block* UD_a , storing the stream time 10:02 of this deletion. Then, we store the AR at position 1 in HT_0 . When the deletion f) of $0 \rightarrow 9$ arrives at 10:08, we find the corresponding insertion in HT_0 at index 3, and replace the ooo UPI of $0 \rightarrow 9$ with a corresponding AR. Details of AR processing are delegated to Section 4.5 and 4.6.

Last garbage collected deletion (LGCD, in short) for vertex s is the stream time of the most recent deletion of $s \rightarrow d$ that has been garbage-collected (Section 4.6.1).

$HT_s(d)$ through the lifecycle of the edge The roles of these different data structures can be understood by examining the *lifecycle* of an edge $s \rightarrow d$ in our system.

- Only when the first update ever to be received by the database for $s \rightarrow d$ is an insertion, we create a UPI, as we did above for the insertion $0 \rightarrow 2$ in Figure 4.1 c).
- An AR exists as long as we have *one or more insertion(s) (respectively, deletion(s)) for which we did not yet receive corresponding deletion(s) (respectively, insertion(s))*. The operation we received *waits* for the one not yet received, in the AR. This concerns: in- or ooo insertions, for which we have not received a deletion; in the future, we may receive such a deletion, or not; and, every deletion for which the current STAL does not contain a possible associated insertion. This happens when all the insertions prior to our deletion, have already been marked as deleted (by other deletion entries).
- An LGCD is created when updates $s \rightarrow d$ have been garbage collected (Section 4.6.1).

VA ensures constant-time access to edges by their source vertex s , for one or multiple parallel threads.

4.4 The stream-time ordered adjacency list (STAL)

For each source vertex s , $STAL_s$ is an append-only list of blocks, storing updates going from s to various destination vertices. $STAL_s$ has some **metadata** (shaded area in Figure 4.2), notably: a **hasDeletes** flag which is true if deletions have been received for source vertex s , and **delNo** the number of slots freed by successive deletions. $STAL_s$ also stores the **stream-time ordered adjacency list (STAL**, in short), which references one or more STAL blocks.

Each **STAL block (STALB)** comprises:

1. **metadata** (shaded in Figure 4.2), notably the boolean flags **hasDeletes** and **hasOOO**, indicating whether the STALB contains, respectively, deletions, or ooo updates, and the number of deleted entries in the STALB;
2. **destNodes** holds $s \rightarrow d$ entries for the given s and various destinations d , sorted in descending stream time order.
3. **IEMs** stores references to **in-order edge entry metadata (IEM** in short, see below) entries, one for each edge update in **destNodes**;

4. **propRef** references a vector of the same size as destNodes and IEMs, storing, for each of the above edge updates, the properties that the edge may have, e.g., edge weight, etc. *Note that the edge properties vectors are not part of $STAL_s$, and actually not even part of our HAL structure; we shown them in the figure to provide the reader with a global picture.*

For instance, in Figure 4.1, we receive insertions of edges $0 \rightarrow 1$, $0 \rightarrow 2$, and $0 \rightarrow 9$ in steps b), c), d), g), h); in Figure 4.2, the destination ids of these updates are stored in destNodes, and the associated metadata in IEMs. For instance, Figure 4.2 illustrates the STALB labeled S_0 , over 128 bytes: 8 bytes for metadata (at the left in gray color), 8 bytes for edge properties (at the right), 56 bytes (7 entries) for destNodes and 56 bytes (7 entries) for IEMs.

Importantly, **STALBs within $STAL_s$ are sorted by stream time in descending order**: let S_0, S_1 be two STALBs such that S_0 appears in $STAL_s$ *before* S_1 . Then, any entry in S_0 is more recent (has a higher stream time) than any entry in S_1 .

Each **IEM** (in-order entry metadata) contains:

1. The **write (transaction) time (WT)**, that is, the time when the entry is received at the database site;
2. The **stream time (ST)** when the entry is emitted by its stream;
3. The **invalidation time metadata (ITM)** of an insertion entry stores information about *the edge deletion most likely associated to this insertion* (recall Section 4.1), if one exists:
 - The stream time S_T of the deletion entry;
 - The transaction time W_T of the deletion entry;
4. The **ooo updates (OOO)** field, initially empty, references a data structure storing ooo insertions *with the same source vertex, and the same or different destination* as the one for of the IEM, whose stream time is less than the IEM's ST, but higher than the STs of the previous IEM (if any). The data structure will be detailed in Section 4.5.1. We store ooo insertions in the same ST order as the in-order ones, but not at the same level. The reason is that in-order insertions

are simply appended in the block holding the most recent insertions (thus, in constant time). To store an ooo insertion's destination and metadata in $STAL_s$ in the order dictated by its ST, some previously received in-order information may need being recopied, a cost we avoid.

For each arriving *in-order insertion*, we create an IEM; for each *ooo insertion*, we create an **ooo entry metadata (OEM)**, resembling an IEM, but without the OOO field. For each *deletion*, we create an ITM, and reference it from the IEM or OEM of the insertion so far received, that is most likely associated to this deletion. Recall from Figure 4.1, the insertions and deletions of edges $0 \rightarrow 1$, $0 \rightarrow 2$, and $0 \rightarrow 9$; Figure 4.2 shows the corresponding data structures created in $STAL_0$. The ITM with ST 10:10 referenced by the IEM with ST 10:09 shows that we pair the deletion of $0 \rightarrow 1$ at 10:10 in Figure 4.1 e) with the insertion at 10:09 in g). The ITM with ST 10:08 referenced by the OEM with ST 10:03 means that for the deletion of $0 \rightarrow 9$ at 10:08 in f), the most likely insertion is at 10:03 in Figure 4.1 d).

Remark. In STAL, a deletion is only stored as an invalidation (ITM) of its most likely insertion. If, upon receiving the deletion, we cannot find such an insertion, the deletion enters the staging area AR, but is not visible in STAL.

The STAL structure ensures the update entries for a given source vertex s are stored *in the descending order of their stream time*. As we will discuss, this ensures $O(1)$ complexity for the operations we expect to be most common: in-order insertions and deletions. Data structures within STAL, in particular the AR, IEMs, OEMs, and ITMs, enable us to provide efficient graph querying with snapshot isolation guarantees even in the presence of ooo updates.

The design principles behind the HAL store (VA and STAL) aimed to strike a good balance between data access locality, which is important for queries, and high ingestion throughput, which is important when the graph changes. This enables both random vertex access and sequential neighborhood access, making it efficient for a range of graph analytics tasks.

In Section 4.5 and 4.6, we provide details of the HAL's internal operations (insertion and deletion), while Section 4.8 discusses the concurrent execution of analytical queries and updates.

4.5 Insertion

A new edge (insertion) entry $s \rightarrow d$, called in this section **the new entry**, arrives at the database site at transaction time W_T , having the originating stream time S_T . We process it as follows:

1. *Locate* in the VA the position corresponding to s , e.g., if the $0 \rightarrow 1$ arrives at 10:00 am (W_T), this is the $VA[0]$.
2. *Lock* this VA entry to prevent conflicts between several write-transactions having the source vertex s .
3. *Compare* the new entry's S_T to the LST value in $VA[s]$.
 - (a) If $LST < S_T$, then $s \rightarrow d$ is an *in-order* update. Section 4.5.1 explains how we handle in-order insertions.
 - (b) Otherwise, it is an *ooo* update; Section 4.5.2 describes how to handle these.
4. *Unlock* $VA[s]$.

At step (3) above, we consider the new entry in-order or ooo by comparing its stream time S_T "only" with the last insertion time in $VA[s]$, while our definition of ooo updates (Section 4.1), more broad, would require comparing S_T with the highest stream time of any operation (insertion or deletion) received so far at the database, regardless of the vertices involved. The condition we test is sufficient to ensure that *at any point, for each source vertex, we can return all its outgoing edges, correctly reflecting all updates received so far, in- or ooo*. Indeed, given that our store is organized by the source vertex, our test suffices to place the new insertion at the right place (dictated by S_T) among all insertions with the same source. Further, any deletions of edges with source vertex s also accumulate in $STAL_s$ and thus will meet our new entry there, ensuring a correct view on our store at any point.

4.5.1 In-order insertion

Since $STAL_s$ stores most recent entries first, we add the new entry to its first $STALB$. If this $STALB$ is full, its size doubles, up to reaching a system-wide upper bound

maxBlockSize. When a STALB has reached this size, the STAL grows by *doubling* the number of blocks, from the initial block to 2, 4 etc. We detail the process since it is important for efficiently finding insertions in the STAL (Section 4.7).

In each doubled-up $STAL_s$, the full STALBs make up the second half of the list. Thus, if we denote full STALBs by gray-filled boxes and blocks with free space by white-filled ones, $STAL_s \rightarrow \boxed{B_0}$ becomes:

- $STAL_s \rightarrow \boxed{B_1} \rightarrow \boxed{B_0}$, then
- $\boxed{B_3} \rightarrow \boxed{B_2} \rightarrow \boxed{B_1} \rightarrow \boxed{B_0}$; here, we store new entries in B_2 , the new STALB closest (in ST order) to B_1 ; $STAL_s$ references the block holding the most recent insertion, thus $STAL_s \rightarrow \boxed{B_2}$.
- When B_2 fills, this becomes $STAL_s \rightarrow \boxed{B_3} \rightarrow \boxed{B_2} \rightarrow \boxed{B_1} \rightarrow \boxed{B_0}$, etc.

We denote by $\boxed{STALB_e}$ the STALB where the new entry is stored, by $\boxed{d_e}$ the new entry's position within $STALB_e$'s entries, and by $\boxed{IEM_e}$ the in-order edge metadata (IEM) created to reflect this insertion (recall Figure 4.2), storing S_T and W_T .

Next, to ensure constant-time access to the newly inserted entry, we create the **Update and Position Indicator (UPI)** for it, call it $\boxed{UPI_e}$, and insert it into HT_s . The UPI stores *information based on which e can be located in $STAL_s$ in the future, even if $STALB_e$ grows, is resized, or moved away, potentially many times, from the head of $STAL_s$.* Specifically, UPI_e stores: the position of the new entry in $STALB_e$; the position of $STALB_e$ in $STAL_s$; and the sizes of $STALB_e$, and $STAL_s$, when the new entry was inserted.

When processing our insertion, four possibilities arise, depending on the previously received updates for $s \rightarrow d$:

- HT_s contains *no entry* for $s \rightarrow d$ (the new entry is the first arrived for this edge): the above steps suffice.
- HT_s contains an UPI for an older insertion $s \rightarrow d$, call it $\boxed{UPI_o}$. Between the insertions described by UPI_o and UPI_e , a deletion, not received yet, may or may not have been sent. To prepare the possible future processing of that ooo deletion, and to enable correct query answering before and after receiving it, UPI_o

must be marked as a previous version of UPI_e ; Section 4.5.1.1 details this.

- HT_s contains a *staging area (AR)* (recall Section 4.3) for $s \rightarrow d$; inserting the new entry is discussed in Section 4.5.1.2.
- HT_s contains a *latest garbage collected deletion (LGCD)*; we describe the associated processing in Section 4.5.1.3.

4.5.1.1 In-order insertion over existing UPI

Inserting over a UPI means an ooo deletion may arrive. We need to update UPI_o to indicate *it is the previous version of UPI_e* , and to mark its entry as deleted.

1. From UPI_o , we locate the previous $s \rightarrow d$ insertion entry, call it e_o : block $STALB_o$, holding e_o at position p .
2. At position p in $STALB_o$'s metadata array IEMs, we find the entry metadata, call it EM_o , associated to e_o : EM_o is an IEM if e_o was in-order, and an OEM otherwise.
3. To record e_o as deleted (by the update not yet received), we create an Invalidation Time Metadata (ITM) initialized with S_T and W_T , and the ITM to the EM_o metadata of UPI_o .
4. In $STALB_o$'s metadata, we set `hasDeletes` to true.

Receiving an insertion when the previous operation was also an insertion, means that both of them need to wait for their possible deletions, in a staging area. In details:

- We create a new AR, call it AR_e with: the destination id d , the latest deletion time (`IDel`) set to 0 (such a deletion has not arrived yet), `oldVer` set to 0 (no garbage collection has claimed a version of $s \rightarrow d$ so far).
- We create two update insertion blocks UI_o and UI_e , for the previous and new entry, with the ST (stream time) from the corresponding metadata: EM_o and IEM_e . We chain the blocks in AR_o in increasing ST order: $UI_o \xrightarrow{\text{next}} UI_e$.
- AR_e replaces UPI_o as the HT_s value for d .

Note that UI_o , UI_e reference UPI_o and UPI_e , which thus remain accessible. When a deletion arrives, first, we pair it with an insertion from AR, and then, use the UPI to locate the insertion entry in STAL to mark its deletion (Section 4.6.2).

4.5.1.2 In-order insertion over existing AR

Let AR_o be the AR for $s \rightarrow d$, stored in $STAL_s$ when the new entry arrives. We first check whether a deletion entry, emitted after e , has already been received at the database. For that, we compare the new entry stream time S_T with $AR_o.IDel$, the stream time of the last deleted $s \rightarrow d$ update (recall Section 4.3). For brevity, we just denote it $IDel$ below.

1. If $S_T < IDel$, a deletion of $s \rightarrow d$ has already been received, following (in stream time order) the new (insertion) entry we are processing now.
2. If $S_T \geq IDel$, then $STAL_s$ has no record of a deletion following e . However, *deletion which we cannot pair with insertions when they arrive, are only recorded in the AR, and not in $STAL_s$* (as will be detailed in Section 4.6). Thus, the staging area may contain an update deletion block UD_o whose stream time is above S_T .
 - (a) If such an UD_o does not exist, the new entry should be added to the store.
 - (b) If UD_o exists, it deletes the new entry e .

In cases (1) and (2b), we **mark the new entry e as deleted** in $STAL_s$, much like in Section 4.5.1.1, except that the AR is already created; we also increase $IDel$ to S_T , reflecting the most recent deletion. Further, in case (2b), we remove UD_o from the AR.

In case (2a), we **create a new update insertion block UI_e** , and insert it in the AR at the right place, based on its S_T . It may happen that UI_e is preceded, in the AR, by a prior insertion materialized by a UI_{prev} block. In that case, the UPI corresponding to UI_{prev} is marked in $STAL_s$ as a previous version of UPI_e , just like we did for UPI_o in Section 4.5.1.1.

4.5.1.3 In-order insertion after GC

Garbage collection (GC) empties the staging area, replacing it with an LGCD. Thus, the first entry after GC needs to restart a staging area. We create a new update insertion block UI_e with S_T and UPI_e ; we create the staging area AR_e whose `oldVer` is copied from the LGCD, and insert UI_e as the next block after AR_e . Finally, AR_e replaces the LGCD in HT_s for destination d . **The complexity** of handling an in-order insertion is $O(1)$, since each step takes only constant time.

4.5.2 Out-of-order insertion

To propagate the out-of-order insertion e in $STAL_s$, we create an out-of-order edge entry metadata (OEM) block OEM_e with S_T as source time, W_T as transaction time. Then, based on S_T , we search in $STAL_s$ for the out-of-order STAL block, call it $OSTALB_e$, where the new entry e fits; our search will first visit one or more STAL blocks (recall Figure 4.2). (i) If $OSTALB_e$ does not exist, we create it with size 1, and connect it to the proper STALB, whose flag `hasOOO` is set to true. (ii) If $OSTALB_e$ exists, if it is full, we double it up like we did for STAL blocks, except that upon reaching a maximum $OSTALB_e$ size, $OSTALB_e$ splits in two smaller blocks, which become the two initial leaf nodes in an Adaptive Radix Tree [63] ART , ordered according to update stream time. From now on, out-of-order updates in $STAL_s$ will be stored in the ART. We add d in $OSTALB_e$'s `destNodes`, and OEM_e in its `OEMs`.

Next, we reflect the insertion in HT_s , through a new out-of-order UPI, denoted UPI_e , initialized with S_T and d . Recall that UPI_e for $s \rightarrow d$ is out-of-order with respect to other destination edges; however, it may be possible that UPI_e is in order with respect to its own edge $s \rightarrow d$. Thus, we follow four possibilities, as in Section 4.5.1, depending on what HT_s stores for d when our insertion arrives:

- *No entry* (e is the first $s \rightarrow d$ insertion ever arrived): we add UPI_e to the hash table HT_s and the processing of HT_s is finished.
- An *UPI* for a previous $s \rightarrow d$ insertion, call it UPI_o . An ooo deletion, not yet received, may or may not have been sent between the insertions described by UPI_o and UPI_e . We discuss this case in Section 4.5.2.1.

#	STAL	AR _o (before)	AR _o (after)
1	i_1	UD ₂	No entry
2	$i_1 i_3 i_5$	UI ₁ $\xrightarrow{\text{next}}$ UI ₅	UI ₁ $\xrightarrow{\text{next}}$ UI ₃ $\xrightarrow{\text{next}}$ UI ₅
3	$i_1 d_2 i_3 i_5$	UI ₅	UI ₃ $\xrightarrow{\text{next}}$ UI ₅
4	$i_1 i_3 i_5$	UI ₃ $\xrightarrow{\text{next}}$ UI ₅	UI ₁ $\xrightarrow{\text{next}}$ UI ₃ $\xrightarrow{\text{next}}$ UI ₅
5	$i_1 i_3 d_4$	UD ₆	UI ₁ $\xrightarrow{\text{next}}$ UD ₆
6	$i_1 i_3$	UD ₂ $\xrightarrow{\text{next}}$ UI ₃	UI ₃
7	$i_1 i_3 d_4$	UD ₆	UI ₁ $\xrightarrow{\text{next}}$ UD ₆
8	$i_1 i_3 d_4$	No entry	UI ₁
9	$i_1 d_2 i_3 i_5 d_6$	No entry	UI ₃
10	$i_1 i_3 i_5 d_6$	UI ₁	UI ₁ $\xrightarrow{\text{next}}$ UI ₃

Table 4.1: Possible cases of ooo insertion over existing AR.

- An AR for $s \rightarrow d$; this means some insertions and/or deletions of $s \rightarrow d$ are waiting for their matching deletions (respectively, insertions). We detail our algorithm in Section 4.5.2.2.
- An LGDV (latest garbage-collected destination version); we describe the associated processing in Section 4.5.2.3.

4.5.2.1 Out-of-order insertion over existing UPI

If S_T is above UPI_o's stream time (e is emitted after the previously received insertion), we mark UPI_o as the previous version of UPI_e in STAL_s, as we did for UPI_o in Section 4.5.1. Otherwise, we mark UPI_e as a previous version of UPI_o. Then, we create a new staging area AR_e, where UPI_o and UPI_e wait for their possible, associated deletions. We create two update insertion blocks UI_o, UI_e and, depending on their stream times, we chain them as UI_o $\xrightarrow{\text{next}}$ UI_e or UI_e $\xrightarrow{\text{next}}$ UI_o in AR_e. Then, AR_e replaces UPI_e as the HT_s value for d .

4.5.2.2 Out-of-order insertion over existing AR

Let $\boxed{\text{AR}_o}$ be the AR for $s \rightarrow d$. Our new, ooo entry e may be older than the most recent garbage-collected $s \rightarrow d$ entry; we test this by comparing S_T with AR_o.oldVer. If S_T is smaller, e had already been *assumed* (even before it was received), and *garbage-collected* together with a subsequent deletion entry. In this case, we ignore e .

Our next steps depend on the entries (deletions and/or insertions of $s \rightarrow d$) in

$STAL_s$ and AR_o . We illustrate our analysis with simple examples in Table 4.1, showing, for each case, the updates in $STAL$ for $s \rightarrow d$, as well as AR_o before and after the insertion. We show each $STAL$ insertion as i and deletion as d , with their stream time as an index, e.g., i_3 is an insertion with $ST=3$. Our new (ooo) entry appears in red. Each AR block is also indexed with its stream time. We use small, consecutive times for illustration only.

We check whether a deletion entry, that would be most likely associated to e , has already been received.

If $S_T \geq AR_o.IDel$, the stream time of the last $s \rightarrow d$ deletion, such a deletion is not in $STAL_s$. However, it may be in AR_o (recall the **Remark** at the end of Section 4.4), under the form of a deletion update block, call it UD_o , with an ST above the new entry's S_T .

- If UD_o exists, we mark e in $STAL$ as deleted by UD_o (through an ITM referenced from OEM_e). We remove UD_o from AR_o (case 1 Table 4.1).
- Otherwise, it is the new insertion who joins the staging area AR_o : we create a new update insertion block UI_e , and insert it in AR_o according to its stream time S_T . We look in $STAL$ for two insertion entries whose ST are *the closest, before and after* S_T : call them i_{prev} , respectively, i_{next} .
 - If i_{prev} exists but has no deletion in $STAL$, we mark i_{prev} as the previous version of e (case 2 in Table 4.1).
 - If i_{prev} exists and it has a deletion in $STAL$, there is nothing more to do: e does not change our current most likely insertion-deletion pairs. This is case 3 in Table 4.1.
 - If e_{next} exists, we mark e as the previous version of e_{next} . This is case 4 in Table 4.1.

If $S_T < AR_o.IDel$, the deletion for e may be either in $STAL$ or in AR_o . In $STAL$, we search for i_{prev} and i_{next} as above.

- If a deletion UD_o waits for the new entry in AR_o :

- If i_{prev} exists, it has a deletion d_{prev} in STAL with a stream time above S_T , we mark e as deleted by d_{prev} , and we mark i_{prev} as the previous version of e . *Because d_{prev} is now associated to e , the previous entry i_{prev} "has lost its most likely deletion", and must wait for the possible future arrival of its deletion:* we create an update insertion block UI_{prev} with i_{prev} 's stream time, and insert it AR_o . This is illustrated in case 5 in Table 4.1.
 - If i_{next} does not exist, or it exists and its stream time is above that of UD_o , this indicates the waiting deletion UD_o should be paired with e . We remove UD_o from AR_o and mark e as deleted by UD_o . This is case 6 in Table 4.1.
 - If i_{next} exists and its stream time is below that of UD_o , the waiting deletion UD_o is closer to i_{next} than to the new entry. We mark e as a previous version of the e_{next} , and make it wait by creating a new UI_e with stream time S_T , and storing it in AR_o . Case 7 in Table 4.1 illustrates this.
- If UD_o does not exist (no deletion was waiting for e in AR_o):
 - If i_{prev} exists, and it has a deletion d_{prev} in STAL with a stream time above S_T , the new entry *steals* the d_{prev} , and we mark i_{prev} as the previous version of e . Now, i_{prev} must wait: we create UPI_{prev} and insert it in AR_o at the proper place. This is illustrated in case 8 in Table 4.1.
 - If i_{prev} exists, and it has a deletion with stream time below S_T , then e must wait for its possible own (later) deletion, which may or may not have been emitted by some source: we store a new UI_e in the AR. This is case 9 in Table 4.1.
 - If i_{prev} exists but it has no deletion in STAL, we mark i_{prev} as the previous version of e . We store a new UI_e for the new entry in the AR. Case 10 in Table 4.1 illustrates this.
 - If i_{next} exists, we mark the new entry as a previous version of e_{next} . This is also shown in case 10 in Table 4.1.

4.5.2.3 Out-of-order insertion after GC

Garbage collection (GC) deletes the version history, replacing it with an LGDV block. Thus, the first entry after GC needs to restart the history for $s \rightarrow d$.

The process here is as for in-order insertions (Section 4.5.1.3): we reopen the staging area for the new entry to wait there. We create a new update insertion block UI_e with S_T and UPI_e ; we create the history head block AR_e whose oldest-version is copied from the LGDV block, and insert UI_e as the next block after AR_e . Finally, AR_e replaces the LGDV in HT_s for destination d .

The complexity of an ooo insertion is $O(\log(|E|))$, in case we need to search in AR_o (sorted by ST).

4.6 Deletion

We now consider a deletion entry $\boxed{e}$ of the edge $s \rightarrow d$ with the stream time $\boxed{S_T}$ and received at transaction time W_T .

We lock $VA[s]$ to prevent conflicts with other updates having the source vertex s . Next, garbage collection (described in Section 4.6.1) may trigger. Then, depending on what $STAL_s$ holds for d :

No entry : Start a staging area AR_e in which the deletion waits, under the form of an update deletion block UD_e with stream time S_T . Store AR_e in HT_s at position d .

LGCD : Reopen the staging area AR_e in which we make the new deletion wait, as above. Copy the LGCD into $AR_e.oldVer$, then store AR_e in HT_s .

AR : If S_T is smaller than $AR_o.oldVer$, e is in the past with respect to insertions and deletions already garbage-collected (e had already been assumed); there is nothing left to do.

If $S_T \geq AR_o.IDel$, we search AR_o for an insertion that might have been waiting for e . This insertion block, call it $\boxed{UI_{prev}}$, should have an ST lower than that of e , but higher than $AR_o.IDel$: an insertion with an ST behind IDel is followed first, by a deletion at IDel, and only later by e (thus, such an insertion cannot be paired with e).

- If UI_{prev} exists, e deletes its corresponding insertion. From UI_{prev} , we access its UPI, call it UPI_{prev} , which may denote an in-order or ooo insertion. We mark UPI_{prev} as deleted in the $STAL_s$, as shown in Section 4.6.2 if UPI_{prev} is in order, or as in Section 4.6.3 if it is ooo. Having paired UI_{prev} with a deletion, we remove it from AR_o . We set $AR_o.IDel$ to S_T .
- Otherwise, we create an update deletion block UD_e , with stream time S_T , and insert it in AR_o .

If $S_T < AR_o.IDel$, we search $STAL_s$ for the closest insertion entry i_{prev} , whose stream time precedes S_T .

- If i_{prev} exists, and it has an associated deletion d_{prev} :
 - If $d_{prev} > S_T$, the current deletion e is closer to i_{prev} than its associated deletion was. Thus, e *steals* i_{prev} , and d_{prev} (re)joins the staging area, to wait for its insertion. We create an update deletion block UD_{prev} with the stream time of d_{prev} , insert it in AR_o , and mark the i_{prev} as deleted by the new entry e .
 - Otherwise, e is certainly not the deletion of i_{prev} , given that d_{prev} was emitted before e . Thus, e needs to wait for its insertion. We insert in AR_o a new update deletion block for e , with stream time S_T .
- If i_{prev} exists, but it has no deletion, we associate e as the deletion of i_{prev} : mark i_{prev} as deleted by e at S_T , and remove UI_{prev} from AR_o , since it has met its deletion.
- If i_{prev} does not exist, e needs to wait for its insertion; we create an update deletion block with stream time S_T , and insert it in AR_o .

UPI Let $\boxed{UPI_o}$ be the previous UPI. To apply the deletion in $STAL_s$, we follow the steps in Section 4.6.2 if UPI_o is in order, respectively, in Section 4.6.3 if it is ooo. In both cases, we then create a staging area AR_e , where e waits for its insertion, and replace UPI_e with AR_e in the HT_s entry at position d .

After the deletion is completed, we unlock $VA[s]$.

4.6.1 Garbage collection

For each block B referenced in $VA[s].invalBlocks$, we compare the block's WT to a system-wide **least recent arrival time among the currently running queries** LQ_T to prevent garbage-collecting entries on which the query with LQ_T may still be working. LQ_T is initialized to 0 when creating the store and is set to Q_T , the least query time among the currently running queries in the database whenever a query arrives.

If $B.WT < LQ_T$, we resize B down to its useful half:

- Create a new block B' (STALB or OSTALB, like B), whose size is half that of B .
- Copy the B entries that are not deleted into B' .
- In the staging area $AR_{s,d}$ for s and d , set oldVer to be highest ST among all deletions of $s \rightarrow d$ in B .
- If $AR_{s,d}$ is empty, and its oldVer is the stream time of the most recent entry for $s \rightarrow d$ in the STAL, this means no entry for $s \rightarrow d$ is still waiting for its corresponding operation, and our store is in a stable state wrt this edge. In this case, replace $AR_{s,d}$ with an LGCD storing oldVer.
- Replace B with B' in $STAL_s$.

4.6.2 Deletion of an in-order insertion

Recall from Section 4.5.1 that an UPI created for an in-order insertion o stores information based on which we can locate the STAL block B holding the insertion, and that insertion's metadata within B , call it IEM_o . To handle the deletion of the in-order insertion:

- We create an ITM block, with the stream time S_T of the deletion, and the current W_T , and reference it from IEM_o .
- In B 's metadata (Section 4.4), we set hasDeletes to true, and increase delNo.
- If delNo reaches 50% of B 's size, we mark B for (future) garbage collection, by adding a reference to it, together with the current transaction time W_T , in $VA[s].invaBlocks$.

The complexity of the above steps is $O(1)$.

4.6.3 Deletion of an ooo insertion

Based on UPI_o , we find the OSTALB block B in STAL_s hosting the ooo insertion; B may be referenced from a STALB, as shown in Figure 4.2, or we may find it by traversing an ART (recall Section 4.5.2). UPI_o also stores the position p of the ooo insertion within B .

- We find OEM_o , the metadata for the insertion, in $B.\text{OEMs}$ at position p .
- Create an ITM block ITM_e with S_T and W_T , and reference it from OEM_e , to indicate that the insertion has been deleted.
- Update B 's by setting `hasDeletes` to true, and increment its `delNo` counter.
- If number of deleted entries in B reaches 50% of its size, we add a reference to B , with the transaction time W_T , in $\text{VA}[s].\text{invalBlocks}$, for future garbage collection.

The complexity is $O(\log(|E|))$, due to searching in the ART.

4.7 HT_s and UPI details

As shown in Sections 4.5 and 4.6, our entry insertion and deletion algorithms repeatedly rely on finding, in HT_s , the entry (UPI, AR, or LGCD) for destination vertex d . Also, these algorithms need to find, based on UPI_e , created for a past insertion entry e , the location in STAL_s where information about this insertion is stored, e.g., the properties of that edge, whether the edge has been deleted, etc. In this section, we detail how the HT, and UPIs, support these operations in constant time.

The HT is created with 2 entries, and doubled up whenever 50% of its entries are taken. It holds exactly one entry in each bucket, and uses the *open addressing* technique [73], as follows.

- When an UPI is created for the *in-order* insertion of $s \rightarrow d$, the UPI is stored in HT_s in the bucket (d modulo current size of HT_s). If this position is already taken, the UPI is stored in the first free position following the initial one.

- An ooo insertion of $s \rightarrow d$ is first stored in STAL_s (Section 4.5.1). It is recorded in stream-timestamp order, attached to a previous in-order $s \rightarrow x$ insertion, as illustrated in Figure 4.2. Note that such an $s \rightarrow x$ insertion is guaranteed to exist: if $s \rightarrow d$ were the first insertion ever with source s , it would not be considered out-of-order. Then, the UPI created for the ooo insertion $s \rightarrow d$ is inserted in HT_s at the position computed as (d modulo current size of HT_s). If this bucket is already occupied, the UPI is inserted into the first available free position following it.
- The AR and/or LGCD replacing an UPI keep the UPI's position in HT_s .

Because of the above, when looking for the UPI corresponding to a given $s \rightarrow d$, we need to check whether HT_s 's bucket whose position corresponds to d is actually about $s \rightarrow d$. We detail how UPIs are processed to locate their corresponding insertion entries in STAL_s . The steps for in-order UPIs are described in Section 4.7.1, and those for ooo UPIs are described in Section 4.7.2.

4.7.1 Processing of in-order UPI

1. In HT_s , at the position (d modulo current size of HT_s), we find a candidate UPI, call it UPI_{cand} .
2. If the 10-bit suffix of $\text{UPI}_{\text{cand}}.d_p$ equals the last 10 bits of d , UPI_{cand} may be about $s \rightarrow d$, but another check is needed; proceed to (3) below. Otherwise, read the next UPI in HT_s , then the next one, etc. and repeat (2) until we find either a sure match (see below), or an empty position, indicating d does not exist in HT_s .
3. Access, in STAL_s , the insertion entry e_{cand} indicated by UPI_{cand} . If the destination node ID in e_{cand} is d , UPI_{cand} is surely about $s \rightarrow d$.

Last but not least, we detail how UPIs encode address of insertions within STAL . Let $\boxed{\text{UPI}_e}$ be the UPI created for an insertion entry e . Let B denote *the STALB closest to e in STAL_s* : if e is in order, B actually stores e ; otherwise, B holds an IEM whose OOO field references a data structure holding ooo insertions, including e . The UPI for an in-order insertion of $s \rightarrow d$ has six fields (Figure 4.4):

isUPI	i_u	d_s	b_s	b_i	s_s	s_i
63	62	61..52	51..48	47..37	36..32	31..0

Figure 4.4: UPI fields and their bit spans.

- s_i stores B 's position in $STAL_s$, *at the time of the insertion*;
- s_s stores $\log_2$ (the size of $STAL_s$ *at the time of the insertion*. Recall (Section 4.5.1) that this size is always a power of 2. On 5 bits, we can store values up to 32, accommodating a 2^{32} maximum number of entries in the $STAL_s$.
- b_i stores the position of this entry in B 's destNodes;
- b_s stores $\log_2$ (the size of B) *at the time of the insertion*. The maximum block size of B is **maxBlockSize**=4096.
- d_s stores an 10-bits suffix of d , the destination vertex ID;
- i_u is 1 in an UPI created for an in-order insertion, and 0 for an ooo insertion;
- isUPI is 1 in an UPI, and 0 in an AR or LGCD replacing it.

To find B and e based on UPI_e , we proceed as follows:

1. From the metadata of $STAL_s$ (dark gray area in Figure 4.2), we get its size.
2. We compute the *current* position of B in $STAL_s$ as:

$$\text{size of } STAL_s - (2^{s_s} - s_i)$$

In the above, 2^{s_s} was the size of $STAL_s$ when e was inserted. When $STAL_s$ doubles (Section 4.5), existing blocks B *remains at the same distance from the tail of $STAL_s$* ; this distance is $2^{s_s} - s_i$. Subtracting this difference from $STAL_s$ leads exactly to the position of B in the current $STAL_s$.

3. Compute the position of the insertion in B .IEMs as: $\text{size of } B - (2^{b_s} - b_i)$. The reasoning behind the calculation is similar to the one above: $2^{b_s} - b_i$ is the distance between the entry e and the end of B at the time of the insertion; this distance does not change as B gets resized.

4.7.2 Processing of ooo UPI

1. In HT_s , at the position (d modulo current size of HT_s), we find a candidate UPI, call it UPI_{cand} .
2. When the 14-bit suffix stored in $UPI_{cand}.d_o$ matches the last 14 bits of d , UPI_{cand} may correspond to $s \rightarrow d$. To confirm this, we need to perform an additional check (see step 3). Otherwise, we continue checking the next entries in HT_s one by one until we either find a possible match or reach an empty slot, which indicates that d is not present.
3. From UPI_{cand} , we extract the stream timestamp S_T encoded in its t_s field.
4. Using S_T , we perform a binary search in $STAL_s$ to locate the STALB block whose timestamp range includes S_T .
5. Inside the identified STALB, we perform another binary search over the IEM array to find the IEM entry whose stream time matches S_T .
6. When the number of ooo entries associated with this IEM is fewer than 512, we follow the $IEM.ooo$ pointer to the corresponding OSTALB block. Otherwise, we follow $IEM.ooo$ into the Adaptive Radix Tree (ART) to locate the appropriate OSTALB.
7. In the OSTALB, we perform a binary search using S_T to identify the correct OEM entry.
8. Using the index of the identified OEM, we access the corresponding entry in the `destNodes` array of the OSTALB. When this destination matches d , we confirm that UPI_{cand} is indeed about $s \rightarrow d$.

The fields of an ooo UPI are shown in Figure 4.5, and described below:

- t_s (bits 0–47): Stores the stream timestamp S_T of the insertion.
- d_o (bits 48–61): Stores the 14-bit suffix of the destination vertex ID d .
- i_u (bit 62): Indicates whether the insertion is ooo (1) or not (0).

- isUPI (bit 63): Indicates whether this entry is a UPI (1) or an AR/LGCD (0).

isUPI	i_u	d_o	t_s
63	62	(48..61)	(0..47)

Figure 4.5: Out-of-order UPI fields and their bit spans.

4.8 Queries

Dynamic graph systems support a large variety of analytics. Popular algorithms well-known implementations, such as Breadth-First Search, PageRank, Community Detection, can be plugged on top of any in-memory graph store giving access to its edges. In this section, we show how HAL *finds all edges whose source vertex is s , at the current database time*, since this is a crucial operations in graph computations. Other data access methods, e.g., finding nodes or edges by values of their properties, can be supported orthogonally to this work, focused on storing and querying the graph structure.

We set LQ_T , the least recent query time among the currently running queries, at the current database time. Then, for each STALB in $STAL_{s,}$ starting with the most recent:

1. If the hasDeletes field of the STALB is true, **deletion checks are needed**: when traversing the metadata entries associated with this STALB, we will check each IEM entry's ITM, to ensure we do not return an edge already deleted. If hasDeletes is false, we can skip this check for all entries in the STALB, thus read it sequentially with no need for random memory accesses, which is more cache-friendly. As our experiments show (Chapter 5), competitor systems which check each traversed edge suffer of their poor cache use.
2. If the hasOOO field of the STALB is true, **ooo checks are needed**: when traversing the STALB, we will check each OOO field, and include any ooo updates there, in the result. If hasOOO is false, the tests can be skipped for this block's entries.
3. Traverse the STALB's IEMs and in parallel destNodes (recall from Section 4.4 that these are same-size arrays filled in parallel). For each IEM entry m , at position pm in IEMs:

- (a) Check whether the transaction time $m.WT$ is less than LQ_T . If this holds: if no deletion or ooo checks are needed, return $destNodes[pm]$, the destination node corresponding to the position of m ; if deletion checks are needed, only if m is not deleted (it has no ITM), return $destNodes[pm]$. The check helps ensuring snapshot isolation: we only read the (non-deleted) edges received and stored before LQ_T .
- (b) If ooo checks are needed, and m has ooo updates, its OOO field references an OSTALB or ART (Section 4.5.2). For each OEM entry in that structure, call it o , we check whether the transaction time $o.WT$ is less than LQ_T . If this holds, and no deletion checks are needed, return o 's destination node. If deletion checks are needed, check o 's ITM field, to see whether o is deleted. If it is not, return its destination.

Reducing edge deletion checks As an optimization, once a query has identified an edge in HAL as deleted, *this is recorded in a bit of the space devoted to storing d* in the insertion block's $destNodes$ (recall Figure 4.3). Thus, a future query learns just by reading $destNodes$ that the edge is deleted, and avoids a random memory access to read the OEM or IEM of the insertion. This optimization is in the spirit of database cracking [48]. This can very significantly speed up analytical queries, as we show in Section 5.6.

The complexity is $O(|R|)$, where R is the result set of edges.

4.9 Summary

This chapter presents the core design of HAL, a novel in-memory graph storage system developed from scratch to support high transaction write throughput and accurate, efficient analytical scans in parallel, even in the presence of ooo updates. While HAL's concurrency model is inspired by LiveGraph [126], it introduces five key innovations in its layout: (i) preserving stream-time order of updates, (ii) using multiple fixed-size adjacency blocks to avoid costly resizes, (iii) separating metadata from destination IDs to speed up scans, (iv) reducing per-edge deletion checks via cracking-inspired techniques, and (v) performing garbage collection using writer threads instead of background threads, hence reducing contention between reader and writer

transactions. The chapter also introduces a novel indexing mechanism that associates the most likely insertion-deletion pairs based on stream time, enabling correct behavior even under ooo updates. It further details the internal procedures for handling insertions and deletions of both in-order and out-of-order updates, and the steps involved in executing analytical queries efficiently on top of the HAL storage engine. The next chapter presents the evaluation of core HAL storage system across a set of benchmark datasets, including both synthetic and real-world graphs.

Chapter 5

Core HAL System Evaluation

We evaluate our system on different workloads, also comparing it with the baselines Stinger [38], GraphOne [58], Llama [76], LiveGraph [126], Teseo [64], and Sortledton [41]. Below, we describe our experimental setup (Section 5.1), our benchmark dataset, and the graph analytics algorithms used (Section 5.2). Then, we study the performance of insertions (Section 5.3), updates (Section 5.4), analytic querying (Section 5.5), concurrent workloads (Section 5.6), workloads with ooo insertions (Section 5.7), and workloads with ooo updates (Section 5.8).

5.1 Hardware and software settings

Our experiments have been executed on a dual-socket Intel Xeon E5-2640 v4 server, with 40 hardware threads and 256 GB of DRAM. Our system is implemented in C++ and compiled with GCC v10.2, with the -O3 optimization flag. We set maximum size occupied by a STALB at 2^{15} bytes, and 2^{13} for an OSTALB. This allows us to store 512 entries per OSTALB, like Sortledton [41] does in its blocks. We report median times over five runs.

5.2 Workloads

We use the synthetic graph datasets graph-500 [50] with scale factors (SF) 22, 24, and 26; the node fan-out in these graphs follows a power-law degree distribution. To study the impact of the edge distribution across nodes, we used the driver of Teseo [64] to generate variants of graph-500 with scale factors 22 and 24, hav-

Dataset	Vertices $ V $	Edges $ E $	Average degree $\overline{D}$
graph500-22, uni-22	2M	64M	26
graph500-24, uni-24	9M	260M	29
graph500-26	33M	1B	33
dota-league	61K	50M	836
edit-wiki	51M	255M	22
yahoo-songs	1.6M	256M	315

Figure 5.1: Graph datasets in our evaluation.

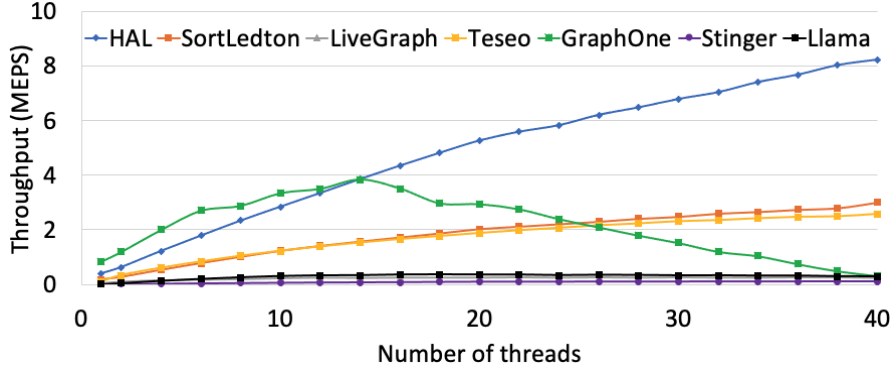


Figure 5.2: Graph500-24 scalability analysis.

ing *uniform node degree distributions*; we denote these graphs uni-22 and uni-24. We also use three real-world graphs, namely dota-league [50], edit-wiki [59] and yahoo-songs [59]. These datasets, used in previous comparable works, are undirected. As in prior work, we replace each undirected edge (s, d) by two directed edges, $s \rightarrow d$ and $d \rightarrow s$. Each edge has just one property, namely weight, that is double precision real number; we generate these weights at random between 0 and 1 with a uniform distribution. The main dataset metrics appear in Tab. 5.1. We use the LDBC graph analytics benchmark [50], from which we use five graph algorithms: Breadth-First Search (**BFS**), PageRank (**PR**), Single-Source Shortest Path (**SSSP**), Community Detection Via Label Propagation (**CDLP**), and the Weakly Connected Components (**WCC**). For fair comparison, we use the algorithm implementations from the Graph Algorithm Platform [17], running on the driver implemented by Teseo [64].

5.3 Edge insertions

To measure insertion performance, we insert successively all the edges of graph500-24, in a random order. All insertions are in order, since no baseline supports ooo updates. Figure 5.2 reports the insertion throughput, measured in Millions of Edges

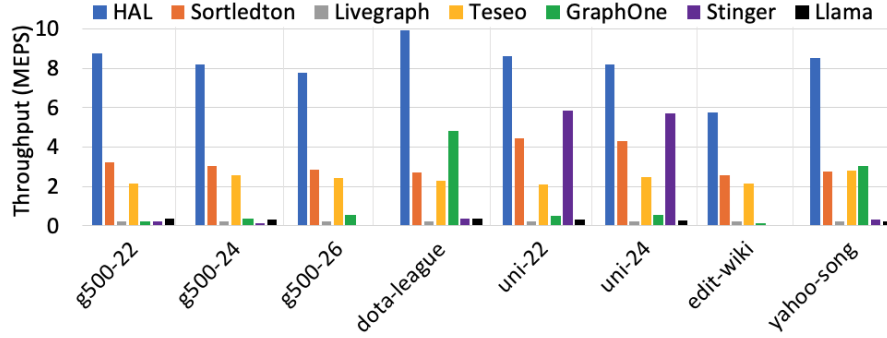


Figure 5.3: Random order insertion throughput.

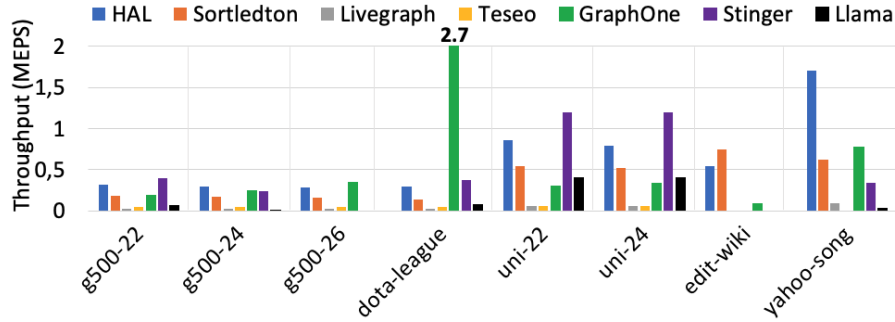


Figure 5.4: Ordered insertion throughput.

Per Second (**MEPS**, in short), as we increase the number of threads (on the x axis) from 1 to 2, 4, etc. until 40. **HAL scales up very well**, better than the other systems, as we increase the number of threads.

HAL outperforms Sortledton and Teseo because these systems pay sorting costs, where the neighborhood blocks are sorted by destination ids, thus leading to contention between writer threads. LiveGraph, Stinger, and Llama do not profit at all from parallelism, because of contention between multiple writer threads, simultaneously trying to *search linearly in the adjacency list (or multi-version CSR)* for the existence of edges. Up to 16 threads, GraphOne performs best, but it doesn't check for edge existence; if this check is added, as shown in [41, 64], the throughput decreases to 5 edges per second. Above 16 threads, contention between threads in the archiving phase strongly degrades its performance. *From now on, unless otherwise specified, we experiment with 40 threads.*

Figure 5.3 shows insertion throughput, again in MEPS, on different systems for our eight datasets in random order. **HAL performs better in all cases.** Sortledton

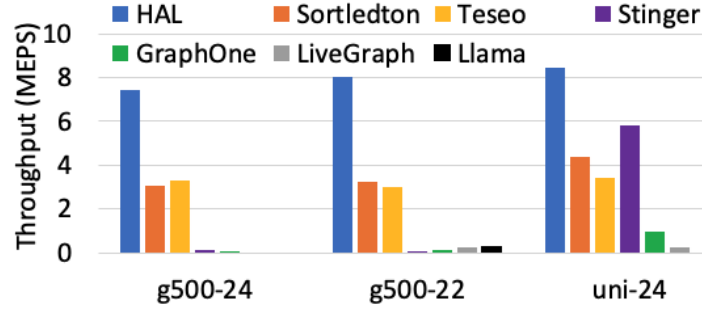


Figure 5.5: Throughput on updates workloads

and Teseo performance are close, due to their similar design. LiveGraph is slow, because it has to repeatedly resize the vectors storing edges, in order to provide fully sequential access to its adjacency lists. GraphOne throughput is generally low; on dota-league and yahoo-song, it performs better, because these datasets have comparatively fewer vertices. This lowers vertex ID translation costs, and reduces write buffer contention. On uni-22 and uni-24, Stinger ranks second in terms of throughput. This is because linear search for checking edge existence is quite fast when there are few edges per source.

Next, we repeat the experiment but with the edges *in the original order*. All the datasets but yahoo-song and wiki are *ordered on the source vertex ID*, while yahoo-song and wiki are also *mostly* sorted by source ID. This amounts to *bursty* workloads where concentrated updates arrive for successive source vertices. Figure 5.4 shows that all systems are affected by bursty insertions (lower throughputs than in Figure 5.3). This is due to higher writer contention, when all threads insert edges into the same source vertex. GraphOne outperformed HAL on two datasets (dota-league, g500-26) because of its unchecked, lock-free edge insertion, particularly effective on datasets with fewer vertices, such as dota-league. In contrast, HAL incurs a secondary index maintenance cost ($HT[s]$), and Teseo and Sortledton need sorting, leading to more writer contention.

On the wiki dataset, with a maximum of 5M edges adjacent to a node, Sortledton slightly outperforms HAL. This dataset translates into high bursts of edges adjacent to the same node.

Lesson learned: Storing each source vertex’s neighborhood list sorted by destina-

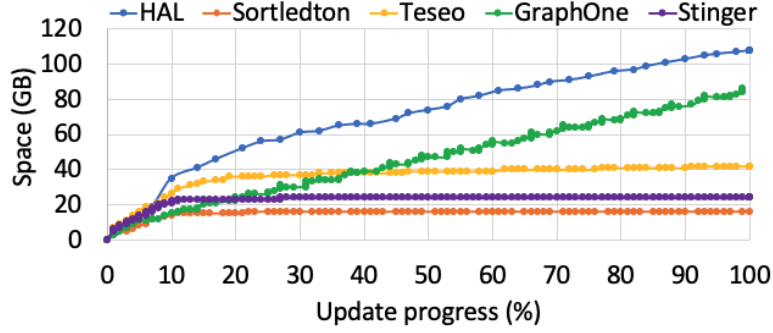


Figure 5.6: Memory usage on Graph500-24 update workload.

tion IDs enables more efficient edge-existence checks, than the linear search cost of the append-only store. However, the sorting cost for an insertion is higher. To make edge existence checks efficient, HAL trades the space of the secondary index HT, for speed. On a uniform graph, linear search is cheaper than maintaining a sorted list. Bursty updates on the same source vertex significantly impact the transactional systems because of the per-source vertex locking.

5.4 Updates (insertions and deletions)

We generated update (insertion, deletion) workloads on g500-22 and g500-24, with power-degree edge distributions over nodes, and the uniform uni-24. The update workloads are generated as introduced by Teseo [64]: the first 10% of the workload operations load the graph, after which the remaining 90% (insertions and deletions) keep the (already large) graph at approximately the same size. The size of the update workload is $10 \times |E|$.

Throughput: Figure 5.5 shows the systems’ throughput. HAL outperform other systems by $2\times$ to $73\times$ in power-law graphs, and by $1.45\times$ to $8.82\times$ on uni-24 workloads because of HAL’s insertion and deletion which run in constant time, vs. $O(\log(|E|))$ for Sortledton and Teseo, $O(|E|)$ for GraphOne, Stinger, LiveGraph, and Llama. The linear search on the adjacency list penalizes less on a uniformly distributed graph (uni-24). We could not run LiveGraph and Llama on g500-24 and uni-24 workloads because of memory limits.

In details, HAL’s advantage is due to: (i) append-only edge updates, as opposed to sorted list and/or linked lists, which both incur sorting costs, and more random

memory accesses, thus lower cache usage; (ii) deletions in constant time, compared to Sortledton and Teseo that need to search in block lists / trees, thus incurring random accesses.

On the power-law graphs, HAL outperforms GraphOne by $56\times$, LiveGraph by $34\times$, Stinger by $73\times$, and Llama by $27\times$, because these systems perform deletions in $O(|E|)$ time complexity. On uni-24, Stinger is only a $1.45\times$ slower than HAL, because its linear search for deletions is less penalizing. This is because the linear search for the deletion operation per source vertex is cheaper, compared with the case of power-law degree-distributed datasets. GraphOne is $8.82\times$ slower than HAL on uni-24: although deletions are processed faster on this graph, the contention on the write buffer and sequential ID translation penalize its performance

Memory consumption: We study the memory consumption of various systems in Figure 5.6. The x -axis is the update progress, while the y -axis shows the store memory size in GBs. Sortleton, Teseo, Stinger exhibit stable memory growth after 10% of the workload and GraphOne memory grows because of partial garbage collection. In contrast, HAL’s memory occupancy increases: HAL’s support for ooo updates requires *maintaining records for each edge in the hash table without deleting them*. Thus, at 10% progress, we found the hash table occupied 6.34GB, whereas at 100% progress, it took 70GB. Comparable systems that do not support ooo updates do not have to pay this price. Also, support for ooo updates requires storing various *metadata fields*, notably IEMs (holding a stream time, transaction time, and the OOO data structures), which occupied roughly 15 GB by the end of the experiment.

Lesson learned: Compared with set-based systems, HAL uses more memory, in exchange for (i) support for out-of-order updates, (ii) more than $2\times$ speed-up, and (iii) the possibility of supporting historical queries based on the stream time (in our future work).

5.5 Analytics

We ran the LDBC graph analytics benchmark [50] on all the systems. As in [41, 64], Figure 5.7 shows the analytics results on the g500 datasets, and on dota-league. BFS and SSSP require random vertex access and sequential neighborhood access: on

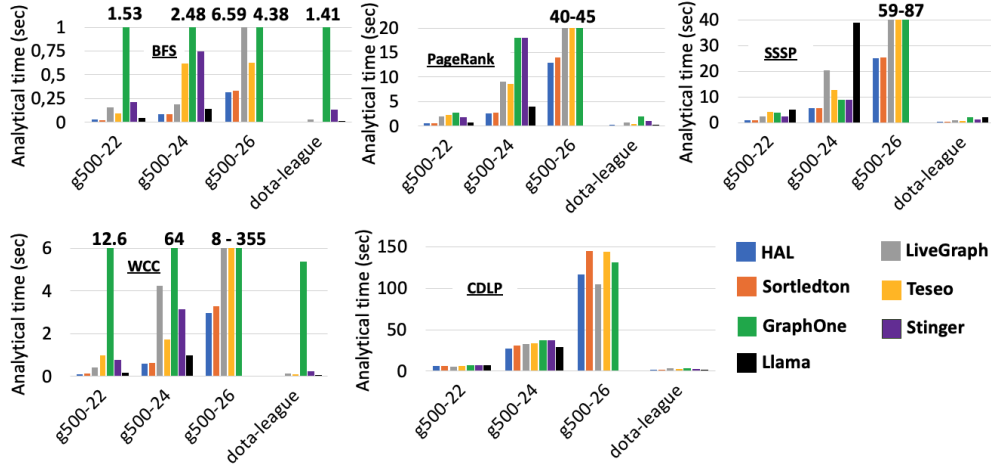


Figure 5.7: Performance evaluation on graph analytics.

these, HAL is comparable to the baseline and outperforms LiveGraph, Teseo, GraphOne, Stinger. On PageRank, WCC, and CDLP, which need sequential vertex access and sequential neighborhood access, HAL outperforms all others, with Sortledton coming closest.

LiveGraph is slower than HAL due to storing edge entry metadata (transaction timestamp, invalidation timestamp, property size) with destination ids, which causes more cache misses. In contrast, HAL stores destination ids separately from the edge entry metadata, hence, fewer cache misses. Also, LiveGraph checks each entry's invalidation time to see if it was deleted. In contrast, HAL stores a boolean hasDeletes in each STAL block (Section 4.4), avoiding the check when the flag is false. **Teseo is slower than HAL** because: (i) Teseo needs a per-edge entry mapping (hash table) from sparse to dense vertex ids in the analytics algorithms, which is costly; (ii) a sorted neighborhood block in Teseo contains up to 512 edges, while a STALB holds 2047 edges, resulting in fewer random accesses. **GraphOne is slower than HAL** since it runs analytics on an ad-hoc merge of the write-store and the read-store; as noted in [41], it has a high overhead for accessing few edges, hence more cache misses. **Stinger is slower than HAL** because of more cache misses as it stores adjacent edges in a linked list of 14-edges blocks, incurring more random accesses to read the list. **Llama is slower than HAL** because it accesses multiple snapshots, hence more pointer chasing and cache misses, slowing traversals such as SSSP and BFS.

Sortledton is marginally slower than HAL on whole-graph algorithms, such as PageRank, WCC, and CDLP, due to the smaller blocks (512 edges for Sortledton, 2047 for HAL), leading to more cache misses in Sortledton. On algorithms accessing a subset of the graph, such as BFS and SSSP, on smaller datasets (graph500-22 and dota-league), **HAL is slightly slower**. This is because random vertex access are more frequent than sequential neighborhood scans, leading to slightly more cache misses for HAL; also, Sortledton reads 24 bytes to access a source vertex, and HAL reads 48.

Lessons learned Set-based systems (Sortledton and Teseo), which only read the latest version of the destination ids, outperform LiveGraph, which must traverse all their versions in the adjacency list, together with per-edge information. LiveGraph is also hampered by the need to check edge invalidation timestamps. HAL’s separation of destination from edge entry metadata speeds up vertex ID accesses. HAL also avoids some edge validity checks (via the `hasDeletes` flag), and is faster by avoiding (like Sortledton) Teseo’s runtime sparse-to-dense vertex ID mapping. GraphOne is slowed down by its merging of the read and write store, and high overhead when accessing few edges, as noted also in [64, 41].

5.6 Concurrent read-write workload

We run the analytics algorithms (BFS, PR) from Section 5.5 concurrently with the updates from Section 5.4. In Figure 5.8, the x -axis shows the combinations of Writer (W) and Reader (R) threads, e.g., 5-5 denotes 5 writer and 5 reader threads, and the y -axis shows the throughput (MEPS) and the analytical algorithms (BFS, PR) running time. We pick these values since in our setting, a total of 40 threads has previously shown best performance; of course, each application may have its own best read-write balance.

HAL outperforms the other systems, because of: (i) insertions and deletions in $O(1)$; (ii) optimized garbage collection (see Section 4.6.1) and database cracking strategy (see Section 4.8). GraphOne is penalized by its the expensive merging of read and write stores. On the uniform workload (uni-24), the insertions and deletions cost is less penalizing for Sortledton and GraphOne compared to HAL, due to the uniform nature of the workload. We could not run LiveGraph because of memory limits, and

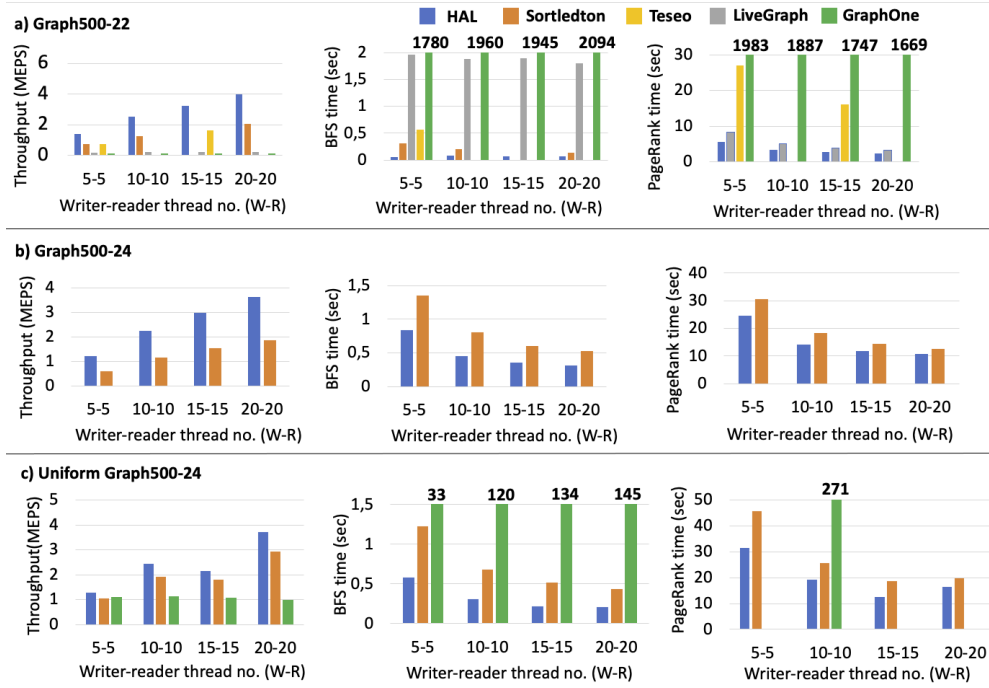


Figure 5.8: Throughput and query performance analysis using varying updates workloads on read/write workloads.

Teseo because of a runtime error.

We also measured the **impact of reducing edge deletion checks** (Section 4.8) by having each query *mark* the edges it finds deleted, so that future queries can avoid random accesses to learn the same thing from an IEM (OEM). In the above experiment, when using 15 writers and 15 reader threads, PageRank takes 11.90s with this optimization, and 50s without, the $4\times$ speed-up.

Lesson learned: On a concurrent read-write workload, HAL outperforms other state-of-the-art systems owing to: (i) append-only storage, which often allows the writer to append data without affecting the reader; (ii) constant-time edge deletions; (iii) avoid some deletion checks in database cracking style, which improves cache locality and optimizes garbage collection. We also observe that GraphOne’s edge copying concurrently with the updates is costly.

5.7 Out-of-order insertions

We now study the performance of our system when faced with a mixture of in-order and ooo insertions. Since prior systems do not support ooo updates, we built

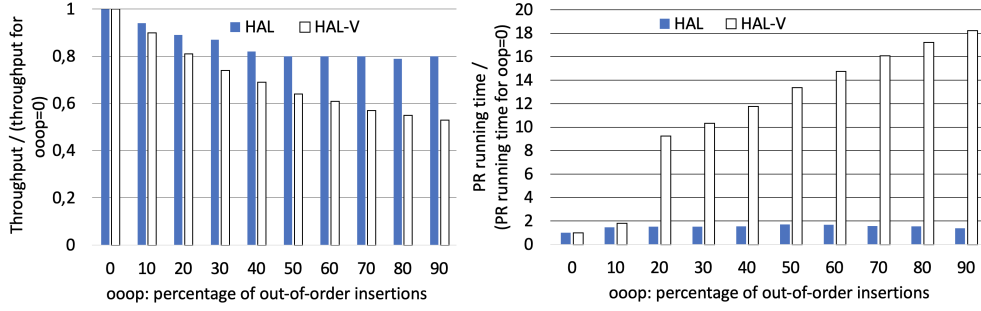


Figure 5.9: Performance on in- and ooo insertions.

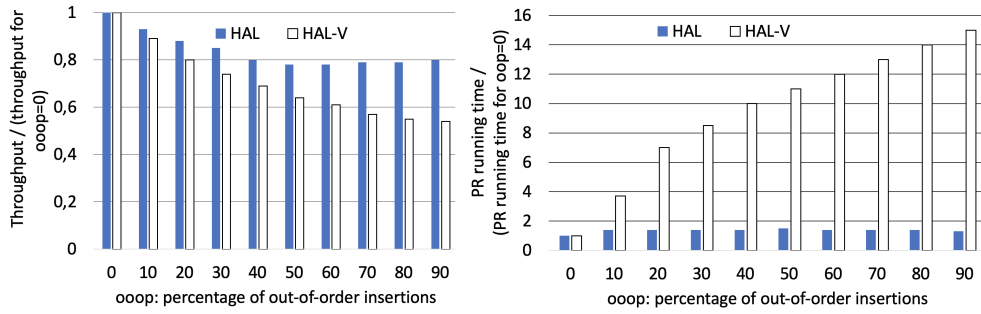


Figure 5.10: Performance on in- and ooo insertions.

a **benchmark** based on graph500-22 and graph500-24, as follows. (i) Sort the (in-
 sersion) workload by the source vertex. (ii) Remove the edges whose source vertex
 has less than 10 edges in the dataset (to facilitate our next steps, see below). We
 thus obtain an ooo insertion list called **OIL** (243M edges between 1.5M vertices in
 graph500-24, and 59M edges between 0.5M vertices in graph500-22). OIL edges are
 given consecutive STs: 1, 2, 3 etc. (iii) For a given **ooo percentage** *ooop*, we swap
 some edges in OIL, to ensure that exactly *ooop* among them are ooo. For instance, if
ooop is 40%, among each 10 successive edges with the same source, we swap the 1st
 with the 5th, making the (now) first four updates ooo. Lacking control of how multiple
 threads intersperse their work, we only use *one thread*.

Recall that HAL stores ooo updates first in OSTALBs, and only lazily creates ART
 (Section 4.5.2). We investigate how this compares to creating an ART at the first ooo
 update received. Specifically, we design a **HAL variant**, called **HAL-V**, which does
 exactly this, and compare it with HAL.

Figure 5.9 and Figure 5.10 (left) show how the **throughput** of HAL and HAL-V is
 impacted by *ooop*, compared to *ooop*=0 (in-order insertions only). HAL is impacted by

20%, whereas HAL-V shows a 40% decline in throughput when 90% of the updates are ooo. This is due to no ART cost in HAL (as long as the OSTALB is not full). In contrast, HAL-V pays an ART insertion cost for each ooo update.

We observed that at 50% ooo updates, the throughput decreased by 22%, with no further decrease observed thereafter. This is because, at the 50% ooo threshold, the costs incurred for binary searches to locate ooo updates in the STAL[s], and the costs for maintaining ooo STALBs (including resizing and ordering by timestamp), are balanced. Beyond this point, there is no further decrease in throughput; the system incurs relatively lower binary search costs but higher maintenance costs for ooo STALBs, as the majority of operations are concentrated within ooo STALBs.

At right in Figure 5.9 and Figure 5.10, we study the impact of ooo insertions on **analytics running time** (specifically, PageRank). For each *ooop* value, we run the respective insertion workload, then run PageRank on the graph, and show its running time divided by the time on a graph without ooo updates. HAL-V is strongly impacted (PageRank takes $15\times$ more for *oop*=90% than for *oop*=0), while HAL is impacted by a factor of $1.3\times$. Both HAL and HAL-V incur a cost for cache misses after every ten entries when accessing the IEM.OOO field to scan the ooo insertions, likely the reason for HAL's performance decrease. HAL-V also incurs additional cache misses for each edge entry read from the ART, whereas HAL incurs only one for every eight entries (in an OSTALB, each ooo destination ID occupies 8 bytes, thus a cache miss after every eight entries.)

Lesson learned: HAL performance is very robust as the proportion of ooo insertions increases. The extra algorithmic complexity incurred by “lazily” creating ARTs for ooo insertions pays off by keeping insertion and analytics very efficient.

5.8 Out-of-order updates

We built an **ooo update benchmark** based on graph500-22 and graph500-24 as follows. (i) For each edge in the datasets, record first its insertion, then its deletion, leading to an OOO update *list* we call **OUL**, whose entries are timestamped 1, 2, etc. (ii) For a given **swap percentage** *sw*, we swap some updates in OUL. For instance, if *sw* is 10%, after each 10 entries in OUL, we select one edge and swap its insertion with

Datasets	0%	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
graph500-22	1.00	1.11	1.10	1.09	1.08	1.10	1.11	1.12	1.14	1.16	1.2
graph500-24	1.00	1.04	1.04	1.04	1.04	1.05	1.06	1.07	1.08	1.1	1.11

Table 5.1: Throughput variations when varying the swap percentage sw , on graph500-24 and graph500-22 workloads.

its deletion. If sw is 100%, among each 10 edges in the list, we swap each insertion with its deletion. As in Section 5.7, we only use *one thread* in this experiment.

Table 5.1 shows no slowdown (even a little improvement) in HAL throughput as sw grows: ooo deletions are fast. We also tested the accuracy of HAL with respect to other systems that do not support ooo updates; we used LiveGraph (Sortledton throws an exception at the first ooo update, while Teseo seems to get “stuck”). For each sw , after loading, each system is asked for all the edges present in the graph. HAL returns an empty graph regardless of sw , which is correct (in OUL, for each insertion with a given ST, there exists a deletion with ST+1). LiveGraph wrongly returns more and more edges as sw grows.

Lesson learned: The ability to properly handle ooo updates is crucial to ensure correctness (transactional querying). HAL achieves this at the cost of more memory.

5.9 Summary

This chapter presents a comprehensive experimental evaluation of HAL, our novel in memory dynamic graph store, across a diverse set of synthetic and real world workloads. HAL is benchmarked against six state of the art systems: Stinger [38], GraphOne [58], Llama [76], LiveGraph [126], Teseo [64], and Sortledton [41] on a variety of graph processing tasks, including edge insertions, updates, concurrent read write workloads, and analytical queries. HAL consistently outperforms all baselines: it achieves up to $2.5\times$ higher update throughput than the best alternative system (Sortledton), and up to $357\times$ speed up on graph analytics tasks depending on the algorithm and dataset characteristics. Notably, HAL is the only system to support ooo insertions and deletions, while ensuring accurate and consistent analytical results.

The next chapter introduces historical query support and arbitrary property handling for nodes and edges in HAL, along with a GUI based interactive use case demonstrating the impact of ooo updates on query accuracy.

Chapter 6

HAL Extensions: Historical and Property Graph Queries

In this chapter, we present two key extensions to our core HAL system. First, we introduce support for historical queries over the HAL system (Section 6.1) and analyze their results in Section 6.3. Second, while the core HAL system is topology-first and supports only a single weight property, we extend the model to allow an arbitrary number of edge and vertex properties, as detailed in Section 6.2, with corresponding results discussed in Section 6.4. Finally, we demonstrate a real-world use case that highlights how ooo updates affect the accuracy of query results over the graph state (Section 6.5).

6.1 Historical queries

In this section, we address the challenge of evaluating *historical queries* over a dynamic graph at a given moment, specified as a *stream time* τ . Recall that the stream time globally orders the moments where updates are emitted in the system, whereas the transaction time is impacted by the order in which updates reach the database. Thus, the stream time is the natural time to use for historical queries. For ease of presentation, below, we consider the query requesting *all the graph edges at stream time τ* ; based on this, more complex queries can also be handled.

The problem, then, is to decide, at a given moment (transaction time) τ_h when the

query is issued, whether each edge $s \rightarrow d$ in the store, was logically present at time τ , with $\tau \leq \tau_h$.

We must guarantee that only updates that both *occurred (were issued) before the* τ , and *were committed before* τ_h are reflected. This is in order to ensure transactional query processing, as updates that were not committed before τ_h are considered not received yet at the database.

For each edge $s \rightarrow d$ in the store, one of the following holds:

- The edge was inserted at or before τ , was never deleted, and the insertion committed before τ_h : this edge is still present in the graph at τ_h , and should be included in the answer.
- The edge was inserted at or before τ , was deleted after τ , and the insertion committed before τ_h : the edge still existed at τ and should be included.
- The edge was inserted and then deleted before or at τ , and the deletion committed before τ_h : the edge no longer existed at τ and must not be in the answer.

For instance, in Figure 4.2, a query is issued at $\tau_h = 10:15$, asking for the edges in **VA[0]** as they were at $\tau = 10:05$.

To make this reasoning, for every source vertex s , HAL uses STAL_s , the stream-time sorted adjacency list storing updates whose source node is s , as described in Section 4.4. We traverse STAL_s to determine which edges were present at τ , considering only edges that had been committed before the query's arrival time τ_h . We proceed for each source vertex s as follows:

A. Locate the starting STALBs using stream time τ . We first perform a binary search on STAL_s using the stream timestamp τ as the search key. This identifies the most recent STALBs, S_p , whose entries could be present at time τ . All STALBs including S_p and older than S_p are traversed in reverse stream-time order. This avoids unnecessary work, bounding the scan to only relevant updates (the others are too recent).

B. Classify the STALB using metadata flags. Before accessing a STALB's entries, we read its `hasDeletes` and `hasOOO` flags (Section 4.4), which state whether the block contains deletions, respectively, ooo updates. We distinguish four cases:

1. **Both `hasDeletes` and `hasOOO` are true:** The STALB contains deletions, and ooo updates. For each IEM entry, we check both the ITM field (to detect deletions) and the OOO field (to include any present ooo updates).
2. **`hasDeletes` is true, `hasOOO` is false:** The STALB contains deletions, but no ooo updates. We check each IEM entry's ITM field, to confirm that it was not deleted before τ .
3. **`hasOOO` is true, `hasDeletes` is false:** The STALB contains ooo updates but no deletion. We follow the OOO field for each entry (if present) to locate relevant updates in OSTALBs or ARTs.
4. **Neither flag is set:** The STALB contains neither deletions nor out-of-order updates. This enables a *fast traversal* (just scan entries sequentially), without the need to check per-entry metadata. This improves cache locality and avoids costly random memory accesses.

In our example, for $\tau = 10:05$ and $\tau_h = 10:15$, the binary search identifies STALB_0 as the relevant block. At this point, it contains both deletions and ooo entries. Thus, STALB_0 is in **Case (1)** among the four above.

C. Locate the first relevant entry within the STALB. On the most recent STALB S_p , perform a binary search on its IEMs array to find the first index e_i of an update whose stream time is $\leq \tau$. This is the point from where we start reading. For older STALBs, which are always full, we scan from index 0 (newest) to the end. HAL also inspects the IEM at index $e_i - 1$ (if present). Although its stream time exceeds τ , it may contain an OOO pointer referencing OEM entries with stream time $\leq \tau$. If such a pointer exists, we process it as described in step (5c) below, ensuring that all relevant ooo updates are considered.

In Figure 4.2, a binary search on the IEMs in STALB_0 using $\tau = 10:05$ locates index 5, corresponding to the insertion entry for the edge $0 \rightarrow 2$. This is the most recent insertion not exceeding τ and serves as the entry point for the scan. We also verify that the IEM entry at index 4 does not contain an OOO field, confirming that the entry at index 5 is the correct starting point.

5. **Scan IEM entries from the position e_i .** We scan IEM entries from e_i toward the oldest in S_p , and do the same for older STALBs (which, by the way HAL works, are guaranteed to be full). For each IEM entry at position $curIndex$ within its block, we evaluate its eligibility based on transaction visibility and metadata as follows:

(a) **Transaction time check:** We first check whether the IEM entry at the $curIndex$ in the STALB was committed before the query arrived, that is, before τ_h , or after. If the entry was committed after τ_h , it was not part of the database at that time, therefore it is skipped.

If the entry was committed before τ_h , and the STALB contains neither deletions nor ooo updates (case (4)), or only ooo updates (case (3)), the entry is considered present, and we include $s \rightarrow d$ in the query result.

(b) **Deletion check (if applicable):** When the current STALBs contains deletions (in cases (2) or (1)) and the IEM entry at the $curIndex$ in the current STALBs has an associated deletion record, we access its ITM block, which holds the metadata for that deletion. We examine the ITM to determine whether the deletion is relevant to the current query. Specifically, we check that the deletion's stream time is later than τ (meaning that the edge was still logically present at τ), and that the insertion (i.e., IEM entry) was committed before the query time τ_h . If both conditions are satisfied, the edge is considered present at τ , and we include $s \rightarrow d$ in the result.

(c) **ooo update check (if applicable):** When the current STALBs includes ooo updates (cases (3) or (1)), and the IEM entry at $curIndex$ has an OOO field, we follow that field to retrieve potential past updates that arrived ooo, were issued before τ , and committed before the query time τ_h . Depending on the number of such entries, as explained in Section 4.2, the OOO field may reference either a single OSTALB (Out-of-order STALB) if there are 512 or fewer entries, or an Adaptive Radix Tree (ART) which is an index over several OSTALBs, if there are more entries. In either case, we need to identify the metadata (OEM) attached to these entries; we collect them

in a set denoted OEM_o . We proceed as follows:

- When the OOO field leads to a single OSTALB, we perform a binary search within this OSTALB using τ to locate OEM entries whose stream time is earlier than τ . We gather these entries in OEM_o .
- When the OOO field leads to an ART, we search the ART for all OSTALBs whose keys are less than or equal to τ . In each retrieved OSTALB, we use binary search to extract the OEM entries whose stream time is earlier than τ , and add them to OEM_o .

For each OEM_o entry, we apply a deletion check as follows:

- If the entry is not marked as deleted (i.e., with `hasDeletes` flag set to false) and committed before the query arrival time τ_h , we include $s \rightarrow d$ in the query result.
- Otherwise, we check if the entry has ever been deleted: if that is the case, it has an associated ITM. Upon finding an ITM, we examine it to determine when the deletion occurred. If the deletion's stream time is later than τ , and if the entry was committed before the query arrival time τ_h , we conclude that the edge $s \rightarrow d$ was still present at τ . Thus, it should be included in the result. If the entry has no ITM, it has ever been present since its insertion, and it is included in the result.

In our working example, continuing the scan in Figure 4.2, we begin at index 5 of $STALB_0$, which contains the IEM_5 entry for edge $0 \rightarrow 2$ with write timestamp (WT) 10:06. Since $WT < \tau_h = 10:15$ and no deletion is recorded, this edge is included in the query result. This IEM_5 has an OOO field, which HAL follows; it leads to an OSTALB marked with `hasDeletes`, thus it contains deletions. Within this OSTALB, we locate an OEM_0 for edge $0 \rightarrow 9$ with WT 10:07 $< \tau_h$, and a deletion committed at 10:08, also before τ_h . Since the deletion's stream time is $\geq \tau = 10:05$, the edge was still present at τ and is included in the result. We then proceed to index 6 of $STALB_0$, which contains the IEM_6 entry for edge $0 \rightarrow 1$ inserted at 10:04, with a deletion committed at 10:04, both before τ_h . Because the deletion's stream time is earlier than τ , edge $0 \rightarrow 1$ is excluded from the query result.

The worst-case computational complexity of evaluating a historical query is $O(\log(|E|) + |R|)$, where $|E|$ is the number of edge entries and $|R|$ is the size of the query result.

6.2 Property-rich nodes and edges

As outlined in Section 4.4, *edge properties* are not stored directly within the graph topology but are instead *referenced* from it. Specifically, for a given source node s , the STAL_s entry, as shown in Figure 4.2, includes a reference to a vector storing the *weight* property of each edge. This property can be viewed as generic, as it is frequently used in graph analytical algorithms such as Single-Source Shortest Path (SSSP).

To enable attaching arbitrary properties on edges, as well as vertex properties, we store them independently within RocksDB [81], *a high-performance, embeddable key-value store optimized for fast storage*. This separation ensures that the graph topology remains compact and thus HAL continues to provide efficient analytical operations on the graph topology, operations that are often central for analytics.

RocksDB organizes key-value data within *column families*, each of which can be seen as an independently managed database within a single RocksDB instance. Each column family consists of one key-value collection (called *memtable*), implemented by a skip list-based structure (SkipListMemTable) [104]. Memtables provide logarithmic-time access and are periodically flushed to disk as Sorted String Tables (SSTables) under RocksDB’s Log-Structured Merge-tree (LSM-tree) architecture. Each column family maintains its own Log-Structured Merge-tree (LSM-tree) [88]. This design allows each column family to be tuned and accessed independently for best performance.

To support flexible and efficient access to properties, we store vertex and edge properties in separate RocksDB column families within RocksDB. The decision reflects the fact that these two categories of data are semantically orthogonal and follow different access and update patterns. By storing them separately, our system stands to benefit from improved data locality, faster point lookups, and higher write throughput.

To encode vertex and edge properties within RocksDB, our system employs *composite keys* that enable direct and efficient access to each property. This departs from existing systems [64, 41, 102, 126], which co-locate all properties of a vertex or edge

and require deserialization of an entire property block to retrieve a single property. Our solution avoids unnecessary data access, reduces cache misses, and improves both query latency and memory efficiency, particularly in analytical workloads that target selective subsets of properties. The details of the vertex and edge property formats are as follows:

- **Vertex properties.** The key used to store a vertex property is of the form

`<updateStreamTime>_<vertexId>_<propertyName>_<op>`. For example, the key `1_1_name_insert` identifies an insert operation for the `name` property of vertex 1 at stream time 1; a possible value is, e.g., `"Khan"`. This design also preserves the update chronological order.

- **Edge properties.** Similarly, an edge property is indexed using the format:

`<updateStreamTime>_<src>-<dst>_<propertyName>_<op>`, where `<op>` indicates the type of operation (e.g., `insert`, `delete`). A key like `1_1-2_createdDate_insert` records the insertion of a `createdDate` property on the edge from vertex 1 to vertex 2 at stream time 1, with a value such as `"01/10/2025"`.

This structure encodes not only the source and destination of the edge, but also the stream timestamp of the update and the property name, enabling a log-style representation of updates. This logging approach is consistent with the design principles of HAL, which stores the graph as a history of updates. Insertions and deletions of properties are allowed both on nodes and on edges. Key-value pairs thus formed are inserted into the memtable of their corresponding column family.

Transactional coordination and consistency. Given that we now store graph information in two distinct stores (the core HAL structure for the topology, the core HAL structure, plus RocksDB for properties), we need to ensure *transactional guarantees* for updates relying on these two stores. For that, we adopt a decoupled but coordinated transactional protocol, with HAL serving as the transaction coordinator. Specifically, property updates are first committed within the (transactional) RocksDB context, using its native optimistic concurrency control mechanism [105]. If that transaction fails, we do not attempt the corresponding topological update in HAL; otherwise, we go forward with the HAL transaction (structure update). Since HAL only appends

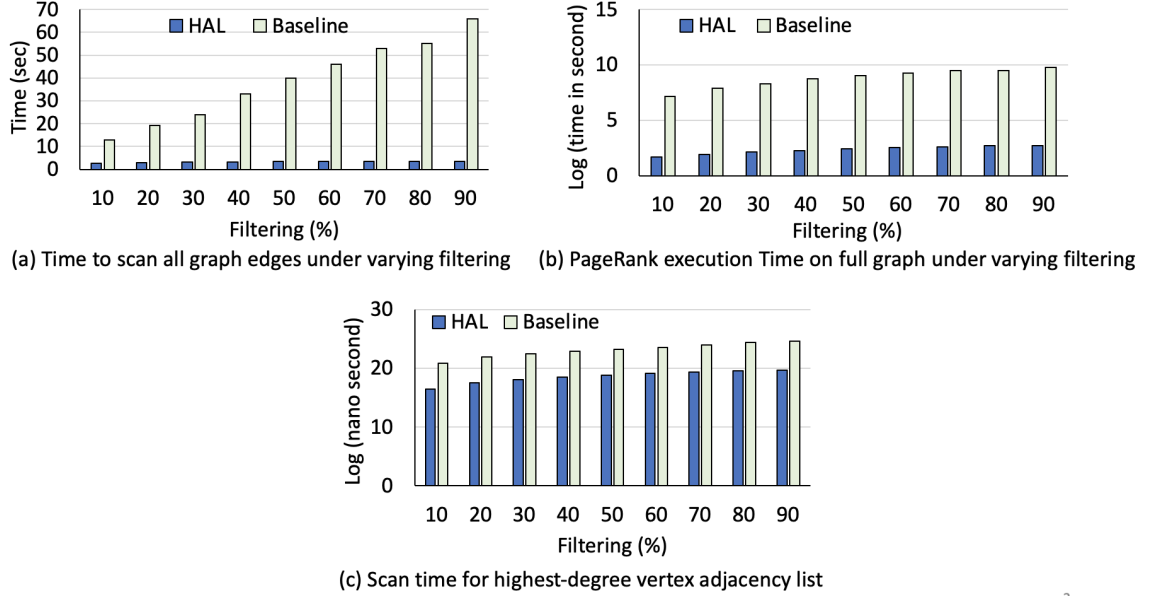


Figure 6.1: Performance of historical queries at different moments in the update history.

updates (that it may later match with other pending updates), HAL update transactions do not fail. This ordering of operations guarantees that no edge or vertex becomes visible in the topology, unless its associated properties are durably persisted.

Historical queries over property graphs As explained above, we have extended (a) the HAL topology store with historical queries, (b) separately (and orthogonally), the storage model with node and edge properties, on which snapshot queries are already supported. The ingredients are in place to combine the two extensions, i.e., answer historical queries over property graphs. This extension is part of our future work.

6.3 Historical queries

To evaluate the performance of historical queries under ooo insertions, we based our experiments on the Graph 500 scale-24 dataset [50], which contains approximately 9 million vertices and 260 million edges. Lacking a benchmark with ooo updates, we reuse our update workload from [3], based on the Graph 500-24 dataset, but featuring 50% ooo insertions. We construct our workload as follows: (i) Sort the

graph500-24 workload (edge insertions) by the source vertex. (ii) Remove the edges whose source vertex has fewer than 10 edges in the dataset (this facilitates our next steps; see below). We thus obtain an *OOO insertion list* called **OIL** (243 million edges, between 1.5 million vertices). Edges in OIL are assigned consecutive stream times: 1, 2, 3 etc. (iii) For a given out-of-order percentage (*ooot*), we swap some edges in OIL to ensure that exactly *ooot* among them are out-of-order. In our case, we use $ooot = 50\%$, resulting in a realistic OIL where approximately half of the edges are disordered.

Since existing systems [64, 41, 102, 126] do not support historical queries in the presence of ooo insertions, we implemented a **baseline** system for comparison. This baseline employs a conventional adjacency list structure, where updates are stored in the order in which they are received (not necessarily the order in which they are emitted), unlike HAL.

To support historical queries, we augment the baseline with a **secondary index** based on an Adaptive Radix Tree (ART). In this index, each key is a stream time, and the corresponding value indicates the position, in the adjacency list, of an update emitted at that stream time (since we assume a global order on updates, there is at most one update per stream time). This secondary ART enables access to data by their stream time, an access pattern not supported by the simple baseline.

To illustrate a varied range of historical queries, we have computed 9 query stream times $\tau_1, \dots, \tau_9$ as follows. For each $1 \leq i \leq 9$, τ_i is the highest stream time among the first $i \times 10\%$ of the edge updates in the above workload. Figure 6.1 presents the performance of historical queries at stream time τ_i , for $1 \leq i \leq 9$; the value plotted on the x axis is $i \times 10\%$. For instance, 10% filtering corresponds to querying 10% of the 243 million total edges. Figure 6.1(a) shows the time required to scan all edges under different filtering levels. Figure 6.1(b) presents the execution time of the PageRank algorithm computed over the filtered edge sets, while Figure 6.1(c) reports the scan time within the adjacency list of the highest-degree vertex, chosen to avoid bias from low-degree vertices.

In all three cases, **HAL** consistently outperforms the baseline system. This is due to HAL’s data layout, which stores edge updates sorted by stream time. As explained

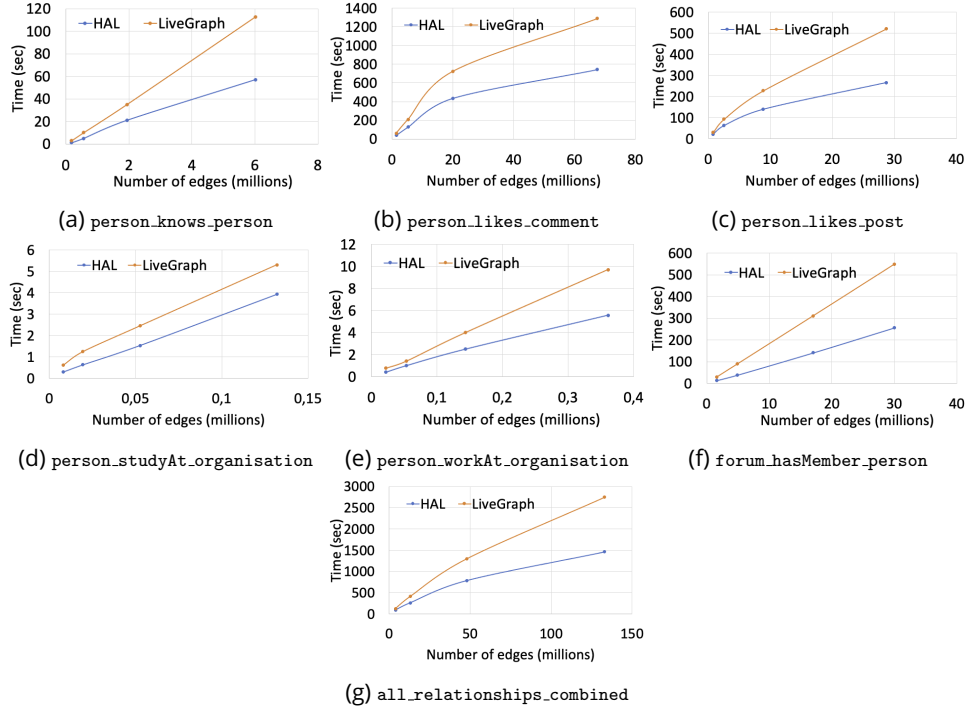


Figure 6.2: Edge insertion time across SNB relationships.

in Section 6.1, HAL leverages this ordering to perform a binary search to quickly identify the position corresponding to the target stream time, followed by a cache-friendly linear scan. In contrast, the baseline system relies on the secondary ART tree index. Although an ART enables efficient lookup of the target stream time in $O(\log |E|)$ time, subsequent scanning suffers due to random-access reads, as the adjacency list is not sorted by stream time—leading to frequent cache misses. This performance gap is clearly visible across all query types in Figure 6.1, demonstrating the efficiency of HAL in supporting historical queries, particularly as filtering levels increase.

Lesson learned: HAL delivers significantly better performance for historical queries under out-of-order insertions due to its stream-time-sorted layout. In contrast, the baseline system suffers from the overhead of random-access patterns introduced by its reliance on secondary indexing.

6.4 Property graph queries

To evaluate the performance of property-aware vertex and edge insertions, we rely on the popular LDBC Social Network Benchmark (SNB) interactive workload [50],

Scale factor (SF)	Number of vertices	Number of edges
1	~3M	~17M
3	~9M	~51M
10	~30M	~170M
30	~90M	~510M

Table 6.1: Estimated number of nodes and edges in LDBC SNB Interactive v1 datasets at different scale factors.

at scale factors 1, 3, 10 and 30; the dataset characteristics are shown in Table 6.1. We present a comparison with LiveGraph [126], one of the most recent transactional graph systems for dynamic graphs. Systems such as [41, 64, 102] provide only partial property graph support, typically limited to a single edge property (e.g., weight) and no support for vertex properties, as they are primarily designed for structural graph updates. In contrast, LiveGraph supports the full LDBC SNB update benchmark, making it a suitable and practical baseline for our evaluation. We evaluate insertion performance in this PG context.

We focus on six benchmark relationships, namely `person_knows_person`, `person_likes_comment`, `person_likes_post`, `person_studyAt_organisation`, `person_workAt_organisation`, `forum_hasMember_person`. These are the only ones in the SNB dataset that include edge properties.

To ensure compatibility with our system and LiveGraph, which both support only integer vertex IDs, we address a subtle yet important issue in the LDBC SNB dataset: overlapping node IDs across different entity types. For example, both `person` and `forum` nodes may have IDs such as 1, 2, etc., leading to collisions when ingesting multiple entity types into a shared graph context. This lack of global uniqueness arises because the SNB benchmark organizes node and edge data across multiple CSV files, where IDs are guaranteed to be unique only within the scope of each node type, not across the entire graph.

To resolve this, we apply a normalization strategy that ensures globally unique vertex identifiers while preserving the integrity of the original dataset. Concretely, we first count the number of nodes for each entity type (e.g., 1000 `person` nodes, 2000 `forum` nodes) and then define a translation offset n_T for each node type T . This offset is added to the original ID of each node of type T , such that the resulting IDs

become globally unique. For instance, we may assign $n_T = 0$ for `person` nodes and $n_T = 1000$ for `forum` nodes, ensuring that the adjusted `forum` node IDs begin where the `person` IDs end.

This ID translation is applied consistently across the dataset: first, to each CSV file containing node data, and second, to each CSV file encoding edge data, by shifting both the source and destination vertex IDs according to their respective types. This preprocessing step allows us to merge all node and edge types into a unified graph structure without ambiguity, thereby enabling fair and collision-free insertion experiments across all supported relationships.

For each relation type, we measure the time required to insert both vertices and edges along with their associated properties. The resulting performance plots are shown in Figure 6.2, where the x-axis represents the number of edges (in millions) and the y-axis indicates the insertion time (in seconds).

Across all evaluated relationships, HAL achieves higher insertion throughput than LiveGraph. This is primarily due to two architectural differences between the systems. First, LiveGraph relies on Bloom filters for edge existence checks, which may return false positives and trigger linear scans of its Transactional Edge Log (TEL). In contrast, HAL maintains a secondary index implemented as a hash table (HT in Section 4.2), allowing constant-time edge existence checks. Second, LiveGraph assigns a single TEL block per source vertex, which leads to frequent resizing as adjacency lists grow. HAL avoids this overhead by distributing edges across multiple fixed-size STALB blocks, reducing the need for costly reallocations.

Lesson learned: HAL’s support for constant-time edge existence checks and multiple sorted adjacency list blocks enable efficient insertions. By separating property storage from the topology and indexing each property individually, HAL maintains a lightweight structure optimized for analytical scans. Despite relying on external storage for vertex and edge properties, HAL consistently outperforms LiveGraph across all evaluated SNB relationships.

6.5 Interactive GUI

For the demonstration of the HAL system [2], we developed an interactive web-based GUI using HTML, CSS, and JavaScript, leveraging the Cytoscape.js library for real-time graph visualization. The GUI consists of a single main HTML file that embeds both styling and JavaScript logic. On the backend, the system is implemented in a single main.cpp file using the Crow C++ web framework to serve WebSocket and REST endpoints. This backend interacts directly with the HAL graph store and processes incoming dynamic updates. The full system includes six core C++ classes from the HAL engine. Regarding the web interface, generative AI tools (specifically ChatGPT) were used during development to assist with the frontend syntax, generate the initial webpage template, and resolve debugging issues encountered during both frontend design and backend integration.

This interface enables users to observe the evolution of a dynamic graph as updates (in particular, ooo updates) arrive and are inserted in the HAL dynamic graph store. Thus, we simulate the emission of updates from several sources at various stream times S_T , and allow controlling, from the GUI, the order of their arrival at the database (or write time, W_T). This enables to experiment with in-order as well as ooo propagation of graph updates. The interface also enables evaluating queries over the graph state such as the HAL system maintains it, or, alternatively, without our special treatment of ooo updates (leading, in some cases, to erroneous answers).

HAL was demonstrated based on interactive benchmarks we built from known benchmarks, e.g., [50], as in [3]. For the demonstration, we start with a sequence of edge insertions and deletions as in [50], but allow users to *control when each update is received at the database*, in order for HAL to exercise its guessing of the most likely associate update, for each incoming one.

The interactive GUI has three main parts (see Figure 6.3):

- **Ground Truth Graph State:** At the top (see Figure 6.3a), we show the so-called ground truth graph state. This is an artefact we build exclusively for the demo, to show what the graph should be *if all the updates were emitted from, and received to, the same site, thus without any delays*. Of course, in practice, in a dynamic, distributed

graph setting, we do not know the ground truth.

- **Updates in Transit** (Figure 6.3b): In this area, we show updates emitted by streams, that are not yet received by the database server, and may be arbitrarily delayed. Users decide on the arrival order by *explicitly releasing updates from transit, in order*.
- **Updates in Database** (Figure 6.3c): This area shows updates in the order they were received in the database. Each update has a database time W_T and a stream time S_T . By default, the updates are ordered by database time, aligning with the approach used in traditional graph database systems. Alternatively, selecting the **order by stream time** option shows the storage order employed by HAL as shown in Figure 6.3d).

The state of the graph is shown for both configurations: ordered by database time (Figure 6.3c) and ordered by stream time (our system HAL, see Figure 6.3d). This allows users to visually compare the graph state across the two systems. To illustrate analytics, users can execute, e.g., shortest path queries on the current graph state, as shown in Figure 6.3c) and Figure 6.3d).

As an example, consider that three updates are emitted successively: the insertion of edge $1 \rightarrow 2$ at stream time 1, followed by its deletion at stream time 2, and the insertion of edge $2 \rightarrow 3$ at stream time 3. These updates are initially placed in the transit area, as shown in Figure 6.3b), whereas the correct graph state is shown in Figure 6.3a). Assume that the database receives these updates in an ooo manner: the deletion of $1 \rightarrow 2$, followed by the insertion of $1 \rightarrow 2$, and finally, the insertion of $2 \rightarrow 3$ at stream time 3. In our demonstration, the user can “make each update arrive” (Figure 6.3b) to the database by clicking the “In Transit” button next to that update.

Figure 6.3c) illustrates the updates stored in the database when ordered by database time. In this case, edge $1 \rightarrow 2$ is (incorrectly) part of the graph, as depicted by a dashed line. However, when the updates are ordered by stream time (our system HAL, Figure 6.3d), the correct graph state is maintained; here, the edge $1 \rightarrow 2$ does not exist. This ensures consistency with the ground truth graph shown in Figure 6.3a).

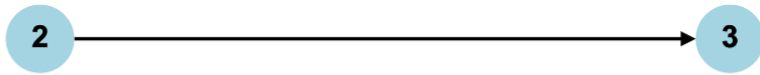
If a shortest path query is executed between nodes 1 and 3, traditional graph databases that order updates by database time (Figure 6.3c) incorrectly indicate that a path exists. However, comparing this with HAL (Figure 6.3d) and the ground truth graph (Figure 6.3a), demo attendees can notice that no such path exists.

6.6 Summary

This chapter introduces two major extensions to the HAL graph store. First, it enables historical queries that reconstruct graph states at any given stream time, while correctly handling ooo insertions and deletions. Second, it extends HAL’s topology-first model to support property-rich graphs with arbitrary edge and vertex properties, which are stored externally in RocksDB. A real-world interactive GUI allows users to visualize the impact of ooo updates on the query accuracy. Experimental results show that HAL significantly outperforms a baseline system on historical queries and achieves higher insertion throughput than LiveGraph on property graph workloads, particularly in the LDBC SNB setting. The next chapter concludes the thesis by summarizing key contributions and outlining future research directions.

a) **Graph Databases with Out-of-order Updates**

Ground Truth: Stream Graph State



Next Update

b) Updates In Transit

Stream Time	Update Type	Source	Target	Delay
1	Insert	1	2	In transit...
2	Delete	1	2	In transit...
3	Insert	2	3	In transit...

Updates In Database:

☒ Order by Database Time ☐ Order by Stream Time

Database Time	Stream Time	Update Type	Source	Target
1	2	Delete	1	2
2	1	Insert	1	2
3	3	Insert	2	3

c) Updates In Transit

Stream Time	Update Type	Source	Target	Delay
-------------	-------------	--------	--------	-------

Updates In Database:

☒ Order by Database Time ☐ Order by Stream Time

Database Time	Stream Time	Update Type	Source	Target
1	2	Delete	1	2
2	1	Insert	1	2
3	3	Insert	2	3

Graph State: Order by Database Time



Shortest Path Query

Source Node: 1

Destination Node: 3

Find Shortest Path

d) Updates In Transit

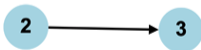
Stream Time	Update Type	Source	Target	Delay
-------------	-------------	--------	--------	-------

Updates In Database:

☐ Order by Database Time ☒ Order by Stream Time

Database Time	Stream Time	Update Type	Source	Target
2	1	Insert	1	2
1	2	Delete	1	2
3	3	Insert	2	3

Graph State: Order by Stream Time



Shortest Path Query

Source Node: 1

Destination Node: 3

Find Shortest Path

Figure 6.3: Screen captures of HAL's interactive GUI.

Chapter 7

Conclusion and perspectives

We summarize the thesis content in Section 7.1, and outline avenues for future work in Section 7.2.

7.1 Conclusion

In this thesis, we presented HAL, a novel in-memory native graph store designed to support scalable and consistent analytics over dynamic, multi-stream graph data—even in the presence of ooo updates. HAL addresses four core challenges in dynamic graph data management: *(i)* scalable ingestion with concurrent analytics; *(ii)* accurate analytics under ooo updates; *(iii)* efficient support for both present and historical graph snapshots; *(iv)* the decoupled handling of structural and property graph updates.

HAL introduces the Stream-Time Ordered Adjacency List (STAL), a novel data structure that maintains edge updates sorted by their original source (stream) time, regardless of arrival order. This is coupled with a lightweight, graph-aware multi-version concurrency control (MVCC) protocol. Our system is capable of processing approximately 7.5 million updates per second, outperforming state-of-the-art competitors like Sortledton by $2.5\times$, and achieves up to $357\times$ speedups in graph analytics tasks depending on workload characteristics.

We further extended HAL to support historical queries and property-rich graphs, aligning with the requirements of the LDBC SNB benchmark. These extensions en-

able HAL to store and manage node and edge properties efficiently. In our experiments, HAL demonstrated significantly higher ingestion throughput than LiveGraph on property graph workloads, confirming its capability to handle property-rich dynamic graphs effectively under high update rates.

7.2 Perspectives

Building on HAL's current features, several extensions can enhance its support for distributed execution, unified data storage, and simplified concurrency control. These include:

- **Distributed Sharding and Replication:** HAL's adjacency list structure naturally supports sharding by source vertex, enabling distributed execution with partitioned and replicated graph segments. This approach can scale HAL to large-scale deployments with high availability and fault tolerance.
- **Unified Storage for Topology and Properties:** Currently, HAL stores graph topology in memory and properties in RocksDB. This separation may complicate transactional coordination and degrade performance. A unified storage design co-locating topology and properties could reduce I/O overhead and simplify update handling.
- **Simplified Concurrency via RocksDB Integration:** Integrating HAL more tightly with RocksDB would allow us to leverage its native optimistic concurrency control (OCC) protocol, potentially replacing HAL's custom MVCC layer and simplifying transactional logic while preserving consistency.

Bibliography

- [1] Daniel J. Abadi, Donald Carney, Ugur Çetintemel, Mitch Cherniack, Christian Convey, Sangdon Lee, Michael Stonebraker, Nesime Tatbul, and Stanley B. Zdonik. Aurora: a new model and architecture for data stream management. *VLDB J.*, 12(2):120–139, 2003.
- [2] Angelos C. Anadiotis, Muhammad G. Khan, and Ioana Manolescu. Catching up with disorder: Dynamic graphs with out-of-order updates. In *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS '25*, page 15–18, New York, NY, USA, 2025. Association for Computing Machinery.
- [3] Angelos Christos Anadiotis, Muhammad Ghufra Khan, and Ioana Manolescu. Dynamic graph databases with out-of-order updates. *PVLDB*, 17(13), 2025.
- [4] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. Foundations of modern query languages for graph databases. *ACM Comput. Surv.*, 50(5), September 2017.
- [5] Renzo Angles, Peter Boncz, Josep Larriba-Pey, Irini Fundulaki, Thomas Neumann, Orri Erling, Peter Neubauer, Norbert Martinez-Bazan, Venelin Kotsev, and Ioan Toma. The linked data benchmark council: a graph and RDF industry benchmarking effort. *SIGMOD Rec.*, 43(1):27–31, May 2014.
- [6] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Alastair Green, Jan Hidders, Bei Li, Leonid Libkin, Victor Marsault, Wim Martens, Filip Murlak, Stefan Plantikow, Ognjen Savkovic, Michael Schmidt, Juan Sequeda, Slawek Staworko, Dominik Tomaszuk, Hannes Voigt, Domagoj Vrgoc, Mingxi

- Wu, and Dusan Zivkovic. Pg-schema: Schemas for property graphs. *Proc. ACM Manag. Data*, 1(2):198:1–198:25, 2023.
- [7] Renzo Angles, Angela Bonifati, Roberto García, and Domagoj Vrgoc. Path-based algebraic foundations of graph query languages. In Alkis Simitsis, Bettina Kemme, Anna Queralt, Oscar Romero, and Petar Jovanovic, editors, *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, pages 783–795. OpenProceedings.org, 2025.
- [8] ArangoDB Inc. Arangodb. Retrieved from <https://www.arangodb.com/docs/stable/data-models.html>, 2025. Accessed: Feb. 10, 2025.
- [9] Arvind Arasu, Shivnath Babu, and Jennifer Widom. The CQL continuous query language: semantic foundations and query execution. *VLDB J.*, 15(2):121–142, 2006.
- [10] Diego Arroyuelo, Adrián Gómez-Brandón, Aidan Hogan, Gonzalo Navarro, Juan L. Reutter, Javiel Rojas-Ledesma, and Adrián Soto. The ring: Worst-case optimal joins in graph databases using (almost) no extra space. *ACM Trans. Database Syst.*, 49(2):5:1–5:45, 2024.
- [11] Diego Arroyuelo, Adrián Gómez-Brandón, Aidan Hogan, Gonzalo Navarro, and Javiel Rojas-Ledesma. Optimizing RPQs over a compact graph representation. *VLDB Journal*, 33(2):349–374, 2024.
- [12] Diego Arroyuelo, Aidan Hogan, Gonzalo Navarro, and Javiel Rojas-Ledesma. Time- and space-efficient regular path queries on graphs, 2021.
- [13] A. Artale, D. Calvanese, R. Kontchakov, and M. Zakharyashev. The DL-Lite family and relations. *Journal of Artificial Intelligence Research*, 36, 2009.
- [14] Paolo Atzeni and Valeria De Antonellis. *Relational database theory*. Benjamin-Cummings Publishing Co., Inc., USA, 1993.
- [15] S. Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and

- Zachary G. Ives. Dbpedia: A nucleus for a web of open data. In *ISWC/ASWC*, 2007.
- [16] Sören Auer, Jens Lehmann, and Sebastian Hellmann. LinkedGeoData: Adding a spatial dimension to the web of data. In Abraham Bernstein, David R. Karger, Tom Heath, Lee Feigenbaum, Diana Maynard, Enrico Motta, and Krishnaprasad Thirunarayan, editors, *The Semantic Web - ISWC 2009*, pages 731–746, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.
- [17] Scott Beamer, Krste Asanovic, and David A. Patterson. The GAP benchmark suite. *CoRR*, abs/1508.03619, 2015.
- [18] Jon C. R. Bennett, Craig Partridge, and Nicholas Sheckman. Packet reordering is not pathological network behavior. *IEEE/ACM Transactions on Networks*, 7(6):789–798, 1999.
- [19] Maciej Besta, Robert Gerstenberger, Emanuel Peter, Marc Fischer, Michał Podstawski, Claude Barthels, Gustavo Alonso, and Torsten Hoefer. Demystifying graph databases: Analysis and taxonomy of data organization, system designs, and graph queries. *ACM Comput. Surv.*, 56(2), September 2023.
- [20] Angela Bonifati, Stefania Dumbrava, and Nicolas Mir. Hierarchical clustering for property graph schema discovery. In Julia Stoyanovich, Jens Teubner, Paolo Guagliardo, Milos Nikolic, Andreas Pieris, Jan Mühlig, Fatma Özcan, Sebastian Schelter, H. V. Jagadish, and Meihui Zhang, editors, *Proceedings of the 25th International Conference on Extending Database Technology, EDBT 2022, Edinburgh, UK, March 29 - April 1, 2022*, pages 2:449–2:453. OpenProceedings.org, 2022.
- [21] Achilles D. Boursianis, Maria S. Papadopoulou, Panagiotis D. Diamantoulakis, Aglaia Liopa-Tsakalidi, Pantelis Barouchas, George Salahas, George K. Karagiannidis, Shaohua Wan, and Sotirios K. Goudos. Internet of things (IoT) and agricultural unmanned aerial vehicles (UAVs) in smart farming: A comprehensive review. *Internet Things*, 18:100187, 2022.
- [22] Tim Bray. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 7159, March 2014.

- [23] Tim Bray, Jean Paoli, C. Michael Sperberg-McQueen, Eve Maler, and François Yergeau. Extensible markup language (xml) 1.0. <https://www.w3.org/TR/2008/REC-xml-20081126/>, 2008.
- [24] Andrey Brito, Christof Fetzer, Heiko Sturzrehm, and Pascal Felber. Speculative out-of-order event processing with software transaction memory. In Roberto Baldoni, editor, *Proceedings of the Second International Conference on Distributed Event-Based Systems, DEBS 2008, Rome, Italy, July 1-4, 2008*, volume 332 of *ACM International Conference Proceeding Series*, pages 265–275. ACM, 2008.
- [25] M. Buron, F. Goasdoué, I. Manolescu, and M.-L. Mugnier. Reformulation-based query answering for RDF graphs with RDFS ontologies. In *ESWC*, 2019.
- [26] Alison Callahan, José Cruz-Toledo, Peter Ansell, and Michel Dumontier. Bio2rdf release 2: Improved coverage, interoperability and provenance of life science linked data. In Philipp Cimiano, Oscar Corcho, Valentina Presutti, Laura Hollink, and Sebastian Rudolph, editors, *The Semantic Web: Semantics and Big Data*, pages 200–212, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [27] Callidus Software Inc. Orientdb. Retrieved from <https://orientdb.com>, 2025. Accessed: Feb. 10, 2025.
- [28] Badrish Chandramouli, Jonathan Goldstein, and David Maier. High-performance dynamic pattern matching over disordered streams. *Proc. VLDB Endow.*, 3(1):220–231, 2010.
- [29] Mengmeng Chang, Zhiming Ding, Zilin Zhao, and Zhi Cai. Heterogeneous modular traffic prediction based on multilayer graph convolutional network. *IEEE Trans. Intell. Transp. Syst.*, 25(7):7805–7817, 2024.
- [30] E. F. Codd. Relational database: A practical foundation for productivity. *Communications of the ACM*, 25(2):109–117, 1982.
- [31] Dario Colazzo and Carlo Sartiani. Typing regular path query languages for data graphs. In James Cheney and Thomas Neumann, editors, *Proceedings of the*

15th Symposium on Database Programming Languages, Pittsburgh, PA, USA, October 25-30, 2015, pages 69–78. ACM, 2015.

- [32] Charles D. Cranor, Theodore Johnson, Oliver Spatscheck, and Vladislav Shkapenyuk. Gigascope: A stream database for network applications. In Alon Y. Halevy, Zachary G. Ives, and AnHai Doan, editors, *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data, San Diego, California, USA, June 9-12, 2003*, pages 647–651. ACM, 2003.
- [33] Ali Davoudian, Liu Chen, and Mengchi Liu. A survey on NoSQL stores. *ACM Comput. Surv.*, 51(2), April 2018.
- [34] Dean De Leo. *Analysis of Dynamic Graphs on Sparse Arrays*. Phd-thesis - research and graduation internal, Vrije Universiteit Amsterdam, September 2022.
- [35] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Fred Zemke. Graph pattern matching in GQL and SQL/PGQ. In *Proceedings of the 2022 International Conference on Management of Data, SIGMOD '22*, page 2246–2258, New York, NY, USA, 2022. Association for Computing Machinery.
- [36] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Fred Zemke. Graph Pattern Matching in GQL and SQL/PGQ. In *Proceedings of the 2022 International Conference on Management of Data, SIGMOD '22*, page 2246–2258, New York, NY, USA, 2022. Association for Computing Machinery.
- [37] Guimin Dong, Mingyue Tang, Zhiyuan Wang, Jiechao Gao, Sikun Guo, Lihua Cai, Robert J. Gutierrez, Bradford Campbell, Laura E. Barnes, and Mehdi Boukhechba. Graph neural networks in IoT: A survey. *ACM Transactions on Sensor Networks*, 19(2):47:1–47:50, 2023.

- [38] David Ediger, Robert McColl, E. Jason Riedy, and David A. Bader. STINGER: high performance data structure for streaming graphs. In *IEEE Conference on High Performance Extreme Computing, HPEC 2012, Waltham, MA, USA, September 10-12, 2012*, pages 1–5. IEEE, 2012.
- [39] Thomas Eiter and Leonid Libkin, editors. *Database Theory - ICDT 2005, 10th International Conference, Edinburgh, UK, January 5-7, 2005, Proceedings*, volume 3363 of *Lecture Notes in Computer Science*. Springer, 2005.
- [40] Abdulhalim Fayad, Tibor Cinkler, and Jacek Rak. Toward 6G optical fronthaul: A survey on enabling technologies and research perspectives. *CoRR*, abs/2406.00308, 2024.
- [41] Per Fuchs, Jana Giceva, and Domagoj Margan. Sortledton: a universal, transactional graph data structure. *Proc. VLDB Endow.*, 15(6):1173–1186, 2022.
- [42] F. Goasdoué, I. Manolescu, and A. Roatis. Efficient query answering against dynamic RDF databases. In *EDBT/ICDT*, 2013.
- [43] Lukasz Golab and M. Tamer Özsu. Issues in data stream management. *SIGMOD Rec.*, 32(2):5–14, 2003.
- [44] Shengnan Guo, Youfang Lin, Huaiyu Wan, Xiucheng Li, and Gao Cong. Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting. *IEEE Trans. Knowl. Data Eng.*, 34(11):5415–5428, 2022.
- [45] Hassan Halawa and Matei Ripeanu. Position paper: bitemporal dynamic graph analytics. In Vasiliki Kalavri and Nikolay Yakovets, editors, *GRADES-NDA '21: Proceedings of the 4th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA), Virtual Event, China, 20 June 2021*, pages 7:1–7:12. ACM, 2021.
- [46] Joseph M. Hellerstein and Michael Stonebraker. *Readings in Database Systems*. MIT Press, Cambridge, MA, USA, 2005.
- [47] Florian Holzschuher and René Peinl. Performance of graph query languages:

- comparison of cypher, gremlin and native access in neo4j. In *EDBT, EDBT '13*, page 195–204, New York, NY, USA, 2013. Association for Computing Machinery.
- [48] Stratos Idreos, Martin L. Kersten, and Stefan Manegold. Database cracking. In *Third Biennial Conference on Innovative Data Systems Research, CIDR 2007, Asilomar, CA, USA, January 7-10, 2007, Online Proceedings*, pages 68–78. www.cidrdb.org, 2007.
- [49] Bitnine Global Inc. Agensgraph. <https://bitnine.net/agensgraph/>. Accessed: 2025-06-05.
- [50] Alexandru Iosup, Tim Hegeman, Wing Lung Ngai, Stijn Heldens, Arnau Prat-Pérez, Thomas Manhardt, Hassan Chafi, Mihai Capota, Narayanan Sundaram, Michael J. Anderson, Ilie Gabriel Tanase, Yinglong Xia, Lifeng Nai, and Peter A. Boncz. LDBC graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms. *Proc. VLDB Endow.*, 9(13):1317–1328, 2016.
- [51] Vinesh Kumar Jain, Arka Prokash Mazumdar, Parvez Faruki, and Mahesh Chandra Govil. Congestion control in internet of things: Classification, challenges, and future directions. *Sustain. Comput. Informatics Syst.*, 35:100678, 2022.
- [52] JanusGraph Authors. Janusgraph: Distributed graph database. <https://janusgraph.org/>, 2017. Accessed 24-April-2024.
- [53] Sadaf Javed, Ali Hassan, Rizwan Ahmad, Waqas Ahmed, Rehan Ahmed, Ahsan Saadat, and Mohsen Guizani. State-of-the-art and future research challenges in UAV swarms. *IEEE Internet of Things Journal*, 11(11):19023–19045, 2024.
- [54] Yuanzhen Ji, Hongjin Zhou, Zbigniew Jerzak, Anisoara Nica, Gregor Hackenbroich, and Christof Fetzer. Quality-driven processing of sliding window aggregates over out-of-order data streams. In Frank Eliassen and Roman Vitenberg, editors, *Proceedings of the 9th ACM International Conference on Distributed Event-Based Systems, DEBS '15, Oslo, Norway, June 29 - July 3, 2015*, pages 68–79. ACM, 2015.

- [55] Paul Juillard, Angela Bonifati, and Andrea Mauri. Interactive graph repairs for neighborhood constraints. In Letizia Tanca, Qiong Luo, Giuseppe Polese, Loredana Caruccio, Xavier Oriol, and Donatella Firmani, editors, *Proceedings 27th International Conference on Extending Database Technology, EDBT 2024, Paestum, Italy, March 25 - March 28*, pages 175–187. OpenProceedings.org, 2024.
- [56] Vojtech Kolomicenko. Analysis and experimental comparison of graph databases. Master’s Thesis, Charles University in Prague, 2013. Accessed: July 18, 2025.
- [57] Jochem Kuijpers, George Fletcher, Tobias Lindaaker, and Nikolay Yakovets. Path indexing in the cypher query pipeline. In Yannis Velegrakis, Demetris Zeinalipour-Yazti, Panos K. Chrysanthis, and Francesco Guerra, editors, *Proceedings of the 24th International Conference on Extending Database Technology, EDBT 2021, Nicosia, Cyprus, March 23 - 26, 2021*, pages 582–587. OpenProceedings.org, 2021.
- [58] Pradeep Kumar and H. Howie Huang. Graphone: A data store for real-time analytics on evolving graphs. *ACM Trans. Storage*, 15(4):29:1–29:40, 2020.
- [59] Jérôme Kunegis. KONECT: the koblenz network collection. In Leslie Carr, Alberto H. F. Laender, Bernadette Farias Lóscio, Irwin King, Marcus Fontoura, Denny Vrandečić, Lora Aroyo, José Palazzo M. de Oliveira, Fernanda Lima, and Erik Wilde, editors, *22nd International World Wide Web Conference, WWW ’13, Rio de Janeiro, Brazil, May 13-17, 2013, Companion Volume*, pages 1343–1350. International World Wide Web Conferences Steering Committee / ACM, 2013.
- [60] Redis Labs. Redisgraph. <https://oss.redis.com/redisgraph/>. Accessed: 2025-06-05.
- [61] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM*, 21(7):558–565, 1978.
- [62] Hoang D. Le and Anh T. Pham. Link-layer retransmission-based error-control

- protocols in FSO communications: A survey. *IEEE Commun. Surv. Tutorials*, 24(3):1602–1633, 2022.
- [63] Viktor Leis, Alfons Kemper, and Thomas Neumann. The adaptive radix tree: Artful indexing for main-memory databases. In Christian S. Jensen, Christopher M. Jermaine, and Xiaofang Zhou, editors, *29th IEEE International Conference on Data Engineering, ICDE 2013, Brisbane, Australia, April 8-12, 2013*, pages 38–49. IEEE Computer Society, 2013.
- [64] Dean De Leo and Peter A. Boncz. Teseo and the analysis of structural dynamic graphs. *Proc. VLDB Endow.*, 14(6):1053–1066, 2021.
- [65] Aljoscha P. Lepping, Hoang Mi Pham, Laura Mons, Balint Rueb, Philipp M. Grulich, Ankit Chaudhary, Steffen Zeuch, and Volker Markl. Showcasing data management challenges for future IoT applications with NebulaStream. *Proc. VLDB Endow.*, 16(12):3930–3933, 2023.
- [66] Hourun Li, Yusheng Zhao, Zhengyang Mao, Yifang Qin, Zhiping Xiao, Jiaqi Feng, Yiyang Gu, Wei Ju, Xiao Luo, and Ming Zhang. A survey on graph neural networks in intelligent transportation systems. *CoRR*, abs/2401.00713, 2024.
- [67] Jin Li, David Maier, Kristin Tufte, Vassilis Papadimos, and Peter A. Tucker. Semantics and evaluation techniques for window aggregates in data streams. In Fatma Özcan, editor, *Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14-16, 2005*, pages 311–322. ACM, 2005.
- [68] Jin Li, Kristin Tufte, Vladislav Shkapenyuk, Vassilis Papadimos, Theodore Johnson, and David Maier. Out-of-order processing: a new architecture for high-performance stream systems. *Proc. VLDB Endow.*, 1(1):274–288, 2008.
- [69] Ming Li, Mo Liu, Luping Ding, Elke A. Rundensteiner, and Murali Mani. Event stream processing with out-of-order data arrival. In *27th International Conference on Distributed Computing Systems Workshops (ICDCS 2007 Workshops)*, June 25-29, 2007, Toronto, Ontario, Canada, page 67. IEEE Computer Society, 2007.

- [70] Rui Li, Fan Zhang, Tong Li, Ning Zhang, and Tingting Zhang. DMGAN: dynamic multi-hop graph attention network for traffic forecasting. *IEEE Trans. Knowl. Data Eng.*, 35(9):9088–9101, 2023.
- [71] Zhaoyi Li, Jiawei Huang, Shiqi Wang, and Jianxin Wang. Achieving low latency for multipath transmission in RDMA based data center network. *IEEE Trans. Cloud Comput.*, 12(1):337–346, 2024.
- [72] Matteo Lissandrini, Davide Mottin, Themis Palpanas, and Yannis Velegrakis. Graph-query suggestions for knowledge graph exploration. In Yennun Huang, Irwin King, Tie-Yan Liu, and Maarten van Steen, editors, *WWW '20: The Web Conference 2020, Taipei, Taiwan, April 20-24, 2020*, pages 2549–2555. ACM / IW3C2, 2020.
- [73] Dapeng Liu and Shaochun Xu. Comparison of hash table performance with open addressing and closed addressing: An empirical study. *Int. J. Networked Distributed Comput.*, 3(1):60–68, 2015.
- [74] Memgraph Ltd. Memgraph. <https://memgraph.com/>. Accessed: 2025-06-05.
- [75] Guohan Lu, Yan Chen, Stefan Birrer, Fabián E. Bustamante, and Xing Li. POPI: a user-level tool for inferring router packet forwarding priority. *IEEE/ACM Trans. Netw.*, 18(1):1–14, 2010.
- [76] Peter Macko, Virendra J. Marathe, Daniel W. Margo, and Margo I. Seltzer. LLAMA: efficient graph analytics using large multiversioned arrays. In Johannes Gehrke, Wolfgang Lehner, Kyuseok Shim, Sang Kyun Cha, and Guy M. Lohman, editors, *31st IEEE International Conference on Data Engineering, ICDE 2015, Seoul, South Korea, April 13-17, 2015*, pages 363–374. IEEE Computer Society, 2015.
- [77] Guido Maier, Antonino Albanese, Michele Ciavotta, Nicola Ciulli, Stefano Giordano, Elisa Giusti, Alfredo Salvatore, and Giovanni Schembra. WatchEDGE: Smart networking for distributed ai-based environmental control. *Computer Networks*, 243:110248, 2024.

- [78] Norbert Martínez-Bazan, M. Ángel Águila Lorente, Victor Muntés-Mulero, David Dominguez-Sal, Sergio Gómez-Villamor, and Josep-L. Larriba-Pey. Efficient graph management based on bitmap indices. In *Proceedings of the 16th International Database Engineering & Applications Symposium, IDEAS'12*, page 110–119, New York, NY, USA, 2012. Association for Computing Machinery.
- [79] Kristyn J. Maschhoff, Robert Vesse, et al. Quantifying performance of cge: A unified scalable pattern mining and search system. In CUG, 2017.
- [80] Robert Campbell McColl, David Ediger, Jason Poovey, Dan Campbell, and David A. Bader. A performance evaluation of open source graph databases. In *Proceedings of the First Workshop on Parallel Programming for Analytics Applications, PPAA '14*, page 11–18, New York, NY, USA, 2014. Association for Computing Machinery.
- [81] Meta Open Source. RocksDB: A persistent key-value store for fast storage. <https://rocksdb.org/>.
- [82] Christopher Mutschler and Michael Philippsen. Distributed low-latency out-of-order event processing for high data rate sensor streams. In *27th IEEE International Symposium on Parallel and Distributed Processing, IPDPS 2013, Cambridge, MA, USA, May 20-24, 2013*, pages 1133–1144. IEEE Computer Society, 2013.
- [83] Christopher Mutschler and Michael Philippsen. Reliable speculative processing of out-of-order event streams in generic publish/subscribe middlewares. In Sharma Chakravarthy, Susan Darling Urban, Peter R. Pietzuch, and Elke A. Rundensteiner, editors, *The 7th ACM International Conference on Distributed Event-Based Systems, DEBS '13, Arlington, TX, USA - June 29 - July 03, 2013*, pages 147–158. ACM, 2013.
- [84] Neo4j. Understanding neo4j's data on disk. <https://neo4j.com/developer/kb/understanding-data-on-disk/>, n.d. Accessed: 2025-07-02.
- [85] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. Fast serializable multi-version concurrency control for main-memory database systems. In

- Timos K. Sellis, Susan B. Davidson, and Zachary G. Ives, editors, *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, pages 677–689. ACM, 2015.
- [86] Oracle. Oracle spatial and graph. Retrieved from <https://www.oracle.com/database/technologies/spatialandgraph.html>, 2025. Accessed: Feb. 10, 2025.
- [87] Oracle. Oracle spatial and graph, n.d. Accessed: July 18, 2025.
- [88] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. The log-structured merge-tree (LSM-tree). *Acta Inf.*, 33(4):351–385, June 1996.
- [89] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Technical report, Stanford Digital Library Technologies Project, 1998.
- [90] Marcus Paradies. Graph pattern matching in sap hana. <https://tinyurl.com/ycxu54pr>, 2017. Presented at the First openCypher Implementers Meeting, February 2017.
- [91] A. Pavan, Kanat Tangwongsan, Srikanta Tirthapura, and Kun-Lung Wu. Counting and sampling triangles from a graph stream. *Proc. VLDB Endow.*, 6(14):1870–1881, September 2013.
- [92] Thomas Pellissier Tanon, Gerhard Weikum, and Fabian Suchanek. Yago 4: A reasonable knowledge base. In *The Semantic Web: 17th International Conference, ESWC 2020, Heraklion, Crete, Greece, May 31–June 4, 2020, Proceedings*, page 583–596, Berlin, Heidelberg, 2020. Springer-Verlag.
- [93] Kenneth Platz, Neeraj Mittal, and S. Venkatesan. Concurrent unrolled skiplist. In *39th IEEE International Conference on Distributed Computing Systems, ICDCS 2019, Dallas, TX, USA, July 7-10, 2019*, pages 1579–1589. IEEE, 2019.
- [94] Yann Ramusat, Silviu Maniu, and Pierre Senellart. Provenance-based algorithms for rich queries over graph databases. In Yannis Velegrakis, Demetris

- Zeinalipour-Yazti, Panos K. Chrysanthis, and Francesco Guerra, editors, *Proceedings of the 24th International Conference on Extending Database Technology, EDBT 2021, Nicosia, Cyprus, March 23 - 26, 2021*, pages 73–84. OpenProceedings.org, 2021.
- [95] Yann Ramusat, Silviu Maniu, and Pierre Senellart. Efficient provenance-aware querying of graph databases with datalog. In Vasiliki Kalavri and Semih Salihoglu, editors, *GRADES-NDA '22: Proceedings of the 5th ACM SIGMOD Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA), Philadelphia, Pennsylvania, USA, 12 June 2022*, pages 4:1–4:9. ACM, 2022.
- [96] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Database Internals*, pages 149–170. O'Reilly Media, 2015.
- [97] Arnau Rovira-Sugranes, Abolfazl Razi, Fatemeh Afghah, and Jacob Chakareski. A review of AI-enabled routing protocols for UAV networks: Trends, challenges, and future outlook. *Ad Hoc Networks*, 130:102790, 2022.
- [98] Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.
- [99] Sherif Sakr, Angela Bonifati, Hannes Voigt, Alexandru Iosup, Khaled Ammar, Renzo Angles, Walid Aref, Marcelo Arenas, Maciej Besta, Peter A. Boncz, Khuzaima Daudjee, Emanuele Della Valle, Stefania Dumbrava, Olaf Hartig, Bernhard Haslhofer, Tim Hegeman, Jan Hidders, Katja Hose, Adriana Iamnitchi, Vasiliki Kalavri, Hugo Kapp, Wim Martens, M. Tamer Özsu, Eric Peukert, Stefan Plankow, Mohamed Ragab, Matei R. Ripeanu, Semih Salihoglu, Christian Schulz, Petra Selmer, Juan F. Sequeda, Joshua Shnavier, Gábor Szárnyas, Riccardo Tommasini, Antonino Tumeo, Alexandru Uta, Ana Lucia Varbanescu, Hsiang-Yun Wu, Nikolay Yakovets, Da Yan, and Eiko Yoneki. The future is big graphs: a community view on graph processing systems. *Commun. ACM*, 64(9):62–71, August 2021.
- [100] Wei Sang, Huiliang Zhang, Xianchang Kang, Ping Nie, Xin Meng, Benoit Boulet,

- and Pei Sun. Dynamic multi-granularity spatial-temporal graph attention network for traffic forecasting. *Inf. Sci.*, 662:120230, 2024.
- [101] Bin Shao, Haixun Wang, and Yatao Li. Trinity: a distributed graph engine on a memory cloud. In *SIGMOD*, SIGMOD '13, page 505–516, New York, NY, USA, 2013. Association for Computing Machinery.
- [102] Jifan Shi, Biao Wang, and Yun Xu. Spruce: a fast yet space-saving structure for dynamic graph storage. *Proc. ACM Manag. Data*, 2(1):27:1–27:26, 2024.
- [103] Larissa Capobianco Shimomura, Nikolay Yakovets, and George Fletcher. Reasoning on property graphs with graph generating dependencies. *Inf. Sci.*, 672:120675, 2024.
- [104] Meta Open Source. RocksDB memtable: In-memory data structures (skip list). <https://github.com/facebook/rocksdb/wiki/MemTable>, 2021. Accessed: 2025-05-29.
- [105] Meta Open Source. RocksDB optimistic concurrency control protocol. <https://github.com/facebook/rocksdb/wiki/Transactions>, 2023. Accessed: 2025-05-29.
- [106] Utkarsh Srivastava and Jennifer Widom. Flexible time management in data stream systems. In Catriel Beerli and Alin Deutsch, editors, *Proceedings of the Twenty-third ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, June 14-16, 2004, Paris, France*, pages 263–274. ACM, 2004.
- [107] Srikanta Tirthapura and David P. Woodruff. A general method for estimating correlated aggregates over a data stream. In Anastasios Kementsietsidis and Marcos Antonio Vaz Salles, editors, *IEEE 28th International Conference on Data Engineering (ICDE 2012), Washington, DC, USA (Arlington, Virginia), 1-5 April, 2012*, pages 162–173. IEEE Computer Society, 2012.
- [108] Aparna Vaikuntam and Vinodh Kumar Perumal. Evaluation of contemporary graph databases. In *Proceedings of the 7th ACM India Computing Conference, COMPUTE '14, New York, NY, USA, 2014*. Association for Computing Machinery.

- [109] Oskar van Rest, Sungpack Hong, Jinha Kim, Xuming Meng, and Hassan Chafi. PGQL: a property graph query language. In *Proceedings of the Fourth International Workshop on Graph Data Management Experiences and Systems, GRADES '16*, New York, NY, USA, 2016. Association for Computing Machinery.
- [110] VelocityDB Inc. Velocitygraph. Retrieved from <https://velocitydb.com/QuickStartVelocityGraph>, 2019. Accessed: Feb. 10, 2025.
- [111] VelocityDB Inc. Velocitygraph. Retrieved from <https://velocitydb.com>, 2019. Accessed: Feb. 10, 2025.
- [112] Denny Vrandečić and Markus Krötzsch. Wikidata: a free collaborative knowledgebase. *Commun. ACM*, 57(10):78–85, September 2014.
- [113] W3C. SPARQL query language for RDF. Online, 2008.
- [114] W3C. SPARQL 1.1 overview. Online, 2013.
- [115] World Wide Web Consortium (W3C). Resource description framework (rdf), 2004. Accessed: February 11, 2025.
- [116] RDF Schema. <https://www.w3.org/TR/rdf12-schema/>.
- [117] Haijun Wang, Haitao Zhao, Jiao Zhang, Dongtang Ma, Jiaxun Li, and Jibo Wei. Survey on unmanned aerial vehicle networks: A cyber physical system perspective. *IEEE Commun. Surv. Tutorials*, 22(2):1027–1070, 2020.
- [118] Gerhard Weikum, Xin Luna Dong, Simon Razniewski, and Fabian Suchanek. *Machine Knowledge: Creation and Curation of Comprehensive Knowledge Bases*. Number 2-4 in Foundations and Trends in Databases. NOW publishers, 2021.
- [119] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. An empirical evaluation of in-memory multi-version concurrency control. *Proc. VLDB Endow.*, 10(7):781–792, 2017.
- [120] Gremlin: Query your graph by traversal. <https://www.puppygraph.com/blog/gremlin-graph>, 2025. Accessed July 2025.

- [121] Rdf 1.1 concepts and abstract syntax. <https://www.w3.org/TR/rdf11-concepts/>, 2014. W3C Recommendation, 25 February 2014.
- [122] Resource description framework (rdf) model and syntax specification. <https://www.w3.org/TR/1999/REC-rdf-syntax-19990222/>, 1999. W3C Recommendation, 22 February 1999.
- [123] Titan. <https://github.com/thinkaurelius/titan/wiki>. Accessed: 2025-07-18.
- [124] Wenjun Yang, Lin Cai, Shengjie Shu, Jianping Pan, and Amir Sepahi. MAMS: mobility-aware multipath scheduler for MPQUIC. *IEEE/ACM Trans. Netw.*, 32(4):3237–3252, 2024.
- [125] Mingyang Zhang, Tong Li, Yue Yu, Yong Li, Pan Hui, and Yu Zheng. Urban anomaly analytics: Description, detection, and prediction. *IEEE Transactions on Big Data*, 8(3):809–826, 2022.
- [126] Xiaowei Zhu, Marco Serafini, Xiaosong Ma, Ashraf Aboulnaga, Wenguang Chen, and Guanyu Feng. Livegraph: A transactional graph storage system with purely sequential adjacency list scans. *Proc. VLDB Endow.*, 13(7):1020–1034, 2020.

Titre: Interrogation efficace de graphes dynamiques provenant de sources multiples

Mots clés: bases de données graphes, flots de données, retards de transmission

Résumé: De nombreuses applications en temps réel, telles que les plateformes financières, les réseaux sociaux et les systèmes de transport, reposent sur des graphes dynamiques construits à partir de multiples flux de données. Les mises à jour en provenance de ces flux peuvent être reçues avec des retards, rendant difficile l'ingestion correcte ainsi que l'interrogation simultanée. Cette thèse présente **HAL**, un *moteur de base de données de graphes dynamiques en mémoire* conçu pour relever ces défis grâce à la structure *Stream-Time Ordered Adjacency List (STAL)*, qui maintient les mises à jour selon l'ordre de leur émission, indépendamment de

leur ordre d'arrivée. Un protocole efficace de contrôle de concurrence multi-version garantit la cohérence des traitements sur le graphe en concurrence avec l'ingestion. HAL atteint un taux d'ingestion de **7,5 millions de mises à jour par seconde**, dépassant les meilleurs systèmes existants d'un facteur de **2,5×**, et offre jusqu'à **357×** d'accélération pour les requêtes analytiques en présence de mises à jour retardées. La thèse détaille la conception du cœur du système HAL, son évaluation sous diverses charges de travail, ainsi que ses extensions pour les requêtes historiques et la gestion de graphes riches en propriétés.

Title: Scalable Analytics on Multi-Streams Dynamic Graphs

Keywords: graph databases, data streams, out-of-order transmission

Abstract: Many real-time applications, such as financial platforms, social networks, and transportation systems, rely on dynamic graphs built from multiple data streams. Handling out-of-order updates while ensuring scalable ingestion, accurate analytics, and efficient storage remains a major challenge. This thesis presents **HAL**, an *in-memory dynamic graph database engine* that addresses these issues through the *Stream-Time Ordered Adjacency List (STAL)* structure, which maintains updates in stream-time order

regardless of arrival sequence. A lightweight, graph-aware multi-version concurrency control protocol guarantees consistency during concurrent ingestion and analytics. HAL achieves up to **7.5 million updates per second**, outperforming existing systems by **2.5×**, and delivers up to **357×** faster analytical queries under out-of-order updates. The thesis details HAL's core design, evaluation under diverse workloads, and its extensions for historical queries and property-rich graph support.